

BÁO CÁO CHUYÊN ĐỀ

HỌC PHẦN: CHUYÊN ĐỀ CÔNG NGHỆ PHẦN MỀM

Chuyên đề: Các thuật toán đối sánh mẫu

(Bản nháp)

Ngày 30 tháng 5 năm 2026

MỤC LỤC

CHƯƠNG 1: TỔNG QUAN CHUYÊN ĐỀ	6
1.1 Giới thiệu về đối sánh mẫu	6
1.2 Các khái niệm cơ bản trong đối sánh mẫu	6
1.3 Các hướng giải quyết bài toán đối sánh mẫu	6
1.4 Cơ chế hoạt động của bài toán đối sánh mẫu.....	7
1.4.1 Cơ chế cửa sổ trượt.....	7
1.4.2 Cơ chế Attempt và Shift.....	7
1.5 Một số thuật toán đối sánh mẫu tiêu biểu	7
1.6 Các hướng tìm kiếm mẫu.....	8
1.6.1 Tìm kiếm mẫu từ trái sang phải	8
1.6.2 Tìm kiếm mẫu từ phải sang trái	8
1.6.3 Tìm kiếm mẫu từ vị trí xác định	9
1.6.4 Tìm kiếm mẫu từ vị trí bất kỳ	9
CHƯƠNG 2: CÁC THUẬT TOÁN TÌM KIẾM MẪU TỪ TRÁI SANG PHẢI.....	10
2.1 Thuật toán Not So Naive.....	10
2.1.1 Giới thiệu thuật toán.....	10
2.1.2 Quy trình thực hiện.....	10
2.1.3 Quy tắc dịch cửa sổ	10
2.1.4 Đánh giá độ phức tạp thuật toán.....	11
2.1.5 Kiểm nghiệm thuật toán.....	11
2.1.6 Cài đặt thuật toán với C++.....	12
2.2 Thuật toán Karp-Rabin.....	13
2.2.1 Giới thiệu thuật toán.....	13
2.2.2 Quy trình thực hiện thuật toán	13
2.2.3 Đánh giá độ phức tạp thuật toán.....	14
2.2.4 Kiểm nghiệm thuật toán.....	14
2.2.5 Cài đặt thuật toán với C++.....	15
2.3 Thuật toán Shift-Or.....	16

2.3.1	Giới thiệu thuật toán.....	16
2.3.2	Quy trình thực hiện thuật toán	16
2.3.3	Đánh giá độ phức tạp thuật toán.....	17
2.3.4	Kiểm nghiệm thuật toán.....	17
2.3.5	Cài đặt thuật toán với C++	18
2.4	Thuật toán Morris-Pratt.....	19
2.4.1	Giới thiệu thuật toán.....	19
2.4.2	Quy trình thực hiện thuật toán	19
2.4.3	Đánh giá độ phức tạp thuật toán.....	20
2.4.4	Kiểm nghiệm thuật toán.....	20
2.4.5	Cài đặt thuật toán với C++	21
2.5	Thuật toán Knuth-Morris-Pratt.....	23
2.5.1	Giới thiệu thuật toán.....	23
2.5.2	Quy trình thực hiện thuật toán	23
2.5.3	Đánh giá độ phức tạp thuật toán.....	24
2.5.4	Kiểm nghiệm thuật toán.....	25
2.5.5	Cài đặt thuật toán với C++	26
2.6	Thuật toán Apostolico-Crochemore.....	28
2.6.1	Giới thiệu thuật toán.....	28
2.6.2	Quy trình thực hiện thuật toán	28
2.6.3	Đánh giá độ phức tạp thuật toán.....	30
2.6.4	Kiểm nghiệm thuật toán.....	30
2.6.5	Cài đặt thuật toán với C++	31
CHƯƠNG 3: CÁC THUẬT TOÁN TÌM KIẾM MẪU TỪ PHẢI SANG TRÁI.....		33
3.1	Thuật toán Apostolico-Giancarlo.....	33
3.1.1	Giới thiệu thuật toán.....	33
3.1.2	Các thành phần của thuật toán Boyer-Moore được sử dụng	33
3.1.3	Quy trình thực hiện thuật toán	34
3.1.4	Đánh giá độ phức tạp thuật toán.....	36

3.1.5	Kiểm nghiệm thuật toán.....	37
3.1.6	Cài đặt thuật toán với C++.....	38
3.2	Thuật toán Turbo BM.....	42
3.2.1	Giới thiệu thuật toán.....	42
3.2.2	Quy trình thực hiện thuật toán	42
3.2.3	Đánh giá độ phức tạp thuật toán.....	43
3.2.4	Kiểm nghiệm thuật toán.....	44
3.2.5	Cài đặt thuật toán với C++.....	46
3.3	Thuật toán Zhu-Takaoka.....	47
3.3.1	Giới thiệu thuật toán.....	47
3.3.2	Quy trình thực hiện thuật toán	47
3.3.3	Đánh giá độ phức tạp thuật toán.....	48
3.3.4	Kiểm nghiệm thuật toán.....	49
3.3.5	Cài đặt thuật toán bằng C++	50
3.4	Thuật toán Berry-Ravindran	53
3.4.1	Giới thiệu thuật toán.....	53
3.4.2	Quy trình thực hiện thuật toán	53
3.4.3	Đánh giá độ phức tạp thuật toán.....	54
3.4.4	Kiểm nghiệm thuật toán.....	55
3.4.5	Cài đặt thuật toán bằng C++	56
CHƯƠNG 4: CÁC THUẬT TOÁN TÌM KIẾM MẪU TỪ VỊ TRÍ XÁC ĐỊNH		57
4.1	Thuật toán Two-Way.....	57
4.1.1	Giới thiệu thuật toán.....	57
4.1.2	Định nghĩa toán học của thuật toán Two-way	57
4.1.3	Quy trình thực hiện thuật toán	58
4.1.4	Đánh giá độ phức tạp thuật toán.....	59
4.1.5	Kiểm nghiệm thuật toán.....	59
4.1.6	Cài đặt thuật toán bằng C++	60
4.2	Thuật toán Optimal Mismatch	64

4.2.1	Giới thiệu thuật toán.....	64
4.2.2	Quy trình thực hiện thuật toán	64
4.2.3	Đánh giá độ phức tạp thuật toán.....	66
4.2.4	Kiểm nghiệm thuật toán.....	67
4.2.5	Cài đặt thuật toán bằng C++	68
4.3	Thuật toán Maximal Shift	71
4.3.1	Giới thiệu thuật toán.....	71
4.3.2	Quy trình thực hiện thuật toán	71
4.3.3	Đánh giá độ phức tạp thuật toán.....	73
4.3.4	Kiểm nghiệm thuật toán.....	73
4.3.5	Kiểm chứng nâng cao trên chuỗi mẫu dài	75
4.3.6	Cài đặt thuật toán bằng C++	76
4.4	Thuật toán Skip Search	79
4.4.1	Giới thiệu thuật toán.....	79
4.4.2	Quy trình thực hiện thuật toán	79
4.4.3	Đánh giá độ phức tạp thuật toán.....	80
4.4.4	Kiểm nghiệm thuật toán.....	81
4.4.5	Cài đặt thuật toán bằng C++	82
4.5	Thuật toán KMP Skip Search.....	83
4.5.1	Giới thiệu thuật toán.....	83
4.5.2	Quy trình thực hiện thuật toán	83
4.5.3	Đánh giá độ phức tạp thuật toán.....	85
4.5.4	Kiểm nghiệm thuật toán.....	86
4.5.5	Cài đặt thuật toán bằng C++	87
CHƯƠNG 5: CÁC THUẬT TOÁN TÌM KIẾM MẪU TỪ VỊ TRÍ BẤT KỲ		90
5.1	Thuật toán Horspool.....	90
5.1.1	Giới thiệu thuật toán.....	90
5.1.2	Quy trình thực hiện thuật toán	90
5.1.3	Đánh giá độ phức tạp thuật toán.....	91

5.1.4	Kiểm nghiệm thuật toán.....	92
5.1.5	Cài đặt thuật toán với C++.....	93
5.2	Thuật toán Quick Search.....	94
5.2.1	Giới thiệu thuật toán.....	94
5.2.2	Quy trình thực hiện thuật toán	94
5.2.3	Đánh giá độ phức tạp thuật toán.....	95
5.2.4	Kiểm nghiệm thuật toán.....	96
5.2.5	Cài đặt thuật toán với C++.....	96
5.3	Thuật toán Tuned Boyer-Moore.....	98
5.3.1	Giới thiệu thuật toán.....	98
5.3.2	Quy trình thực hiện thuật toán	98
5.3.3	Đánh giá độ phức tạp thuật toán.....	99
5.3.4	Kiểm nghiệm thuật toán.....	100
5.3.5	Cài đặt thuật toán bằng C++	101
5.4	Thuật toán Smith.....	103
5.4.1	Giới thiệu thuật toán.....	103
5.4.2	Quy trình thực hiện thuật toán	103
5.4.3	Quá trình so khớp và tìm kiếm.....	103
5.4.4	Đánh giá độ phức tạp thuật toán.....	104
5.4.5	Kiểm nghiệm thuật toán.....	104
5.4.6	Cài đặt thuật toán với C++.....	105
5.5	Thuật toán Raita.....	107
5.5.1	Giới thiệu thuật toán.....	107
5.5.2	Quy trình thực hiện thuật toán	107
5.5.3	Đánh giá độ phức tạp thuật toán.....	108
5.5.4	Kiểm nghiệm thuật toán.....	109
5.5.5	Cài đặt thuật toán bằng C++	109
DANH MỤC THAM KHẢO		111

CHƯƠNG 1: TỔNG QUAN CHUYÊN ĐỀ

1.1 Giới thiệu về đối sánh mẫu

Đối sánh mẫu (pattern matching) là một chủ đề rất quan trọng trong lĩnh vực xử lý văn bản. Các thuật toán đối sánh mẫu là những thành phần cơ bản được sử dụng trong nhiều hệ thống phần mềm thực tiễn và các hệ điều hành hiện nay. Ngoài ra, chúng còn thể hiện nhiều phương pháp lập trình có ảnh hưởng lớn đến các lĩnh vực khác của khoa học máy tính như thiết kế hệ thống và phần mềm.

Bên cạnh vai trò thực tiễn, đối sánh mẫu cũng giữ vị trí quan trọng trong khoa học máy tính lý thuyết nhờ cung cấp nhiều bài toán mang tính thử thách cao.

Mặc dù dữ liệu hiện nay được lưu trữ dưới nhiều hình thức khác nhau, văn bản vẫn là phương tiện trao đổi thông tin chủ yếu. Điều này đặc biệt rõ ràng trong: ngôn ngữ học, văn học, xử lý dữ liệu, sinh học phân tử. Trong các lĩnh vực này, dữ liệu thường xuất hiện dưới dạng chuỗi ký tự rất lớn như: từ điển, kho ngữ liệu, chuỗi nucleotide, chuỗi amino acid. Khối lượng dữ liệu ngày càng tăng nhanh khiến yêu cầu về hiệu năng của các thuật toán xử lý chuỗi trở nên vô cùng quan trọng.

1.2 Các khái niệm cơ bản trong đối sánh mẫu

Đối sánh mẫu là bài toán tìm một hoặc tất cả các lần xuất hiện của một chuỗi ký tự (gọi là mẫu, pattern) trong một văn bản (text).

Mẫu được ký hiệu bởi: $x = x[0 \dots m - 1]$, với độ dài bằng m .

Văn bản được ký hiệu bởi: $y = y[0 \dots n - 1]$, với độ dài bằng n .

Cả hai chuỗi đều được xây dựng từ một tập hữu hạn các ký tự gọi là một bảng chữ cái (*alphabet*), ký hiệu bởi Σ , có kích thước bằng σ .

1.3 Các hướng giải quyết bài toán đối sánh mẫu

Tùy thuộc vào dữ liệu nào được cho trước, bài toán đối sánh mẫu có thể được giải theo hai hướng:

(1) Tiền xử lý mẫu: sử dụng automata (là mô hình toán học dùng để mô tả cách một hệ thống xử lý chuỗi ký tự theo từng trạng thái), hoặc sử dụng tính chất tổ hợp của chuỗi.

(2) Tiền xử lý văn bản sử dụng cấu trúc chỉ mục, cây, automata.

Trong thực tế, nhiều thuật toán tập trung vào hướng tiền xử lý mẫu để tăng tốc quá trình tìm kiếm.

1.4 Cơ chế hoạt động của bài toán đối sánh mẫu

Các thuật toán đối sánh mẫu thường hoạt động bằng cách sử dụng một cửa sổ (*window*) có kích thước bằng độ dài mẫu m .

1.4.1 Cơ chế cửa sổ trượt

Quy trình thực hiện gồm các bước:

- (1) Đặt cửa sổ lên đầu văn bản.
- (2) So sánh các ký tự trong cửa sổ với mẫu. Nếu khớp hoàn toàn thì ghi nhận kết quả.
- (3) Dịch cửa sổ sang phải.
- (4) Lặp lại cho đến khi cửa sổ vượt quá cuối văn bản.

1.4.2 Cơ chế Attempt và Shift

Mỗi lần thử so sánh được gọi là attempt.

Sau mỗi lần thử, cửa sổ được: shift (*dịch*) sang phải. Nếu cửa sổ đang nằm trên đoạn: $y[j \dots j + m - 1]$ thì vị trí thử hiện tại của thuật toán là: j

1.5 Một số thuật toán đối sánh mẫu tiêu biểu

(1) Thuật toán Knuth-Morris-Pratt	(2) Thuật toán Zhu-Takaoka
(3) Thuật toán Karp-Rabin	(4) Thuật toán Berry-Ravindran
(5) Thuật toán Shift-Or	(6) Thuật toán Smith
(7) Thuật toán Morris-Pratt	(8) Thuật toán Raita
(9) Thuật toán Automat hữu hạn	(10) Thuật toán Reverse Factor
(11) Thuật toán Simon	(12) Thuật toán Turbo Reverse Factor
(13) Thuật toán Colussi	(14) Thuật toán Forward Dawg Matching
(15) Thuật toán Galil-Giancarlo	(16) Thuật toán Backward Nondeterministic Dawg Matching (BNDM)
(17) Thuật toán Apostolico-Crochemore	(18) Thuật toán Backward Oracle Matching
(19) Thuật toán Not So Naive	(20) Thuật toán Galil-Seiferas
(21) Thuật toán Turbo BM	(22) Thuật toán Two-Way

(23) Thuật toán Apostolico-Giancarlo	(24) Thuật toán String Matching on Ordered Alphabets
(25) Thuật toán Reverse Colussi	(26) Thuật toán Optimal Mismatch
(27) Thuật toán Horspool	(28) Thuật toán Maximal Shift
(29) Thuật toán Quick Search	(30) Thuật toán Skip Search
(31) Thuật toán Tuned Boyer-Moore	(32) Thuật toán KMP Skip Search
(33) Thuật toán Galil-Giancarlo	(34) Thuật toán Alpha
(35) Thuật toán Skip Search	

1.6 Các hướng tìm kiếm mẫu

1.6.1 Tìm kiếm mẫu từ trái sang phải

Là phương pháp tìm kiếm mẫu theo hướng từ trái sang phải, được sử dụng phổ biến trong nhiều thuật toán kinh điển.

Gồm các thuật toán sau:

(1) Thuật toán Karp and Rabin	(2) Thuật toán Automat hữu hạn
(3) Thuật toán Shift-Or	(4) Thuật toán Forward Dawg Matching
(5) Thuật toán Morris-Pratt	(6) Thuật toán Apostolico-Crochemore
(7) Thuật toán Knuth-Morris-Pratt	(8) Thuật toán Not So Naive
(9) Thuật toán Simon	

1.6.2 Tìm kiếm mẫu từ phải sang trái

Thực hiện đối sánh từ phải sang trái thường cho hiệu năng tốt trong thực tế nhờ khả năng bỏ qua nhiều phép so sánh không cần thiết.

Gồm các thuật toán sau:

(1) Thuật toán Apostolico-Giancarlo	(2) Thuật toán Backward Oracle Matching
(3) Thuật toán Turbo BM	(4) Thuật toán Zhu-Takaoka

(5) Thuật toán Reverse Colussi	(6) Thuật toán Berry-Ravindran
(7) Thuật toán Reverse Factor	(8) Thuật toán Backward Nondeterministic Dawg Matching (BNDM)
(9) Thuật toán Turbo Reverse Factor	

1.6.3 Tìm kiếm mẫu từ vị trí xác định

Các cận lý thuyết tốt nhất đạt được khi các phép so sánh được thực hiện theo một vị trí xác định.

Gồm các thuật toán sau:

(1) Thuật toán Galil-Seiferas	(2) Thuật toán Maximal Shift
(3) Thuật toán Two-Way	(4) Thuật toán Skip Search
(5) Thuật toán Colussi	(6) Thuật toán KMP Skip Search
(7) Thuật toán Galil-Giancarlo	(8) Thuật toán Alpha Skip Search
(9) Thuật toán Sunday Optimal Mismatch	(10) Thuật toán String Matching on Ordered Alphabets

1.6.4 Tìm kiếm mẫu từ vị trí bất kỳ

Gồm các thuật toán sau:

(1) Thuật toán Horspool	(2) Thuật toán Smith
(3) Thuật toán Quick Search	(4) Thuật toán Raita
(5) Thuật toán Tuned Boyer-Moore	

CHƯƠNG 2: CÁC THUẬT TOÁN TÌM KIẾM MẪU TỪ TRÁI SANG PHẢI

2.1 Thuật toán Not So Naive

2.1.1 Giới thiệu thuật toán

Thuật toán Not So Naive là một biến thể của thuật toán tìm kiếm chuỗi cơ bản (naive pattern matching). Mục tiêu của thuật toán là giảm số lần so sánh ký tự trong quá trình tìm kiếm mẫu trong văn bản.

Điểm khác biệt chính của Not So Naive nằm ở thứ tự so sánh ký tự trong mẫu. Thay vì so sánh từ trái sang phải như thông thường, thuật toán sẽ so sánh các ký tự từ vị trí 1 (ký tự thứ hai của mẫu) đến $m-1$ trước, sau đó mới so sánh ký tự đầu tiên.

2.1.2 Quy trình thực hiện

Giả sử có chuỗi văn bản y với độ dài n , chuỗi mẫu x với độ dài m . Thuật toán Not So Naive hoạt động theo các bước sau:

(1) Đặt mẫu lên văn bản: mẫu x được đặt lần lượt tại từng vị trí j của chuỗi y , tương ứng với đoạn $y[j .. j+m-1]$.

(2) So sánh phần bên trong mẫu: tại mỗi vị trí j , trước hết so sánh ký tự $x[1]$ với $y[j+1]$.

- Nếu $x[1]$ khác $y[j+1]$, vị trí này bị loại bỏ ngay.
- Nếu $x[1]$ trùng $y[j+1]$, tiếp tục so sánh toàn bộ đoạn $x[2..m-1]$ với $y[j+2..j+m-1]$.

(3) Kiểm tra ký tự đầu tiên:

- Nếu toàn bộ các ký tự từ 1 đến $m-1$ đều khớp, tiếp tục so sánh $x[0]$ với $y[j]$.
- Nếu khớp, ghi nhận tìm thấy mẫu tại vị trí j trong văn bản.

2.1.3 Quy tắc dịch cửa sổ

Sau mỗi lần so khớp hoặc không khớp tại vị trí j , thuật toán quyết định tăng j dựa trên việc so sánh giữa $x[1]$ và $y[j+1]$, và quan hệ giữa $x[0]$ và $x[1]$. Cụ thể như sau:

(1) Dịch 2 vị trí nếu thỏa mãn một trong hai điều kiện dưới đây:

- $x[0] = x[1]$ và $x[1] \neq y[j+1]$
- $x[0] \neq x[1]$ và $x[1] = y[j+1]$

(2) Trong các trường hợp còn lại, dịch sang phải 1 vị trí.

2.1.4 Đánh giá độ phức tạp thuật toán

2.1.4.1 Độ phức tạp không gian

Thuật toán Not So Naive không sử dụng cấu trúc dữ liệu phụ trợ, chỉ cần một vài biến để lưu vị trí so sánh và dịch mẫu. Vì vậy, độ phức tạp không gian của cả giai đoạn tiền xử lý và tìm kiếm đều là $S(n) = O(1)$.

2.1.4.2 Độ phức tạp thời gian

(1) Giai đoạn tiền xử lý: thuật toán chỉ kiểm tra quan hệ giữa hai ký tự đầu của mẫu để xác định bước dịch, nên thời gian tiền xử lý là $T_{preprocessing} = O(1)$.

(2) Giai đoạn tìm kiếm:

Trường hợp trung bình: nhờ cơ chế dịch mẫu 1 hoặc 2 ký tự, nhiều vị trí được loại bỏ sớm nên số phép so sánh giảm đáng kể. Vì vậy thuật toán có tính chất hơi dưới tuyến tính trong thực tế, tức là $T_{average} \leq O(n)$.

Trường hợp xấu nhất: tại mỗi vị trí của văn bản có thể phải so sánh gần như toàn bộ mẫu, do đó độ phức tạp là $T_{worst} = O(n \times m)$.

2.1.5 Kiểm nghiệm thuật toán

2.1.5.1 Dữ liệu vào

Cho hai chuỗi dưới đây. Thực hiện tìm kiếm mẫu x trong văn bản y

x="abca"
y="ababcabcabcabc"

2.1.5.2 Quy trình kiểm nghiệm thuật toán

Text Y character														Pattern X	
pos j	0	1	2	3	4	5	6	7	8	9	10	11	12	13	a b c a
y[j]	a	b	a	b	c	a	b	c	a	b	c	a	b	c	

Kiểm nghiệm thuật toán									
Bước	j	x	y[j..j+m-1] (1)	x[1] == y[j+1]? (2)	x[2..m-1] == y[j+2..j+m-1]? (3)	x[0] == y[j]? (4)	x[0]==x[1] (5)	Shift step = 1 + ((2) XOR (5))	Output
1	0	abca	abab	b==b (true)	ca==ab (false)		a==b (false)	2	
2	2	abca	abca	b==b (true)	ca==ca (true)	a==a (true)	a==b (false)	2	2
3	4	abca	cabc	b==a (false)			a==b (false)	1	
4	5	abca	abca	b==b (true)	ca==ca (true)	a==a (true)	a==b (false)	2	5
5	7	abca	cabc	b==a (false)			a==b (false)	1	
6	8	abca	abca	b==b (true)	ca==ca (true)	a==a (true)	a==b (false)	2	8
7	10	abca	cabc	b==a (false)			a==b (false)		

Sau khi kiểm nghiệm, kết quả cho thấy mẫu x="abca" tồn tại trong văn bản y="ababcabcabcabc" tại 3 vị trí là {2, 5, 8}

2.1.6 Cài đặt thuật toán với C++

```
void notSoNaive(string pattern, string text) {
    int m = pattern.size();
    int n = text.size();

    int j, k, ell;

    // Tiền xử lý: xác định k và ell dựa trên 2 ký tự đầu tiên của pattern
    if (pattern[0] == pattern[1]) {
        k = 2; // Nếu 2 ký tự đầu giống nhau, có thể bỏ qua cả 2 khi mismatch
        ell = 1; // Nếu match, chỉ cần shift 1 để kiểm tra lại
    } else {
        k = 1; // Nếu 2 ký tự đầu khác nhau, chỉ cần bỏ qua 1 ký tự khi mismatch
        ell = 2; // Nếu match, có thể bỏ qua 2 ký tự vì đã kiểm tra x[0] và x[1]
    }

    // Tìm kiếm pattern trong text
    j = 0;
    while (j <= n - m) {

        if (pattern[1] != text[j + 1]) {
            j += k; // Mismatch ở x[1], shift theo k
        } else {
            // Kiểm tra tiếp phần còn lại của pattern (bỏ qua x[1] đã match)
            if (text.substr(j + 2, m - 2) == pattern.substr(2, m - 2) &&
                // Đảm bảo x[0] cũng match với y[j]
                text[j] == pattern[0]) {
                cout << "Pattern occurs at shift " << j << endl;
            }
            j += ell; // Match ở x[1], shift theo ell
        }
    }
}
```

2.2 Thuật toán Karp-Rabin

2.2.1 Giới thiệu thuật toán

Thuật toán Karp-Rabin Algorithm là một thuật toán tìm kiếm mẫu trong chuỗi văn bản, được phát triển bởi hai nhà khoa học Richard M. Karp và Michael O. Rabin.

Ý tưởng chính của thuật toán là sử dụng hàm băm (hash function) để so sánh mẫu với các đoạn con của văn bản thay vì so sánh từng ký tự tại mọi vị trí. Nhờ đó, số lần so sánh ký tự được giảm đáng kể trong thực tế.

Thuật toán áp dụng kỹ thuật rolling hash để cập nhật giá trị băm khi cửa sổ trượt, tối ưu việc tính toán. Tuy nhiên, do có thể xảy ra va chạm hash, khi hai giá trị băm bằng nhau vẫn cần kiểm tra lại từng ký tự để đảm bảo kết quả chính xác.

Thuật toán hiệu quả khi:

- Tìm kiếm nhiều mẫu trong cùng một văn bản.
- Tìm kiếm trên văn bản dài.
- Các phép tính hàm băm được thực hiện nhanh.

2.2.2 Quy trình thực hiện thuật toán

Giả sử có chuỗi văn bản y với độ dài n , chuỗi mẫu x với độ dài m . Thuật toán Karp-Rabin hoạt động theo các bước sau:

(1) Tính giá trị hash ban đầu:

- Tính giá trị hash cho chuỗi mẫu x .
- Tính giá trị hash cho đoạn đầu tiên của văn bản $y[0..m-1]$.

(2) Tại mỗi vị trí j của văn bản, thực hiện so sánh: $hash(x)$ với $hash(y[j..j+m-1])$.

- Nếu hai giá trị hash khác nhau, thuật toán tiếp tục dịch cửa sổ tìm kiếm.
- Nếu hai giá trị hash bằng nhau, so sánh trực tiếp từng ký tự của: $x[0..m-1]$ và $y[j..j+m-1]$.
 - Nếu toàn bộ các ký tự đều khớp, tìm thấy mẫu tại vị trí j trong văn bản
 - Nếu có ít nhất một ký tự không khớp, kết luận đây là trường hợp va chạm hash và tiếp tục tìm kiếm tại vị trí kế tiếp.

(3) Dịch cửa sổ tìm kiếm một đơn vị và tính lại hash của cửa sổ tại vị trí mới bằng kỹ thuật rolling hash mà không cần tính lại từ đầu

(4) Thuật toán tiếp tục thực hiện các bước trên cho đến khi văn bản được duyệt hết.

2.2.3 Đánh giá độ phức tạp thuật toán

2.2.3.1 Độ phức tạp không gian

Thuật toán Karp–Rabin chỉ sử dụng một số biến phụ để: lưu giá trị hash, duyệt chuỗi, cập nhật rolling hash.

Thuật toán không cần thêm cấu trúc dữ liệu phụ thuộc vào kích thước đầu vào, do đó độ phức tạp không gian là: $S(n) = O(1)$

2.2.3.2 Độ phức tạp thời gian

(1) Giai đoạn tiền xử lý: thuật toán tính hash của mẫu, tính hash ban đầu của văn bản. Các thao tác này cần duyệt qua m ký tự của mẫu, nên độ phức tạp là: $T_{preprocessing} = O(m)$

(2) Giai đoạn tìm kiếm:

Thuật toán kiểm tra tất cả $n - m + 1$ vị trí trong văn bản. Tại mỗi vị trí cần so sánh hash trong thời gian $O(1)$, nếu hash trùng thì so sánh trực tiếp tối đa m ký tự.

Trường hợp trung bình: do xác suất va chạm hash thấp nên số lần so sánh ký tự ít. Độ phức tạp trung bình: $T_{average} = O(n + m)$

Trường hợp xấu nhất: khi xảy ra nhiều va chạm hash, thuật toán phải so sánh gần như toàn bộ mẫu tại nhiều vị trí: $T_{worst} = O(m \times n)$ (ví dụ với mẫu "aaaa" và văn bản "aaaaaaaaaa")

2.2.4 Kiểm nghiệm thuật toán

2.2.4.1 Dữ liệu vào

Cho hai xâu dưới đây. Thực hiện tìm kiếm mẫu x trong văn bản y

x="abca"
y="ababcabcabcabc"

2.2.4.2 Quy trình kiểm nghiệm thuật toán

Text and Pattern character															Full string
pos j	0	1	2	3	4	5	6	7	8	9	10	11	12	13	
y[j]	a	b	a	b	c	a	b	c	a	b	c	a	b	c	ababcabcabcabc
x[j]	a	b	c	a											abca

Bitmask S						Vector state R				
pos	3	2	1	0	Ghi chú	pos	3	2	1	0
S[a]	0	1	1	0	(trong pattern: a nằm ở vị trí 0 và 3) -> S[a][0] = S[a][3] = 0	R[j]	1	1	1	1
S[b]	1	1	0	1	(trong pattern: b nằm ở vị trí 1) -> S[b][1] = 0					
S[c]	1	0	1	1	(trong pattern: c nằm ở vị trí 2) -> S[c][2]= 0					
others	1	1	1	1						

Kiểm nghiệm thuật toán								
Bước	j	y[j]	Vector state R (trước)	SHIFT(R)	Bitmask S[y[j]]	Vector state R (cập nhật)	R[m-1]==0?	Output pos = j - m - 1
1	0	a	1 1 1 1	1 1 1 0	0 1 1 0	1 1 1 0	1==0 (false)	
2	1	b	1 1 1 0	1 1 0 0	1 1 0 1	1 1 0 1	1==0 (false)	
3	2	a	1 1 0 1	1 0 1 0	0 1 1 0	1 1 1 0	1==0 (false)	
4	3	b	1 1 1 0	1 1 0 0	1 1 0 1	1 1 0 1	1==0 (false)	
5	4	c	1 1 0 1	1 0 1 0	1 0 1 1	1 0 1 1	1==0 (false)	
6	5	a	1 0 1 1	0 1 1 0	0 1 1 0	0 1 1 0	0==0 (true)	2
7	6	b	0 1 1 0	1 1 0 0	1 1 0 1	1 1 0 1	1==0 (false)	
8	7	c	1 1 0 1	1 0 1 0	1 0 1 1	1 0 1 1	1==0 (false)	
9	8	a	1 0 1 1	0 1 1 0	0 1 1 0	0 1 1 0	0==0 (true)	5
10	9	b	0 1 1 0	1 1 0 0	1 1 0 1	1 1 0 1	1==0 (false)	
11	10	c	1 1 0 1	1 0 1 0	1 0 1 1	1 0 1 1	1==0 (false)	
12	11	a	1 0 1 1	0 1 1 0	0 1 1 0	0 1 1 0	0==0 (true)	8
13	12	b	0 1 1 0	1 1 0 0	1 1 0 1	1 1 0 1	1==0 (false)	
14	13	c	1 1 0 1	1 0 1 0	1 0 1 1	1 0 1 1	1==0 (false)	

Sau khi kiểm nghiệm, kết quả cho thấy mẫu x="abca" tồn tại trong văn bản y="ababcabcabcabc" tại 3 vị trí là {2, 5, 8}

2.2.5 Cài đặt thuật toán với C++

```
// Hàm băm rolling: tính toán hash cho cửa sổ tiếp theo dựa trên hash hiện tại
#define REHASH(a, b, h, d) (((h) - (a) * (d)) << 1) + (b))

void karpRabin(string pattern, string text) {
    int m = pattern.length(), n = text.length();

    int d = 1;    // d = 2^(m-1), trọng số của ký tự đầu tiên trong cửa sổ (MSB)
    int hx = 0, hy = 0, i, j;

    // Tiền xử lý: tính 2^(m-1) để dùng khi loại bỏ ký tự đầu trong rolling hash
    for (i = 1; i < m; i++) d = (d << 1);

    // Hash ban đầu của pattern và cửa sổ đầu tiên của text
    for (i = 0; i < m; i++) {
        hx = (hx << 1) + pattern[i];
        hy = (hy << 1) + text[i];
    }

    // Tìm kiếm pattern trong text
    j = 0;
    while (j <= n - m) {
        // So sánh hash + kiểm tra chuỗi thật
        if (hx == hy && text.substr(j, m) == pattern) {
            cout << "Pattern found at index " << j << endl;
        }
        // Cập nhật hash cửa sổ tiếp theo
        if (j < n - m) hy = REHASH(text[j], text[j + m], hy, d);
        j++;
    }
}
```

2.3 Thuật toán Shift-Or

2.3.1 Giới thiệu thuật toán

Thuật toán Shift-Or (hay Bi-tap) là một thuật toán tìm kiếm chuỗi sử dụng kỹ thuật xử lý bit (bitwise). Thuật toán biểu diễn trạng thái khớp của mẫu bằng các vector bit, từ đó thay vì so sánh từng ký tự, thuật toán thực hiện các phép toán dịch bit và OR để xác định vị trí khớp trong văn bản.

Thuật toán đặc biệt hiệu quả khi độ dài mẫu không vượt quá kích thước từ nhớ (machine word) của máy tính, giúp các phép toán bit được thực hiện rất nhanh trên phần cứng.

Điểm nổi bật của thuật toán là khả năng cập nhật trạng thái khớp theo từng ký tự của văn bản chỉ bằng các phép toán bit đơn giản, không cần so sánh chuỗi trực tiếp trong hầu hết trường hợp.

2.3.2 Quy trình thực hiện thuật toán

Giả sử có chuỗi văn bản y với độ dài n , chuỗi mẫu x với độ dài m . Thuật toán Shift-Or hoạt động theo các bước sau:

2.3.2.1 Tiền xử lý mẫu: khởi tạo bitmask S và vector trạng thái R

(1) Khởi tạo dữ liệu: Xây dựng bảng bitmask S cho từng ký tự c trong bảng chữ cái σ . $S[c]$ là một số nguyên dạng bitmask có m bit, với mỗi vị trí j trong mẫu:

- Nếu $x[j] = c \rightarrow$ bit j của $S[c] = 0$
- Nếu $x[j] \neq c \rightarrow$ bit j của $S[c] = 1$

(2) Khởi tạo vector trạng thái R ban đầu (thường là toàn 1, tức là chưa có ký tự nào khớp). Vector R dùng để theo dõi mức độ khớp của các tiền tố mẫu trong quá trình duyệt.

2.3.2.2 Quy trình so khớp và tìm kiếm

(1) Với mỗi ký tự $y[j]$ trong văn bản, cập nhật vector trạng thái R theo công thức:

$$R = (R \ll 1) \text{ OR } S[y[j]].$$

- $R \ll 1$: dịch trái vector trạng thái một bit (mở rộng khớp theo chuỗi)
- OR với $S[y[j]]$ để cập nhật vị trí không khớp

(2) Sau mỗi lần cập nhật R , nếu $R[m - 1] = 0$, tìm thấy mẫu tại vị trí j trong văn bản.

(3) Lặp lại bước 1 và 2 cho đến khi duyệt hết toàn bộ chuỗi văn bản.

2.3.3 Đánh giá độ phức tạp thuật toán

2.3.3.1 Độ phức tạp không gian

Thuật toán cần lưu σ bitmask cho bảng chữ cái và một vector trạng thái R có kích thước m . Do đó, độ phức tạp không gian là: $S(n) = O(m + \sigma)$

2.3.3.2 Độ phức tạp thời gian

(1) Giai đoạn tiền xử lý: Thuật toán xây dựng các vector bit $S[c]$ cho từng ký tự trong mẫu, do đó độ phức tạp là: $T_{preprocessing} = O(m + \sigma)$

(2) Giai đoạn tìm kiếm: Thuật toán duyệt toàn bộ chuỗi văn bản một lần và tại mỗi ký tự thực hiện các phép toán bit (SHIFT và OR), nên độ phức tạp là: $T_{search} = O(n)$. Với thuật toán Shift-Or, thời gian tìm kiếm không phụ thuộc vào độ dài mẫu và bảng chữ cái.

2.3.4 Kiểm nghiệm thuật toán

2.3.4.1 Dữ liệu vào

Cho hai xâu dưới đây. Thực hiện tìm kiếm mẫu x trong văn bản y

$x = \text{"abca"}$
 $y = \text{"ababcbcabcbabc"}$

2.3.4.2 Quy trình kiểm nghiệm thuật toán

(1) Giai đoạn tiền xử lý: khởi tạo vector bit $S[c]$ và vector trạng thái R

Text and Pattern character														Full string	
pos j	0	1	2	3	4	5	6	7	8	9	10	11	12		13
y[j]	a	b	a	b	c	a	b	c	a	b	c	a	b	c	ababcbcabcbabc
x[j]	a	b	c	a											abca

Bitmask S						Vector state R				
pos	3	2	1	0	Ghi chú	pos	3	2	1	0
S[a]	0	1	1	0	(trong pattern: a nằm ở vị trí 0 và 3) -> S[a][0] = S[a][3] = 0	R[j]	1	1	1	1
S[b]	1	1	0	1	(trong pattern: b nằm ở vị trí 1) -> S[b][1] = 0					
S[c]	1	0	1	1	(trong pattern: c nằm ở vị trí 2) -> S[c][2] = 0					
others	1	1	1	1						

(2) Giai đoạn tìm kiếm:

Sau khi kiểm nghiệm, kết quả cho thấy mẫu $x = \text{"abca"}$ tồn tại trong văn bản $y = \text{"ababcbcabcbabc"}$ tại 3 vị trí là $\{2, 5, 8\}$

Kiểm nghiệm thuật toán								
Bước	j	y[j]	Vector state R (trước)	SHIFT(R)	Bitmask S[y[j]]	Vector state R (cập nhật)	R[m-1]==0?	Output pos = j - m - 1
1	0	a	1 1 1 1	1 1 1 0	0 1 1 0	1 1 1 0	1==0 (false)	
2	1	b	1 1 1 0	1 1 0 0	1 1 0 1	1 1 0 1	1==0 (false)	
3	2	a	1 1 0 1	1 0 1 0	0 1 1 0	1 1 1 0	1==0 (false)	
4	3	b	1 1 1 0	1 1 0 0	1 1 0 1	1 1 0 1	1==0 (false)	
5	4	c	1 1 0 1	1 0 1 0	1 0 1 1	1 0 1 1	1==0 (false)	
6	5	a	1 0 1 1	0 1 1 0	0 1 1 0	0 1 1 0	0==0 (true)	2
7	6	b	0 1 1 0	1 1 0 0	1 1 0 1	1 1 0 1	1==0 (false)	
8	7	c	1 1 0 1	1 0 1 0	1 0 1 1	1 0 1 1	1==0 (false)	
9	8	a	1 0 1 1	0 1 1 0	0 1 1 0	0 1 1 0	0==0 (true)	5
10	9	b	0 1 1 0	1 1 0 0	1 1 0 1	1 1 0 1	1==0 (false)	
11	10	c	1 1 0 1	1 0 1 0	1 0 1 1	1 0 1 1	1==0 (false)	
12	11	a	1 0 1 1	0 1 1 0	0 1 1 0	0 1 1 0	0==0 (true)	8
13	12	b	0 1 1 0	1 1 0 0	1 1 0 1	1 1 0 1	1==0 (false)	
14	13	c	1 1 0 1	1 0 1 0	1 0 1 1	1 0 1 1	1==0 (false)	

2.3.5 Cài đặt thuật toán với C++

2.3.5.1 Hàm tiền xử lý: tính giá trị lim và lập bảng bitmask S

```
const int ASIZE = 256; // Số lượng ký tự có thể xuất hiện (ASCII) - 256 ký tự

unsigned int preSo(string pattern, unsigned int S[]) {
    unsigned int j, lim;

    // Khởi tạo bảng S với tất cả bit được đặt là 1 (tức là ~0 - NOT 0)
    for (int i = 0; i < ASIZE; i++) S[i] = ~0;

    lim = 0; // lim dùng để xác định điều kiện match

    for (int i = 0, j = 1; i < pattern.size(); i++, j <= 1) {
        // Cập nhật bảng S: Đặt bit j thành 0 cho ký tự pattern[i]
        S[(unsigned char) pattern[i]] &= ~j;
        lim |= j;
    }
    // Dịch lim và đảo bit để tạo mask kiểm tra match
    lim = ~(lim >> 1);
    return lim;
}
```

2.3.5.2 Hàm tìm kiếm bằng thuật toán Shift-Or

```
void ShiftOr(string pattern, string text) {
    int m = pattern.size(), n = text.size();
    unsigned int S[ASIZE], lim = preSo(pattern, S), state = ~0;

    for (int j = 0; j < n; j++) {
        state = (state << 1) | S[(unsigned char)text[j]];
        // Nếu trạng thái nhỏ hơn lim, nghĩa là đã tìm thấy một khớp
        if (state < lim) cout << "Match at position " << (j - m + 1) << endl;
    }
}
```

2.4 Thuật toán Morris-Pratt

2.4.1 Giới thiệu thuật toán

Thuật toán Morris-Pratt là một thuật toán tìm kiếm chuỗi cổ điển, được thiết kế nhằm cải thiện hạn chế của thuật toán Brute Force bằng cách tận dụng thông tin đã so khớp trước đó để tránh so sánh lại các ký tự không cần thiết.

Thay vì quay lui trên văn bản khi xảy ra sai khác, thuật toán Morris-Pratt sử dụng một cấu trúc tiền xử lý gọi là bảng `mpNext` để xác định vị trí thích hợp trong mẫu cần quay lại khi có sai khác.

Ý tưởng cốt lõi của thuật toán là: khi một đoạn tiền tố của mẫu đã khớp với văn bản, nếu xảy ra sai khác, ta không cần bắt đầu lại từ đầu mẫu mà có thể dịch chuyển mẫu sang phải dựa trên độ dài biên (border) của đoạn đã khớp.

Nhờ cơ chế này, thuật toán đảm bảo mỗi ký tự trong văn bản chỉ được xử lý một số lần giới hạn, giúp tăng hiệu suất tìm kiếm đáng kể.

2.4.2 Quy trình thực hiện thuật toán

Giả sử có chuỗi văn bản y độ dài n và chuỗi mẫu x độ dài m . Thuật toán Morris-Pratt được thực hiện theo quy trình sau.

2.4.2.1 Tiền xử lý mẫu: lập bảng `mpNext`

Với mỗi vị trí trong mẫu, thuật toán cần xác định độ dài của border dài nhất của đoạn tiền tố tương ứng so với vị trí đang xét (nếu không tồn tại, độ dài được quy ước là -1), với:

- Prefix (tiền tố) là đoạn con nằm ở đầu chuỗi.
- Suffix (hậu tố) là đoạn con nằm ở cuối chuỗi.
- Border của một chuỗi là đoạn con vừa là prefix vừa là suffix của chuỗi đó, nhưng không được trùng với toàn bộ chuỗi.

Kết quả thu được bảng `mpNext` dùng để xác định vị trí quay lui khi xảy ra sai khác trong quá trình tìm kiếm.

2.4.2.2 Quy trình so khớp và tìm kiếm

(1) Đặt hai biến chỉ số: i trở vào mẫu, j trở vào văn bản, bắt đầu so sánh từ vị trí đầu tiên của cả hai chuỗi.

(2) So sánh ký tự: so sánh ký tự hiện tại của mẫu $x[i]$ với ký tự văn bản $y[j]$.

- Nếu trùng khớp, tăng cả hai chỉ số để tiếp tục so sánh các ký tự tiếp theo.

- Nếu không trùng khớp, di chuyển chỉ số i về vị trí thích hợp trong mẫu dựa trên $mpNext$.

(3) Kiểm tra kết quả khớp

- Nếu chỉ số i đạt đến độ dài m , nghĩa là toàn bộ mẫu đã khớp, ghi nhận tìm thấy mẫu tại vị trí j trong văn bản
- Sau đó, tiếp tục tìm kiếm các lần xuất hiện tiếp theo bằng cách dùng $mpNext$ để cập nhật trạng thái.

(4) Lặp lại quá trình cho đến khi duyệt hết chuỗi văn bản.

2.4.3 Đánh giá độ phức tạp thuật toán

2.4.3.1 Độ phức tạp không gian

Thuật toán cần lưu bảng $mpNext$ có kích thước m , dùng để lưu thông tin về border của các tiền tố trong mẫu nhằm hỗ trợ việc quay lui khi xảy ra sai khác. Do vậy, $S(n) = O(m)$

2.4.3.2 Độ phức tạp thời gian

(1) Giai đoạn tiền xử lý: xây dựng bảng $mpNext$ bằng cách phân tích chính chuỗi mẫu. Mỗi ký tự trong mẫu được xử lý một số lần hữu hạn, nên: $T_{preprocessing} = O(m)$

(2) Giai đoạn tìm kiếm: Thuật toán duyệt chuỗi văn bản một lần từ trái sang phải. Khi xảy ra sai khác, thuật toán sử dụng bảng $mpNext$ để xác định vị trí tiếp tục trong mẫu mà không quay lại văn bản. Nhờ đó, mỗi ký tự trong văn bản chỉ bị xử lý tối đa một số lần hữu hạn. Do vậy: $T_{search} = O(n + m)$

2.4.4 Kiểm nghiệm thuật toán

2.4.4.1 Dữ liệu vào

Cho hai xâu dưới đây. Thực hiện tìm kiếm mẫu x trong văn bản y

```
x="aaba"
y="aaabaaabaaabaa"
```

2.4.4.2 Quy trình kiểm nghiệm thuật toán

(1) Giai đoạn tiền xử lý: tính toán mảng $mpNext$

Text and Pattern character															Full string
pos j	0	1	2	3	4	5	6	7	8	9	10	11	12	13	
y[j]	a	a	b	a	a	b	a	a	c	a	a	b	a	a	aaabaaabaaabaa
x[j]	a	a	b	a											aaba

Các bước xây dựng mảng mpNext[i]					
i	x[0..i-1]	Các prefix	Các suffix	Border	mpNext[i] = Longest Border length
0	""	Không	Không	Không	-1
1	a	""	""	""	0
2	aa	"", a	"", a	"", "a"	1
3	aaa	"", a, aa	"", b, ab	""	0
4	aaab	"", a, aa, aaa	"", a, ba, aba	"", "a"	1

(2) Giai đoạn tìm kiếm mẫu:

Kiểm nghiệm thuật toán										
Bước	i	j	x[i]	y[j]	x[i] == y[j]?	i (cập nhật) và giải thích		j (cập nhật)	Kết quả so khớp	Output pos = j - i
1	0	0	a	a	a==a (true)	1	i += 1	1	Khớp	
2	1	1	a	a	a==a (true)	2	i += 1	2	Khớp	
3	2	2	b	b	b==b (true)	3	i += 1	3	Khớp	
4	3	3	a	a	a==a (true)	4	i += 1	4	Khớp	
5	4	4				1	i=mpNext[4]=1	4	Tìm thấy mẫu	0
6	1	4	a	a	a==a (true)	2	i += 1	5	Khớp	
7	2	5	b	b	b==b (true)	3	i += 1	6	Khớp	
8	3	6	a	a	a==a (true)	4	i += 1	7	Khớp	
9	4	7				1	i=mpNext[4]=1	7	Tìm thấy mẫu	3
10	1	7	a	a	a==a (true)	2	i += 1	8	Khớp	
11	2	8	b	c	b==c (false)	1	i=mpNext[2]=1	8	Sai khác	
12	1	8	a	c	a==c (false)	0	i=mpNext[1]=0	8	Sai khác	
13	0	8	a	c	a==c (false)	-1	i=mpNext[0]=-1	8	Sai khác	
14	-1	8				0	i += 1	9	Ngoài mảng	
15	0	9	a	a	a==a (true)	1	i += 1	10	Khớp	
16	1	10	a	a	a==a (true)	2	i += 1	11	Khớp	
17	2	11	b	b	b==b (true)	3	i += 1	12	Khớp	
18	3	12	a	a	a==a (true)	4	i += 1	13	Khớp	
19	4	13				1	i=mpNext[4]=1	13	Tìm thấy mẫu	9
20	1	13	a	a	a==a (true)	2	i += 1	14	Khớp	

Sau khi kiểm nghiệm, kết quả cho thấy mẫu x="aaba" tồn tại trong văn bản y="aaabaaabaaabaa" tại 3 vị trí là {0, 3, 9}

2.4.5 Cài đặt thuật toán với C++

2.4.5.1 Hàm tiền xử lý: lập bảng mpNext

```
// Hàm tiền xử lý tạo bảng mpNext
vector<int> preMp(const string& pattern) {
    int m = pattern.length();
    vector<int> mpNext(m + 1);

    int i = 0;
    int j = -1;
    mpNext[0] = -1;
```

```
while (i < m) {
    // Nếu sai khác, quay lại vị trí mpNext của j
    while (j > -1 && pattern[i] != pattern[j]) j = mpNext[j];

    // Nếu khớp, tăng i và j lên
    i++;
    j++;
    mpNext[i] = j;
}
return mpNext;
}
```

2.4.5.2 Hàm tìm kiếm bằng thuật toán Morris-Pratt

```
// Hàm tìm kiếm Morris-Pratt
void morrisPratt(const string& pattern, const string& text) {
    int m = pattern.length();
    int n = text.length();

    vector<int> mpNext = preMp(pattern);

    int i = 0, j = 0;

    while (j < n) {
        // Sai khác, quay lại vị trí mpNext của i
        while (i > -1 && pattern[i] != text[j]) i = mpNext[i];

        // Khớp, tăng i và j lên
        i++;
        j++;

        // Tìm thấy mẫu
        if (i >= m) {
            cout << "Match at position: " << (j - i) << endl;

            // Quay lại vị trí mpNext của i và tiếp tục tìm kiếm
            i = mpNext[i];
        }
    }
}
```

2.5 Thuật toán Knuth-Morris-Pratt

2.5.1 Giới thiệu thuật toán

Thuật toán Knuth-Morris-Pratt (KMP) là một thuật toán tìm kiếm chuỗi cải tiến từ Morris-Pratt, nhằm tối ưu hóa quá trình dịch chuyển khi xảy ra sai khác bằng cách sử dụng thông tin “tiên đoán” sâu hơn về cấu trúc của mẫu.

Thay vì chỉ dựa vào border như thuật toán Morris-Pratt, thuật toán KMP sử dụng khái niệm tagged border (biên có điều kiện) để xác định chính xác vị trí có thể quay lui trong mẫu, giúp giảm số lần so sánh trong quá trình tìm kiếm.

Ý tưởng cốt lõi của thuật toán là: khi xảy ra sai khác giữa mẫu và văn bản, ta không chỉ xét phần border của đoạn đã khớp mà còn đảm bảo ký tự tiếp theo trong mẫu khác với ký tự gây sai khác, nhằm tránh lặp lại ngay sai khác vừa xảy ra.

Nhờ cải tiến này, KMP giảm số lần so sánh trung bình so với MP và đảm bảo hiệu quả tuyến tính trong mọi trường hợp.

2.5.2 Quy trình thực hiện thuật toán

Giả sử có chuỗi văn bản y độ dài n và chuỗi mẫu x độ dài m . Thuật toán Knuth-Morris-Pratt được thực hiện theo quy trình sau.

2.5.2.1 Tiên xử lý mẫu: lập bảng $kmpNext$

Khác với thuật toán Morris-Pratt, bảng $kmpNext$ không chỉ lưu border dài nhất mà còn xét điều kiện ký tự tiếp theo.

Với mỗi vị trí trong mẫu, xác định độ dài của tagged border dài nhất của đoạn text từ vị trí đầu đến vị trí đang xét (nếu không tồn tại, quy ước độ dài là -1). Trong đó:

- Prefix (tiền tố) là đoạn con nằm ở đầu chuỗi.
- Suffix (hậu tố) là đoạn con nằm ở cuối chuỗi.
- Border là đoạn con vừa là prefix vừa là suffix, nhưng không được trùng với toàn bộ chuỗi.
- Tagged border cũng là một border của tiền tố, nhưng được bổ sung thêm một điều kiện bắt buộc: ký tự đi ngay sau tiền tố phải khác với ký tự đi ngay sau hậu tố.

Kết quả thu được bảng $kmpNext$, dùng để xác định vị trí quay lui tối ưu khi xảy ra sai khác trong quá trình tìm kiếm.

2.5.2.2 Quy trình so khớp và tìm kiếm

(1) Đặt hai biến chỉ số: i trở vào chuỗi mẫu, j trở vào chuỗi văn bản, bắt đầu so sánh từ vị trí đầu tiên của cả hai chuỗi.

(2) So sánh ký tự hiện tại của mẫu $x[i]$ với ký tự văn bản $y[j]$

- Nếu trùng khớp: tăng đồng thời i và j để tiếp tục so sánh các ký tự tiếp theo.
- Nếu sai khác: sử dụng bảng kmpNext để cập nhật lại vị trí của mẫu bằng cách đưa i về vị trí thích hợp trong mẫu theo giá trị kmpNext. Sau đó, tiếp tục so sánh lại với ký tự hiện tại của văn bản.

(3) Kiểm tra kết quả khớp

- Nếu i đạt đến giá trị m , nghĩa là toàn bộ mẫu đã khớp trong văn bản, ghi nhận tìm thấy mẫu tại vị trí j trong văn bản.
- Sau đó, tiếp tục tìm kiếm các lần xuất hiện tiếp theo bằng cách sử dụng kmpNext để cập nhật trạng thái.

(4) Quá trình được lặp lại cho đến khi duyệt hết chuỗi văn bản.

2.5.3 Đánh giá độ phức tạp thuật toán

2.5.3.1 Độ phức tạp không gian

Thuật toán cần lưu bảng kmpNext có kích thước m , dùng để lưu thông tin về các border có điều kiện của các tiền tố trong mẫu nhằm tối ưu hóa quá trình quay lui. Do đó độ phức tạp không gian của thuật toán là: $S(n) = O(m)$

2.5.3.2 Độ phức tạp thời gian

(1) Giai đoạn tiền xử lý: xây dựng bảng kmpNext bằng cách phân tích chính chuỗi mẫu, trong đó mỗi ký tự có thể được xử lý một số lần hữu hạn nhờ cơ chế quay lui trong mẫu. Do vậy độ phức tạp của thuật toán trong giai đoạn tiền xử lý là, $T_{preprocessing} = O(m)$

(2) Giai đoạn tìm kiếm: thực hiện so khớp từ trái sang phải trên chuỗi văn bản.

Nhờ cơ chế sử dụng bảng kmpNext để điều chỉnh vị trí trong mẫu khi xảy ra sai khác, thuật toán đảm bảo tổng số phép so sánh tuyến tính theo kích thước dữ liệu đầu vào.

Do đó, độ phức tạp thời gian của giai đoạn tìm kiếm là: $T_{search} = O(n + m)$, trong đó n là độ dài chuỗi văn bản, và độ phức tạp này không phụ thuộc vào kích thước bảng chữ cái.

2.5.4 Kiểm nghiệm thuật toán

2.5.4.1 Dữ liệu vào

Cho hai chuỗi dưới đây. Thực hiện tìm kiếm mẫu x trong văn bản y

$x = \text{"aaba"}$

$y = \text{"aaabaaabaaabaa"}$

2.5.4.2 Quy trình kiểm nghiệm thuật toán

(1) Giai đoạn tiền xử lý: tính toán mảng $kmpNext$

Text and Pattern character															Full string
pos j	0	1	2	3	4	5	6	7	8	9	10	11	12	13	
y[j]	a	a	b	a	a	b	a	a	c	a	a	b	a	a	aaabaabaaabaa
x[j]	a	a	b	a											aaba

Xác định border				
i	$x[0..i-1]$	Prefix	Suffix	Border
0	""	Không	Không	Không
1	a	""	""	""
2	aa	"" , a	"" , a	"" , a
3	aab	"" , a, aa	"" , b, ab	""
4	aaba	"" , a, aa, aaa	"" , a, ba, aba	"" , a

Duyệt từng border để xác định $kmpNext[i]$						
i	Border	$x[\text{border.size}()]$ (1)	$x[i]$ (2)	isTaggedBorder? $= ((1) \neq (2))$	Tagged Border	$kmpNext[i]$ $= \max(\text{taggedBorder.size}())$
0	Không				Không	-1
1	""	a	a	false	Không	-1
2	""	a	b	true	"" , a	1
2	a	a	b	true		
3	""	a	a	false	Không	-1
4	""	a		true	"" , a	1
4	a	a		true		

Mảng $kmpNext$	
i	$kmpNext[i]$
0	-1
1	-1
2	1
3	-1
4	1

(2) Giai đoạn tìm kiếm mẫu:

Sau khi kiểm nghiệm, kết quả cho thấy mẫu $x = \text{"aaba"}$ tồn tại trong văn bản $y = \text{"aaabaaabaaabaa"}$ tại 3 vị trí là $\{0, 3, 9\}$

Điểm khác biệt cốt lõi giữa hai thuật toán MP và KMP nằm ở bước 12 tại vị trí sai khác $j = 8$ (ký tự 'c').

Thay vì phải lười tuần tự qua nấc trung gian $mpNext[1] = 0$ và tốn thêm phép so sánh như MP, KMP áp dụng quy tắc Tagged Border để nhảy cóc thẳng về $kmpNext[1] = -1$, giải quyết vị trí sai khác chỉ trong 3 bước con (ít hơn MP 1 bước).

Dù trên testcase quy mô nhỏ này sự chênh lệch chưa quá lớn, nhưng khi dữ liệu tăng lên và sai khác có tính tích lũy, khả năng nhảy bậc vượt trội này giúp KMP tối ưu hóa tài nguyên CPU, giảm độ trễ và đạt tốc độ tìm kiếm nhanh hơn hẳn so với MP.

Kiểm nghiệm thuật toán										
Bước	i	j	x[i]	y[j]	x[i] == y[j]?	i (cập nhật) và giải thích		j (cập nhật)	Kết quả so khớp	Output pos = j - i
1	0	0	a	a	a==a (true)	1	i += 1	1	Khớp	
2	1	1	a	a	a==a (true)	2	i += 1	2	Khớp	
3	2	2	b	b	b==b (true)	3	i += 1	3	Khớp	
4	3	3	a	a	a==a (true)	4	i += 1	4	Khớp	
5	4	4				1	i=kmpNext[4]=1	4	Tìm thấy mẫu	0
6	1	4	a	a	a==a (true)	2	i += 1	5	Khớp	
7	2	5	b	b	b==b (true)	3	i += 1	6	Khớp	
8	3	6	a	a	a==a (true)	4	i += 1	7	Khớp	
9	4	7				1	i=kmpNext[4]=1	7	Tìm thấy mẫu	3
10	1	7	a	a	a==a (true)	2	i += 1	8	Khớp	
11	2	8	b	c	b==c (false)	1	i=kmpNext[2]=1	8	Sai khác	
12	1	8	a	c	a==c (false)	-1	i=kmpNext[1]=-1	8	Sai khác	
13	-1	8				0	i += 1	9	Ngoài mảng	
14	0	9	a	a	a==a (true)	1	i += 1	10	Khớp	
15	1	10	a	a	a==a (true)	2	i += 1	11	Khớp	
16	2	11	b	b	b==b (true)	3	i += 1	12	Khớp	
17	3	12	a	a	a==a (true)	4	i += 1	13	Khớp	
18	4	13				1	i=kmpNext[4]=1	13	Tìm thấy mẫu	9
19	1	13	a	a	a==a (true)	2	i += 1	14	Khớp	

2.5.5 Cài đặt thuật toán với C++

Mặc dù phần kiểm nghiệm thuật toán là kiểm nghiệm tĩnh, tức là mô tả quá trình khớp chuỗi theo từng bước con tuần tự (thể hiện việc con trở quay lui khi gặp sai khác), nhưng mã nguồn thực thi lại tối ưu hóa luồng chạy này nhờ tư duy Quy hoạch động.

2.5.5.1 Hàm tiền xử lý mẫu: lập bảng kmpNext

```
// Hàm tiền xử lý tạo bảng kmpNext (Tagged Border)
vector<int> preKmp(const string& pattern) {
    int m = pattern.length();
    vector<int> kmpNext(m + 1);

    int i = 0, j = -1;
    kmpNext[0] = -1;

    while (i < m) {
        // Nếu sai khác, quay lại vị trí kmpNext của j
        while (j > -1 && pattern[i] != pattern[j]) j = kmpNext[j];

        i++; j++;

        // Nếu ký tự kế tiếp trùng nhau và chưa vượt quá m
        // kế thừa giá trị bước nhảy để tối ưu tagged border
        if (i < m && pattern[i] == pattern[j]) kmpNext[i] = kmpNext[j];
        else kmpNext[i] = j; // Gán giá trị bước nhảy bằng độ dài border hiện tại
    }

    return kmpNext;
}
```

Mảng `kmpNext` không được tính toán độc lập mà được xây dựng bằng cách truy vết lại các bài toán con đã giải trước đó ($kmpNext[i] = kmpNext[j]$).

Điều kiện kiểm tra ký tự kế tiếp `if (pattern[i] == pattern[j])` cho phép mã nguồn gom nhiều bước lùi thủ công trong bảng kiểm nghiệm tĩnh thành duy nhất một bước nhảy cóc về trạng thái an toàn sâu hơn (ví dụ: nhảy thẳng từ 1 về -1 tại bước 12).

2.5.5.2 Hàm tìm kiếm bằng thuật toán Knuth-Morris-Pratt

```
// Hàm tìm kiếm Knuth-Morris-Pratt
void knuthMorrisPratt(const string& pattern, const string& text) {
    int m = pattern.length();
    int n = text.length();

    vector<int> kmpNext = preKmp(pattern);

    int i = 0, j = 0;

    while (j < n) {
        // Sai khác, quay lại vị trí kmpNext của i
        while (i > -1 && pattern[i] != text[j]) i = kmpNext[i];

        i++;
        j++;

        // Tìm thấy mẫu
        if (i >= m) {
            cout << "Match at position: " << (j - i) << endl;
            // Quay lại vị trí kmpNext của i và tiếp tục tìm kiếm
            i = kmpNext[i];
        }
    }
}
```

2.6 Thuật toán Apostolico-Crochemore

2.6.1 Giới thiệu thuật toán

Thuật toán Apostolico–Crochemore là một thuật toán tìm kiếm chuỗi cải tiến dựa trên nền tảng của Knuth–Morris–Pratt (KMP), nhằm tối ưu số phép so sánh ký tự trong quá trình tìm kiếm bằng cách kết hợp cơ chế dịch chuyển của bảng *kmpNext* với chiến lược so khớp hai pha trên mẫu.

Khác với KMP chỉ duy trì một trạng thái duy nhất trong quá trình so khớp, thuật toán Apostolico–Crochemore sử dụng thêm một cơ chế phân tách vị trí so sánh trong mẫu thông qua tham số ℓ (ell), từ đó chia mẫu thành hai vùng xử lý: vùng tiền tố và vùng hậu tố. Điều này giúp thuật toán tận dụng được thông tin đã so khớp ở cả đầu và giữa mẫu, giảm đáng kể số lần so sánh lại các ký tự.

Ý tưởng cốt lõi của thuật toán là: trong mỗi cửa sổ trên văn bản, thuật toán không chỉ tiến hành so khớp tuần tự như KMP mà còn khai thác kết quả tiền xử lý của *kmpNext* để quyết định thứ tự so sánh tối ưu giữa các vị trí trong mẫu, đồng thời cập nhật nhanh vị trí dịch chuyển khi xảy ra sai khác.

2.6.2 Quy trình thực hiện thuật toán

Giả sử có chuỗi văn bản y độ dài n và chuỗi mẫu x độ dài m . Thuật toán Apostolico–Crochemore được thực hiện theo quy trình sau.

2.6.2.1 Tiền xử lý mẫu: lập bảng *kmpNext* và vị trí ℓ

(1) Thuật toán xây dựng bảng *kmpNext* cho mẫu x , tương tự như trong thuật toán KMP. Bảng này lưu thông tin về border có điều kiện của các tiền tố, dùng để xác định vị trí quay lui khi xảy ra sai khác.

(2) Thuật toán xác định vị trí đặc biệt ℓ :

- Nếu $x = c^m$ (mẫu chỉ gồm một ký tự lặp lại) thì $\ell = 0$
- Ngược lại, ℓ là vị trí đầu tiên sao cho ký tự tại đó khác $x[0]$

Từ đó, mẫu được chia thành hai vùng: trái từ 0 đến $\ell - 1$, phải từ ℓ đến $m - 1$ nhằm tối ưu thứ tự so sánh trong quá trình tìm kiếm.

2.6.2.2 Khởi tạo quy trình tìm kiếm

Thuật toán sử dụng một cửa sổ độ dài m trượt trên chuỗi văn bản y . Tại mỗi vị trí bắt đầu j , thuật toán so sánh mẫu x với đoạn $y[j..j + m - 1]$.

Thuật toán duy trì một trạng thái ba thành phần: i, j, k , trong đó:

- j là vị trí bắt đầu cửa sổ trên văn bản
- k là độ dài prefix của mẫu đã khớp, thỏa mãn điều kiện

$$x[0..k-1] = y[j..j+k-1]; 0 \leq k \leq \ell$$

- i là vị trí đang so sánh trong mẫu, với: $\ell \leq i \leq m$

Thuật toán khởi tạo giá trị ban đầu của bộ 3 (i, j, k) là: $(i, j, k) = (\ell, 0, 0)$

2.6.2.3 Quy trình so khớp và tìm kiếm

Trong mỗi cửa sổ, các ký tự được so sánh theo thứ tự ưu tiên vùng phải trước $\ell, \ell + 1, \dots, m - 1$. Nếu khớp toàn bộ vùng phải, quay lại duyệt vùng trái $0, 1, \dots, \ell - 1$.

Khi xuất hiện sai khác, thuật toán sử dụng bảng *kmpNext* để quyết định dịch chuyển. Xử lý sai khác theo các trường hợp sau:

Trường hợp	Điều kiện	Giải pháp
$i = \ell$	$x[i] = y[j + i]$	So sánh ký tự tiếp theo trong mẫu: $(i, j, k) \rightarrow (i + 1, j, k)$
	$x[i] \neq y[j + i]$	Chuyển sang cửa sổ mới, quay lại vị trí bắt đầu vùng phải của mẫu và dịch cửa sổ sang phải: $(i, j, k) \leftarrow (\ell, j + 1, \max(0, k - 1))$
$\ell < i < m$	$x[i] = y[j + i]$	So sánh ký tự tiếp theo trong mẫu: $(i, j, k) \rightarrow (i + 1, j, k)$
	$x[i] \neq y[j + i]$ và $kmpNext[i] \leq \ell$	Tái khởi tạo trạng thái theo border ngắn, dịch cửa sổ sang phải và quay lại vùng phải của mẫu: $(i, j, k) \leftarrow (\ell, j + i - kmpNext[i], \max(0, kmpNext[i]))$
	$x[i] \neq y[j + i]$ và $kmpNext[i] > \ell$	Tận dụng border dài hơn, giữ lại một phần so khớp lớn hơn để tránh so sánh lại: $(i, j, k) \leftarrow (kmpNext[i], j + i - kmpNext[i], \ell)$
$i = m$	$k < \ell$	Tiếp tục so sánh prefix: kiểm tra $x[k]$ với $y[j + k]$. Nếu $x[k] = y[j + k]$ thì $(i, j, k) \rightarrow (i, j, k + 1)$, Nếu ngược lại xử lý như $i = \ell$
	$k = \ell$	Ghi nhận một lần xuất hiện của mẫu tại vị trí j . Cập nhật trạng thái theo kmpNext tương tự điều kiện sai khác của trường hợp $(\ell < i < m)$

Thuật toán lặp lại quá trình so khớp cho đến khi cửa sổ vượt độ dài chuỗi văn bản y .

2.6.3 Đánh giá độ phức tạp thuật toán

2.6.3.1 Độ phức tạp không gian

Thuật toán cần lưu bảng *kmpNext* kích thước m , một số biến trạng thái (i, j, k, ℓ) . Do đó, độ phức tạp không gian của thuật toán là: $S(n) = O(m)$

2.6.3.2 Độ phức tạp thời gian

(1) Giai đoạn tiền xử lý: Thuật toán xây dựng bảng *kmpNext* và xác định ℓ . Mỗi vị trí trong mẫu được xử lý với số lần hữu hạn nhờ cơ chế quay lui theo border, do đó:

$$T_{\text{preprocessing}} = O(m)$$

(2) Giai đoạn tìm kiếm:

- Thuật toán duyệt chuỗi văn bản từ trái sang phải với cơ chế dịch chuyển của *kmpNext*. Nhờ đó, mỗi ký tự trong văn bản chỉ tham gia so sánh một số hữu hạn lần, do vậy:

$$T_{\text{search}} = O(n)$$

- Số phép so sánh: trong trường hợp xấu nhất, thuật toán Apostolico – Crochemore thực hiện không quá $\frac{3}{2}n$ phép so sánh ký tự.

2.6.4 Kiểm nghiệm thuật toán

2.6.4.1 Dữ liệu vào

Cho hai chuỗi dưới đây. Thực hiện tìm kiếm mẫu x trong văn bản y

$x = \text{"aaba"}$
 $y = \text{"aaabaaabaabaa"}$

2.6.4.2 Quy trình kiểm nghiệm thuật toán

(1) Giai đoạn tiền xử lý: tính toán mảng *kmpNext*

Text and Pattern character														Full string	
pos j	0	1	2	3	4	5	6	7	8	9	10	11	12		13
y[j]	a	a	b	a	a	b	a	a	c	a	a	b	a	a	aabaabaacaabaa
x[j]	a	a	b	a											aaba

Xác định border				
i	x[0..i-1]	Prefix	Suffix	Border
0	""	Không	Không	Không
1	a	""	""	""
2	aa	"" , a	"" , a	"" , a
3	aab	"" , a, aa	"" , b, ab	""
4	aaba	"" , a, aa, aaa	"" , a, ba, aba	"" , a

Xác định l để chia vùng	
l = Vị trí đầu tiên khác x[0] = 2	
Vùng trái [0, l-1]	[0, 1]
Vùng phải [l, m-1]	[2, 3]

Duyệt từng border để xác định kmpNext[i]						
i	Border	x[border.size()] (1)	x[i] (2)	isTaggedBorder? = ((1) != (2))	Tagged Border	kmpNext[i] = max(taggedBorder.size())
0	Không				Không	-1
1	""	a	a	false	Không	-1
2	""	a	b	true	"", a	1
2	a	a	b	true		
3	""	a	a	false	Không	-1
4	""	a		true	"", a	1
4	a	a		true		

Mảng kmpNext	
i	kmpNext[i]
0	-1
1	-1
2	1
3	-1
4	1

(2) Giai đoạn tìm kiếm:

Kiểm nghiệm thuật toán										
Bước	i	j	k	y[j..j+m-1]	x[i]=y[j+i]?	i (new)	j (new)	k (new)	Giải thích	Output
1	2	0	0	aaba	b==b (true)	3	0	0	i = l và match → áp dụng (i,j,k) → (i+1,j,k).	
2	3	0	0	aaba	a==a (true)	4	0	0	l < i < m và match → tiếp tục tăng i.	
3	4	0	0	aaba	a==a (true)	4	0	1	i = m, k < l và prefix match → tăng k.	
4	4	0	1	aaba	a==a (true)	4	0	2	i = m, k < l và prefix match → tăng k.	
5	4	0	2			2	3	1	i = m, k = l → ghi nhận match tại j = 0. kmpNext[4] = 1 ≤ l nên cập nhật trạng thái thành (2,3,1).	0
6	2	3	1	aaba	b==b (true)	3	3	1	i = l và match → tăng i.	
7	3	3	1	aaba	a==a (true)	4	3	1	l < i < m và match → tăng i.	
8	4	3	1	aaba	a==a (true)	4	3	2	i = m, k < l và prefix match → tăng k.	
9	4	3	2			2	6	1	i = m, k = l → ghi nhận match tại j = 3. kmpNext[4] = 1 ≤ l nên cập nhật trạng thái thành (2,6,1).	3
10	2	6	1	aaca	b==c (false)	2	7	0	i = l và mismatch → dịch cửa sổ sang phải: (2,7,0).	
11	2	7	0	acaa	b==a (false)	2	8	0	i = l và mismatch → tiếp tục dịch cửa sổ: (2,8,0).	
12	2	8	0	caab	b==a (false)	2	9	0	i = l và mismatch → tiếp tục dịch cửa sổ: (2,9,0).	
13	2	9	0	aaba	b==b (true)	3	9	0	i = l và match → tăng i.	
14	3	9	0	aaba	a==a (true)	4	9	0	l < i < m và match → tăng i.	
15	4	9	0	aaba	a==a (true)	4	9	1	i = m, k < l và prefix match → tăng k.	
16	4	9	1	aaba	a==a (true)	4	9	2	i = m, k < l và prefix match → tăng k.	
17	4	9	2			2	12	1	i = m, k = l → ghi nhận match tại j = 9. kmpNext[4] = 1 ≤ l nên cập nhật trạng thái thành (2,12,1).	9
18	2	12	1						Cửa sổ vượt quá độ dài văn bản → kết thúc thuật toán.	

Sau khi kiểm nghiệm, kết quả cho thấy mẫu x="aaba" tồn tại trong văn bản y="aaabaaabaaabaa" tại 3 vị trí là {0, 3, 9}

2.6.5 Cài đặt thuật toán với C++

2.6.5.1 Hàm tiền xử lý: lập bảng kmpNext (xem thuật toán KMP)

2.6.5.2 Hàm tìm kiếm bằng thuật toán Apostolico-Crochemore

```
// Hàm tìm kiếm Apostolico-Crochemore
void apostolicoCrochemore(const string& pattern, const string& text) {
    int m = pattern.length();
    int n = text.length();

    vector<int> kmpNext = preKmp(pattern);
    int i, j, k, ell;
```

```
// Tính toán giá trị ell đặc biệt
for (ell = 1; pattern[ell - 1] == pattern[ell]; ell++);
if (ell == m) ell = 0;

// Thiết lập trạng thái ban đầu cho bộ ba (i, j, k)
i = ell;
j = k = 0;
while (j <= n - m) {
    // Pha 1: So khớp vùng phải (Hậu tố) từ vị trí i
    while (i < m && pattern[i] == text[i + j]) ++i;

    // Pha 2: Nếu vùng phải khớp hoàn toàn, kiểm tra vùng trái từ vị trí k
    if (i >= m) {
        while (k < ell && pattern[k] == text[j + k]) ++k;
        // Nếu vùng trái cũng khớp, tìm thấy mẫu tại vị trí j
        if (k >= ell) cout << "Match at position: " << j << endl;
    }

    // Cập nhật dịch chuyển cửa sổ và bộ ba trạng thái
    j += (i - kmpNext[i]);
    if (i == ell) k = max(0, k - 1);
    else if (kmpNext[i] <= ell) {
        k = max(0, kmpNext[i]);
        i = ell;
    }
    else {
        k = ell;
        i = kmpNext[i];
    }
}
}
```

CHƯƠNG 3: CÁC THUẬT TOÁN TÌM KIẾM MẪU TỪ PHẢI SANG TRÁI

3.1 Thuật toán Apostolico-Giancarlo

3.1.1 Giới thiệu thuật toán

Thuật toán Apostolico-Giancarlo là một biến thể cải tiến của thuật toán Boyer-Moore, được đề xuất nhằm khắc phục hạn chế của Boyer-Moore trong việc quên các ký tự đã khớp sau mỗi lần dịch chuyển mẫu.

Trong thuật toán Boyer-Moore, sau mỗi lần thử khớp (attempt), mặc dù đã biết một đoạn hậu tố của mẫu trùng với văn bản, thuật toán vẫn có thể phải so sánh lại các ký tự đó ở lần thử tiếp theo. Điều này làm xuất hiện nhiều phép so sánh dư thừa trong trường hợp xấu nhất.

Thuật toán Apostolico-Giancarlo cải tiến vấn đề này bằng cách bổ sung bảng skip để ghi nhớ độ dài đoạn hậu tố của mẫu đã được xác nhận khớp với văn bản ở các lần thử trước. Nhờ đó, thuật toán có thể bỏ qua một số phép so sánh không cần thiết trong các lần tìm kiếm tiếp theo.

Ngoài bảng *skip*, thuật toán còn sử dụng bảng *suff*, chứa thông tin về các hậu tố của mẫu. Các dữ liệu này hỗ trợ thuật toán xác định chính xác vị trí sai khác và lựa chọn bước dịch chuyển tối ưu.

Ý tưởng cốt lõi của thuật toán là khi một đoạn hậu tố của mẫu đã được xác nhận khớp với văn bản ở một lần thử trước, thuật toán sẽ ghi nhớ kết quả đó để tránh phải so sánh lại trong các lần thử sau.

3.1.2 Các thành phần của thuật toán Boyer-Moore được sử dụng

Thuật toán Apostolico-Giancarlo kế thừa trực tiếp hai cơ chế dịch chuyển của thuật toán Boyer-Moore

3.1.2.1 Mảng Bad Character Heuristic

Khi xảy ra sai khác tại vị trí i : $x[i] \neq y[i + j]$, thuật toán sử dụng ký tự gây sai khác $y[i + j]$ để xác định khoảng dịch chuyển.

Bảng Bad Character được định nghĩa như sau:

$$bmBc[c] = \begin{cases} m & \text{nếu } c \text{ không xuất hiện trong mẫu} \\ m - i - 1 & \text{nếu } c = x[i] \end{cases}$$

Ý nghĩa của bảng như sau:

- Nếu ký tự gây sai khác không xuất hiện trong phần bên trái của mẫu, mẫu dịch vượt qua vị trí hiện tại.

- Nếu ký tự đó xuất hiện trong mẫu, mẫu được dịch sao cho lần xuất hiện gần nhất của ký tự đó trùng với vị trí đang xét trong văn bản.

3.1.2.2 Mảng hậu tố *suff*

Thuật toán Apostolico–Giancarlo sử dụng trực tiếp mảng *suff* của Boyer–Moore để xây dựng bảng good suffix và hỗ trợ quá trình tìm kiếm.

Mảng *suff* được định nghĩa: $suff[i] = \max \{k \mid x[i - k + 1..i] = x[m - k..m - 1]\}$

Trong đó: *suff*[*i*] biểu diễn độ dài lớn nhất của đoạn hậu tố kết thúc tại vị trí *i* đồng thời trùng với hậu tố của toàn bộ mẫu.

Thuật toán Apostolico–Giancarlo không thay đổi cách tính *suff*, mà kế thừa trực tiếp từ Boyer–Moore. Tuy nhiên, AG sử dụng thêm thông tin này trong cơ chế skip để tránh lặp lại các phép so sánh đã thực hiện trước đó.

3.1.2.3 Good Suffix Heuristic

Khi sai khác xảy ra tại vị trí *i*, đoạn: $x[i + 1 .. m - 1]$ đã khớp với văn bản được gọi là *good suffix*.

Thuật toán tìm cách dịch mẫu sao cho: hoặc đoạn hậu tố này xuất hiện lại trong mẫu, hoặc một tiền tố của mẫu khớp với hậu tố đã xét.

Bảng good suffix được xây dựng từ mảng *suff*. Mục tiêu của heuristic này là tận dụng tối đa phần đã khớp để giảm số lần so sánh lại.

3.1.3 Quy trình thực hiện thuật toán

Giả sử có chuỗi văn bản *y* độ dài *n* và chuỗi mẫu *x* độ dài *m*. Thuật toán Apostolico–Giancarlo được thực hiện theo quy trình sau.

3.1.3.1 Tiền xử lý mẫu: lập ba bảng *bmBc*, *suff*, *bmGs*

(1) Bảng Bad Character *bmBc*: bảng này lưu khoảng dịch chuyển tương ứng với từng ký tự trong bảng chữ cái.

$$bmBc[c] = \begin{cases} m & \text{nếu } c \text{ không xuất hiện trong mẫu} \\ m - i - 1 & \text{nếu } c = x[i] \end{cases}$$

(2) Bảng hậu tố *suff*:

Mảng *suff* được định nghĩa: $suff[i] = \max \{k \mid x[i..m - 1] = x[m - k..m - 1]\}$

Trong đó: *suff*[*i*] là độ dài lớn nhất của đoạn vừa là hậu tố của pattern kết thúc tại vị trí *i* đồng thời cũng vừa là hậu tố của toàn bộ mẫu. Đặc biệt: $suff[m - 1] = m$

(3) Bảng Good Suffix *bmGs*

Từ bảng *suff*, thuật toán xây dựng bảng: *bmGs[i]*, trong đó: *bmGs[i]* biểu diễn khoảng dịch chuyển khi sai khác xảy ra tại vị trí *i*. Bảng *bmGs* được xây dựng như sau

Bước 1: Khởi tạo: gán toàn bộ phần tử của bảng <i>bmGs</i> bằng độ dài mẫu: $bmGs[i] = m$ Bước 2: Xử lý trường hợp hậu tố cũng là tiền tố - Duyệt mảng <i>suff</i> từ phải sang trái. - Nếu $suff[i] = i + 1$ thì hậu tố bắt đầu tại vị trí <i>i</i> cũng đồng thời cũng là tiền tố của mẫu. - Các vị trí chưa được cập nhật trong <i>bmGs</i> sẽ được gán: $bmGs[j] = m - 1 - i$ Bước 3: Xử lý trường hợp hậu tố xuất hiện lại trong mẫu, duyệt các vị trí từ trái sang phải và cập nhật: $bmGs[m - 1 - suff[i]] = m - 1 - i$ Bước này giúp xác định khoảng dịch chuyển khi hậu tố đã khớp xuất hiện lại ở vị trí khác trong mẫu.
--

(4) Bảng ghi nhớ *skip*: gán $skip[t] = 0$ với mọi vị trí *t*. Trong quá trình so khớp, bảng *skip* dùng để lưu thông tin sau:

$$skip[j + m - 1] = \max \{k \mid x[m - k..m - 1] = y[j + m - k..j + m - 1]\}$$

Trong đó: $skip[j + m - 1]$ là độ dài hậu tố của mẫu đã được xác nhận khớp với văn bản tại lần thử bắt đầu ở vị trí *j*.

3.1.3.2 Khởi tạo quy trình tìm kiếm

Đặt cửa sổ mẫu tại vị trí: $y[j..j + m - 1]$

Thuật toán thực hiện so sánh từ phải sang trái: $x[m - 1] \rightarrow x[m - 2] \rightarrow \dots \rightarrow x[0]$

3.1.3.3 Quy trình so khớp và tìm kiếm

Tại vị trí đang xét *i*, thuật toán sử dụng: $k = skip[i]$ và $suff[i]$ để quyết định cách xử lý tiếp theo.

Trong quá trình này, nếu không có thông tin từ *skip*, thuật toán thực hiện so sánh trực tiếp giữa $x[i]$ và $y[i + j]$

Trong quá trình so khớp, thuật toán chia thành bốn trường hợp.

(1) Trường hợp 1: tìm thấy mẫu hoàn chỉnh tại vị trí *j* của văn bản

- Điều kiện: $k > suff[i]$ và $suff[i] = i + 1$
- Thuật toán thực hiện:

- Ghi nhận tìm thấy mẫu tại vị trí j của văn bản
- Cập nhật: $skip[j + m - 1] = m$
- Dịch chuyển: $shift = bmGs[0]$

(2) Trường hợp 2: $k > suff[i]$ và $suff[i] \leq i$

- Trường hợp sai khác xảy ra tại: $x[i - suff[i]] \neq y[i + j - suff[i]]$
- Cập nhật: $skip[j + m - 1] = m - 1 - i + suff[i]$
- Dịch chuyển:

$$shift = \max(bmGs[i], bmBc[y[i + j - suff[i]]] - m + 1 + i)$$

(3) Trường hợp 3: $k < suff[i]$

- Sai khác xảy ra tại: $x[i - k] \neq y[i + j - k]$
- Cập nhật: $skip[j + m - 1] = m - 1 - i + k$
- Dịch chuyển theo: $shift = \max(bmGs[i], bmBc[y[i + j - k]] - m + 1 + i)$

(4) Trường hợp 4: $k = suff[i]$ (trường hợp đặc biệt)

Đoạn hậu tố đã khớp hoàn toàn phù hợp với cấu trúc hậu tố của mẫu, thuật toán không cần so sánh lại đoạn đã biết khớp, bỏ qua đoạn: $y[i + j - k + 1..i + j]$

Thuật toán tiếp tục so sánh: $x[i - k]$ với $y[i + j - k]$

3.1.3.4 Cập nhật và tiếp tục tìm kiếm

Sau mỗi lần xử lý, cửa sổ được dịch sang phải theo giá trị: $shift = \max(bmBc, bmGs)$

Trường hợp sau khi dịch: nếu phần của bảng skip vẫn còn hữu ích, thuật toán dịch chuyển lại các giá trị tương ứng. Ngược lại các giá trị được đặt về 0.

Quá trình lặp lại cho đến khi: $j > n - m$ tức là cửa sổ mẫu vượt khỏi phạm vi văn bản.

3.1.4 Đánh giá độ phức tạp thuật toán

3.1.4.1 Độ phức tạp không gian

Thuật toán Apostolico-Giancarlo sử dụng một số cấu trúc dữ liệu chính

- Bảng $suff$ có kích thước m , lưu thông tin so khớp hậu tố của mẫu.
- Bảng $skip$ có kích thước tối đa m , lưu trong cửa sổ trượt khi tìm kiếm.
- Các bảng dịch chuyển (Bad Character và Good Suffix) kế thừa từ Boyer-Moore, cần không gian $O(\sigma)$.

Do đó, tổng không gian bộ nhớ cần thiết là: $S(n) = O(m + \sigma)$, với m là độ dài mẫu, σ là kích thước bảng ký tự

3.1.4.2 Độ phức tạp thời gian

(1) Giai đoạn tiền xử lý: trong giai đoạn này, thuật toán xây dựng: bảng suff, các bảng dịch chuyển Bad Character và Good Suffix.

Mỗi ký tự trong mẫu được xử lý với số lần hữu hạn trong quá trình tính toán các bảng phụ trợ, do đó độ phức tạp thời gian ở giai đoạn tiền xử lý là: $T_{preprocessing} = O(m + \sigma)$

(2) Giai đoạn tìm kiếm: trong quá trình tìm kiếm, mẫu được trượt trên văn bản từ trái sang phải. Ở mỗi vị trí, thuật toán so sánh ký tự từ phải sang trái và sử dụng:

- Thông tin từ bảng suff
- Thông tin đã ghi nhớ trong bảng skip
- Các quy tắc dịch chuyển của Boyer–Moore

Điểm quan trọng của thuật toán Apostolico-Giancarlo là:

- Các đoạn hậu tố đã từng được so khớp sẽ không bị so sánh lại
- Do đó giảm đáng kể số lần lặp lại phép so sánh giữa các cửa sổ chồng lấn

Nhờ cơ chế này, mỗi ký tự trong văn bản chỉ bị tham gia vào một số lượng hữu hạn phép so sánh. Do vậy, độ phức tạp thuật toán tại pha tìm kiếm là: $T_{search} = O(n)$

(3) Thuật toán Apostolico-Giancarlo đảm bảo số phép so sánh ký tự không quá: $\frac{3}{2}n$

3.1.5 Kiểm nghiệm thuật toán

3.1.5.1 Dữ liệu vào

Cho hai xâu dưới đây. Thực hiện tìm kiếm mẫu x trong văn bản y

x="aaba"
y="aaabaaabaaabaa"

3.1.5.2 Quy trình kiểm nghiệm thuật toán

(1) Giai đoạn tiền xử lý: tính toán bảng bmBc, suff, bmGs

Text and Pattern character															Full string
pos j	0	1	2	3	4	5	6	7	8	9	10	11	12	13	
y[j]	a	a	b	a	a	b	a	a	c	a	a	b	a	a	aabaabaacaabaa
x[j]	a	a	b	a											aaba

CHƯƠNG 3: CÁC THUẬT TOÁN TÌM KIẾM MẪU TỪ PHẢI SANG TRÁI

Chạy bảng bmBc						
Bước	i	pattern [i]	bmBc[a]	bmBc[b]	bmBc [khác]	Giải thích
0	-	-	4	4	4	
1	0	a	3	4	4	bmBc[a]=4-1-0=3
2	1	a	2	4	4	bmBc[a]=4-1-1=2
3	2	b	2	1	4	bmBc[b] = 4-1-2=1

Bảng bmBc	
chr	bmBc [chr]
a	2
b	1
khác	4

Bảng suff: suff[i]=maxk x[i-k+1...i]=x[m-k...m-1]				
i	x[i]	suffix x[0..i]	Hệ số x khớp dài nhất với suffix x[0..i]	Suff[i]
0	a	a	a	1
1	a	a, aa	a	1
2	b	b, ab, aab	Không	0
3	a	a, ba, aba, aaba	aaba	4

Chạy bảng bmGs						
Bước	i	j	bmGs trước	Điều kiện	Cập nhật	bmGs sau
1	init	-	[4, 4, 4, 4]			
2	2	-	[4, 4, 4, 4]	suff[2] = 0 != 2+1		[4, 4, 4, 4]
3	1	-	[4, 4, 4, 4]	suff[1] = 1 != 1+1		[4, 4, 4, 4]
4	0	j = 0	[4, 4, 4, 4]	suff[0] = 1 == 0+1	bmGs[j], j=0..2	[3, 3, 3, 4]
5	suffix i=0	2	[3, 3, 3, 4]	m - 1 - suff[0] = 2	bmGs[2] = 3	[3, 3, 3, 4]
6	suffix i=1	2	[3, 3, 3, 4]	m - 1 - suff[1] = 2	bmGs[2] = 2	[3, 3, 2, 4]
7	suffix i=2	3	[3, 3, 2, 4]	m - 1 - suff[2] = 3	bmGs[3] = 1	[3, 3, 2, 1]

Bảng bmGs	
i	bmGs[i]
0	3
1	3
2	2
3	1

(2) Giai đoạn tìm kiếm

Kiểm nghiệm thuật toán													
Bước	j	i	x[i]	y[j+i]	k = skip[i]	suff[i]	x[i] == y[j+i]?	skip [j+m-1]	i next	j next	Giải thích i next	Giải thích j next	Output
1	0	3	a	a	0	1	TRUE	-	2	0	i-- do khớp	tiếp tục	
2	0	2	b	b	0	0	TRUE	-	1	0	i-- do khớp	tiếp tục	
3	0	1	a	a	0	0	TRUE	-	0	0	i-- do khớp	tiếp tục	
4	0	0	a	a	0	1	MATCH	4	-	3	i<0 (match xong)	j += bmGs[0] = 0 + 3 = 3	0
5	3	3	a	a	4	1	TRUE	-	2	3	i-- do khớp	tiếp tục	
6	3	2	b	b	2	0	TRUE	-	1	3	i-- do khớp	tiếp tục	
7	3	1	a	a	0	0	TRUE	-	0	3	i-- do khớp	tiếp tục	
8	3	0	a	a	0	1	MATCH	4	-	6	i<0 (match xong)	j += bmGs[0] = 3 + 3 = 6	3
9	6	3	a	a	0	1	TRUE	-	2	6	i--	tiếp tục	
10	6	2	b	b	0	0	TRUE	-	1	6	i--	tiếp tục	
11	6	1	a	c	0	0	FALSE	1	-	9	mismatch	j += max(bmBc -(m-1-i),bmGs)	
12	9	3	a	a	0	1	TRUE	-	2	9	i--	tiếp tục	
13	9	2	b	b	0	0	TRUE	-	1	9	i--	tiếp tục	
14	9	1	a	a	0	0	TRUE	-	0	9	i--	tiếp tục	
15	9	0	a	a	0	1	MATCH	4	-	12	i<0	j += bmGs[0] = 9 + 3 = 12	9

Sau khi kiểm nghiệm, kết quả cho thấy mẫu x="aaba" tồn tại trong văn bản y="aaabaaabaaabaa" tại 3 vị trí là {0, 3, 9}

3.1.6 Cài đặt thuật toán với C++

3.1.6.1 Hàm tiền xử lý: lập bảng bmBc, bmGs, suff (kế thừa từ thuật toán Boyer-Moore)

```

const int ASIZE = 256;

// Tiền xử lý bảng bad character
void preBmBc(const string &pattern, const string &text, vector<int> &bmBc) {
    int m = pattern.size();
    // Khởi tạo bảng bad character với giá trị mặc định là m
    bmBc.assign(ASIZE, m);

    for (int i = 0; i < m - 1; i++) {
        // Lưu khoảng dịch chuyển của ký tự pattern[i]
        bmBc[(unsigned char)pattern[i]] = m - i - 1;
    }
}

// Tính mảng suff
void suffixes(const string &pattern, const string &text, vector<int> &suff) {
    int m = pattern.size();
    suff.assign(m, 0);

    int f = 0, g = m - 1;

    // Suffix dài nhất kết thúc tại vị trí m-1 là chính nó
    suff[m - 1] = m;

    for (int i = m - 2; i >= 0; i--) {
        // Tái sử dụng kết quả đã tính trước đó
        if (i > g && suff[i + m - 1 - f] < i - g) {
            // Suffix tại i có độ dài bằng suff tại vị trí tương ứng
            suff[i] = suff[i + m - 1 - f];
        } else {
            // Mở rộng đoạn khớp suffix
            if (i < g) g = i;
            f = i;

            while (g >= 0 && pattern[g] == pattern[g + m - 1 - f]) g--;

            // Tính độ dài suffix khớp tại vị trí i
            suff[i] = f - g;
        }
    }
}

// Tiền xử lý bảng good suffix
void preBmGs(const string &pattern, const string &text, const vector<int> &suff,
vector<int> &bmGs) {
    int m = pattern.size();

    // Khởi tạo bảng good suffix với giá trị mặc định là m
    bmGs.assign(m, m);

    int j = 0;

```

```
// Xử lý trường hợp suffix cũng là prefix
for (int i = m - 1; i >= 0; i--) {
    // Nếu suff[i] bằng i + 1, nghĩa là suffix tại i cũng là prefix
    if (suff[i] == i + 1) {
        for (; j < m - 1 - i; j++) {
            // Cập nhật bước dịch cho các vị trí chưa được cập nhật
            if (bmGs[j] == m) bmGs[j] = m - 1 - i;
        }
    }
}

// Xử lý trường hợp suffix xuất hiện lại bên trong mẫu
for (int i = 0; i <= m - 2; i++) {
    bmGs[m - 1 - suff[i]] = m - 1 - i;
}
}
```

3.1.6.2 Cài đặt thuật toán Apostolico-Giancarlo

```
// Thuật toán Apostolico-Giancarlo
void apostolicoGiancarlo(const string &pattern, const string &text) {
    int m = pattern.size();
    int n = text.size();

    vector<int> bmBc, bmGs, suff, skip;

    preBmBc(pattern, text, bmBc);
    suffixes(pattern, text, suff);
    preBmGs(pattern, text, suff, bmGs);

    // Khởi tạo bảng skip
    skip.assign(m, 0);

    int j = 0;
    while (j <= n - m) {
        // So khớp từ phải sang trái
        int i = m - 1;

        int shift = 1;

        while (i >= 0) {
            int k = skip[i];
            int s = suff[i];
            // Có thông tin skip từ lần dò trước
            if (k > 0) {
                // Đoạn đã biết khớp dài hơn suffix của pattern tại i
                if (k > s) {
                    if (i + 1 == s) i = -1;
                    else i -= s;
                    break;
                }
            }
        }
    }
}
```

```
        else {
            i -= k;
            if (k < s) break;
        }
    }
    // Không có thông tin skip, so sánh trực tiếp
    else {
        if (pattern[i] == text[i + j]) i--;
        // Nếu không khớp, dừng lại
        else break;
    }
}

// Tìm thấy pattern
if (i < 0) {
    cout << "Match at position: " << j << "\n";
    // Cập nhật bảng skip cho lần dò tiếp theo
    skip[m - 1] = m;
    // Dịch pattern sang phải để tìm kiếm lần tiếp theo
    shift = bmGs[0];
}
// Không tìm thấy pattern, tính bước dịch theo bmBc và bmGs
else {
    // Lưu độ dài suffix đã khớp
    skip[m - 1] = m - 1 - i;

    // Chọn bước dịch lớn nhất giữa bad character và good suffix
    shift = max(bmGs[i], bmBc[(unsigned char)text[i + j]] - m + 1 + i);
}
j += shift;

// Cập nhật lại bảng skip sau khi dịch
if (shift < m) {
    for (int t = 0; t < m - shift; t++) skip[t] = skip[t + shift];
    for (int t = m - shift; t < m; t++) skip[t] = 0;
}
// Nếu dịch đủ xa để không còn thông tin skip, reset bảng skip
else fill(skip.begin(), skip.end(), 0);
}
```

3.2 Thuật toán Turbo BM

3.2.1 Giới thiệu thuật toán

Thuật toán Turbo Boyer–Moore (Turbo-BM) là một cải tiến của Boyer–Moore, được thiết kế nhằm giảm số phép so sánh dư thừa trong quá trình tìm kiếm mẫu bằng cách ghi nhớ phần hậu tố đã khớp của lần dò trước đó.

Khác với Boyer–Moore truyền thống, Turbo-BM không chỉ dựa vào hai heuristic dịch chuyển quen thuộc (bad character và good suffix), mà còn bổ sung thêm một cơ chế bộ nhớ.

(1) Ghi lại độ dài đoạn hậu tố của mẫu đã khớp với văn bản ở lần dò trước.

(2) Sử dụng thông tin này để: bỏ qua các phép so sánh lặp lại, và có thể thực hiện các bước nhảy lớn hơn (turbo shift).

Ý tưởng cốt lõi là: nếu một đoạn hậu tố đã từng được khớp, thuật toán không cần kiểm tra lại toàn bộ đoạn đó trong lần dò tiếp theo, đồng thời có thể tận dụng mối quan hệ chồng lấp giữa các lần khớp để tăng bước dịch chuyển.

3.2.2 Quy trình thực hiện thuật toán

Giả sử có chuỗi văn bản y độ dài n và chuỗi mẫu x độ dài m . Thuật toán Apostolico-Giancarlo được thực hiện theo quy trình sau.

3.2.2.1 Tiền xử lý mẫu: lập bảng $bmBc$ và $bmGs$

(1) Xây dựng bảng $bmBc$ (bad character) và $bmGs$ (good suffix) từ mẫu x (xem lại thuật toán Apostolico-Giancarlo)

(2) Thuật toán khởi tạo biến ghi nhớ $u = 0$, trong đó u là độ dài hậu tố của mẫu đã khớp ở lần dò trước.

(3) Thuật toán đặt vị trí bắt đầu tìm kiếm: $j = 0$

3.2.2.2 Quy trình so khớp và tìm kiếm

Tại vị trí j , tiến hành so sánh các ký tự của mẫu với văn bản từ phải sang trái xét từ $i = m - 1$ lùi về 0 ($x[i]$ với $y[i + j]$).

(1) Kiểm tra khả năng nhảy đoạn:

Trong quá trình so sánh, nếu $u > 0$ và con trỏ i lùi đến vị trí phân đoạn đã biết là khớp từ lần thử trước ($i == m - 1 - shift$), thuật toán sẽ nhảy bỏ qua phân đoạn này bằng cách giảm i đi một khoảng đúng bằng u ($i = i - u$).

(2) Nếu so sánh thành công đến hết mẫu ($i < 0$), thuật toán thực hiện:

- Ghi nhận vị trí j .

- Cập nhật: $u = m - shift$
- Xác định bước dịch: $shift = bmGs[0]$

(3) Nếu có sai khác, tại vị trí sai i , xác định: $v = m - 1 - i$

- Thuật toán tính các bước dịch dựa trên các đại lượng:
 - Turbo shift: $turboShift = u - v$
 - Bad character shift: $bcShift = bmBc[y[i + j]] - m + 1 + i$
 - Good suffix shift: $gsShift = bmGs[i]$
- Chọn bước dịch lớn nhất giữa Turbo shift, Bad character, Good suffix shift:

$$shift = \max(turboShift, bcShift, gsShift).$$

(4) Cập nhật sau mỗi lần dịch

- Trường hợp 1: nếu bước dịch cuối cùng được chọn theo Good suffix ($shift = bmGs[i]$), thì cập nhật $u = \min(m - shift, v)$.
- Trường hợp 2: nếu bước dịch không chọn theo Good suffix:
 - Nếu $turboShift < bcShift$, nâng bước dịch lên $shift = \max(shift, u + 1)$ để tránh việc hai ký tự khác nhau trong mẫu bị ép trùng vị trí trên văn bản.
 - Thuật toán đặt lại biến ghi nhớ: $u = 0$.

(5) Cập nhật vị trí cửa sổ tiếp theo trên văn bản: $j = j + shift$. Sau đó, tiếp tục so khớp cho đến khi cửa sổ vượt quá giới hạn văn bản ($j > n - m$).

3.2.3 Đánh giá độ phức tạp thuật toán

3.2.3.1 Độ phức tạp không gian

Thuật toán Turbo-BM sử dụng các cấu trúc dữ liệu:

- Bảng $bmBc$ và $bmGs$ kế thừa từ thuật toán Boyer–Moore, có kích thước phụ thuộc vào bảng chữ cái σ .
- Biến ghi nhớ u lưu độ dài hậu tố đã khớp, chỉ sử dụng một biến đơn.
- Không sử dụng thêm cấu trúc dữ liệu động đáng kể.

Do đó, độ phức tạp không gian là: $S(n) = O(m + \sigma)$

3.2.3.2 Độ phức tạp thời gian

(1) Giai đoạn tiền xử lý: việc xây dựng các bảng bmBc và bmGs phụ thuộc trực tiếp vào độ dài mẫu m và bảng ký tự σ . Mỗi ký tự trong mẫu được xử lý hữu hạn lần. Do đó: $T_{preprocessing} = O(m + \sigma)$

(2) Giai đoạn tìm kiếm

Trong quá trình tìm kiếm, mẫu được trượt trên văn bản từ trái sang phải. Tại mỗi vị trí, thuật toán so sánh từ phải sang trái nhưng có các tối ưu sau:

- Bỏ qua một phần ký tự đã được ghi nhớ từ lần dò trước (u),
- Kết hợp bad character, good suffix và turbo shift để tăng bước dịch.

Nhờ cơ chế ghi nhớ hậu tố và tránh so sánh lặp lại, mỗi ký tự trong văn bản chỉ tham gia vào số lượng hữu hạn phép so sánh. Vì vậy: $T_{search} = O(n)$

(3) Turbo-BM giới hạn số phép so sánh ký tự trong trường hợp xấu nhất. Nhờ cơ chế turbo shift và ghi nhớ hậu tố, mỗi ký tự trong văn bản bị so sánh tối đa hằng số lần. Do đó, số phép so sánh được chặn bởi: $C(n) \leq 2n$

3.2.4 Kiểm nghiệm thuật toán

3.2.4.1 Dữ liệu vào

Cho hai xâu dưới đây. Thực hiện tìm kiếm mẫu x trong văn bản y

$x = \text{"aaba"}$
 $y = \text{"aaabaaabaaba"}$

3.2.4.2 Quy trình kiểm nghiệm thuật toán

(1) Giai đoạn tiền xử lý: lập bảng bmBc, suff, bmGs

Text and Pattern character														Full string	
pos j	0	1	2	3	4	5	6	7	8	9	10	11	12		13
y[j]	a	a	b	a	a	b	a	a	c	a	a	b	a	a	aabaabaacaabaa
x[j]	a	a	b	a											aaba

CHƯƠNG 3: CÁC THUẬT TOÁN TÌM KIẾM MẪU TỪ PHẢI SANG TRÁI

Chạy bảng bmBc						
Bước	i	pattern [i]	bmBc[a]	bmBc[b]	bmBc [khác]	Giải thích
0	-	-	4	4	4	
1	0	a	3	4	4	bmBc[a]=4-1-0=3
2	1	a	2	4	4	bmBc[a]=4-1-1=2
3	2	b	2	1	4	bmBc[b] = 4-1-2=1

Bảng bmBc	
chr	bmbc [chr]
a	2
b	1
khác	4

Bảng suff: suff[i]=max{x[i-k+1...i]=x[m-k...m-1]}				
i	x[i]	suffix x[0..i]	Hậu tố x khớp dài nhất với suffix x[0..i]	Suff[i]
0	a	a	a	1
1	a	a, aa	a	1
2	b	b, ab, aab	Không	0
3	a	a, ba, aba, aaba	aaba	4

Chạy bảng bmGs						
Bước	i	j	bmGs trước	Điều kiện	Cập nhật	bmGs sau
1	init	-	[4, 4, 4, 4]			
2	2	-	[4, 4, 4, 4]	suff[2] = 0 != 2+1		[4, 4, 4, 4]
3	1	-	[4, 4, 4, 4]	suff[1] = 1 != 1+1		[4, 4, 4, 4]
4	0	j = 0	[4, 4, 4, 4]	suff[0] = 1 == 0+1	bmGs[j], j=0..2	[3, 3, 3, 4]
5	suffix i=0	2	[3, 3, 3, 4]	m - 1 - suff[0] = 2	bmGs[2] = 3	[3, 3, 3, 4]
6	suffix i=1	2	[3, 3, 3, 4]	m - 1 - suff[1] = 2	bmGs[2] = 2	[3, 3, 2, 4]
7	suffix i=2	3	[3, 3, 2, 4]	m - 1 - suff[2] = 3	bmGs[3] = 1	[3, 3, 2, 1]

Bảng bmGs	
i	bmGs[i]
0	3
1	3
2	2
3	1

(2) Giai đoạn tìm kiếm

Kiểm nghiệm thuật toán											
Bước	j	i	x[i]	y[j+i]	Hành động	u	shift	Giải thích siêu ngắn	i mới	j mới	Output
1	0	3	a	a	Khớp	0		i-- do khớp	2	0	
2	0	2	b	b	Khớp	0		i-- do khớp	1	0	
3	0	1	a	a	Khớp	0		i-- do khớp	0	0	
4	0	0	a	a	Khớp	0		i-- do khớp	-1	0	
5	0	-1			MATCH	0	3	Khớp hết, dịch bmGs[0]=3. Lưu u=4-3=1.	3	3	0
6	3	3	a	a	Khớp	1		i-- do khớp	2	3	
7	3	2	b	b	Khớp	1		Không nhảy vì i != m - 1 - shift. i-- do khớp	1	3	
8	3	1	a	a	Khớp	1		i-- do khớp	0	3	
9	3	0	a	a	Khớp	1		i-- do khớp	-1	3	
10	3	-1			MATCH	1	3	Khớp hết, dịch bmGs[0]=3. Giữ u=4-3=1.	3	6	3
11	6	3	a	a	Khớp	1		i-- do khớp	2	6	
12	6	2	b	c	MISMATCH	1		Không khớp, dừng quét	2	6	
13	6	2			Tính dịch	1	3	max(bcShift, bmGs[2], u+1)=3. Reset u=0.	3	9	
14	9	3	a	a	Khớp	0		i-- do khớp	2	9	
15	9	2	b	b	Khớp	0		i-- do khớp (u=0 nên không nhảy).	1	9	
16	9	1	a	a	Khớp	0		i-- do khớp	0	9	
17	9	0	a	a	Khớp	0		i-- do khớp	-1	9	
18	9	-1			MATCH	0	3	Khớp hết, dịch bmGs[0]=3. Lưu u=4-3=1.	3	12	9
19	12				KẾT THÚC	1		Cửa sổ vượt text. Dừng thuật toán.	-	-	

Sau khi kiểm nghiệm, kết quả cho thấy mẫu x="aaba" tồn tại trong văn bản y="aaabaaabaaabaa" tại 3 vị trí là {0, 3, 9}

3.2.5 Cài đặt thuật toán với C++

3.2.5.1 *Hàm tiền xử lý: lập bảng bmBc, bmGs, suff (kế thừa từ thuật toán Boyer-Moore, xem lại phần cài đặt của thuật toán Apostolico-Giancarlo)*

3.2.5.2 Cài đặt thuật toán TurboBM

```
// Thuật toán Turbo Boyer-Moore tìm kiếm pattern trong text
void turboBM(const string &pattern, const string &text) {
    int m = pattern.size(), n = text.size();

    vector<int> bmBc, bmGs, suff;

    preBmBc(pattern, bmBc);
    suffixes(pattern, suff);
    preBmGs(pattern, suff, bmGs);

    int j = 0, u = 0; // lưu độ dài đoạn hậu tố đã khớp ở lần dò trước
    int shift = m;

    while (j <= n - m) {
        int i = m - 1;
        while (i >= 0 && pattern[i] == text[i + j]) {
            --i;
            // bỏ qua phần đã được xác nhận khớp từ lần dò trước
            if (u != 0 && i == m - 1 - shift) i -= u;
        }

        if (i < 0) {
            cout << "Match at position: " << j << "\n";
            shift = bmGs[0]; // dịch khi khớp hoàn toàn
            u = m - shift;   // cập nhật độ dài hậu tố khớp
        }
        else {
            int v = m - 1 - i; // độ dài hậu tố đã khớp trước mismatch

            int turboShift = u - v;
            int bcShift = bmBc[(unsigned char)text[i + j]] - m + 1 + i;

            shift = max({turboShift, bcShift, bmGs[i]});

            // giữ lại phần hậu tố có ích
            if (shift == bmGs[i]) u = min(m - shift, v);
            else {
                if (turboShift < bcShift) shift = max(shift, u + 1);
                u = 0; // reset nếu không dùng good suffix
            }
        }
        j += shift; // dịch cửa sổ sang phải
    }
}
```

3.3 Thuật toán Zhu-Takaoka

3.3.1 Giới thiệu thuật toán

Thuật toán Zhu-Takaoka (được đề xuất bởi Zhu và Takaoka vào năm 1987) là một biến thể mở rộng dựa trên cơ chế ký tự xấu (Bad-character shift) của thuật toán Boyer-Moore nguyên bản.

Hai tác giả nhận thấy rằng việc chỉ sử dụng một ký tự đơn lẻ tại vị trí xảy ra sai khác để tính toán khoảng dịch thường xuyên dẫn đến hiệu ứng trùng lặp, khiến cửa sổ chỉ trượt được những khoảng rất ngắn (đặc biệt là đối với các bảng chữ cái có kích thước nhỏ).

Để khắc phục triệt để nhược điểm này, Zhu-Takaoka đề xuất sử dụng hai ký tự văn bản liên tiếp tại cuối cửa sổ để tính toán khoảng dịch chuyển ký tự xấu. Thay vì dùng một mảng một chiều, thuật toán xây dựng một ma trận hai chiều đại diện cho mọi cặp ký tự có thể xảy ra. Ý tưởng này mang lại một số ưu điểm sau:

(1) Việc xét một cặp ký tự liên tiếp làm giảm đáng kể xác suất xuất hiện ngẫu nhiên của chúng trong mẫu, từ đó giúp thuật toán dễ dàng tìm được các khoảng trượt dài hơn, bỏ qua nhiều vị trí kiểm tra thừa.

(2) Thuật toán vẫn duy trì sự kết hợp song song với bảng dịch chuyển khớp hậu tố (bmGs), chọn khoảng trượt lớn nhất giữa hai cơ chế để đảm bảo tính tối ưu.

(3) Quá trình kiểm tra các ký tự bên trong cửa sổ được thực hiện một cách cổ điển từ phải sang trái, giúp thuật toán tận dụng sức mạnh của cấu trúc hậu tố tốt khi xảy ra sai lệch.

3.3.2 Quy trình thực hiện thuật toán

Giả sử có chuỗi văn bản y độ dài n và chuỗi mẫu x độ dài m . Quy trình của thuật toán bao gồm hai giai đoạn chính:

3.3.2.1 Tiền xử lý: lập bảng $ztBc$ và $bmGs$

(1) Xây dựng ma trận ký tự xấu hai chiều $ztBc$:

- Khởi tạo toàn bộ ma trận kích thước $|\Sigma| \times |\Sigma|$ với giá trị mặc định bằng chiều dài mẫu m .
- Với mọi ký tự i trong bảng chữ cái, thiết lập giá trị dịch chuyển cho các cặp kết thúc bởi ký tự đầu tiên của mẫu: $ztBc[i][x[0]] = m - 1$.
- Duyệt qua mẫu từ trái sang phải từ chỉ số 1 đến $m-2$. Tại mỗi vị trí, cập nhật khoảng cách của cặp ký tự liên tiếp $(x[i-1], x[i])$ tới đuôi mẫu: $ztBc[x[i-1]][x[i]] = m - 1 - i$

(2) Xây dựng mảng dịch chuyển hậu tố tốt $bmGs$: Tính toán mảng một chiều dựa trên cấu trúc các chuỗi con lặp lại của mẫu tương tự như thuật toán Boyer-Moore gốc.

3.3.2.2 Quá trình so khớp và tìm kiếm

Vị trí ban đầu của cửa sổ đặt tại chỉ số $j = 0$. Vòng lặp chính duy trì khi cửa sổ chưa vượt quá giới hạn văn bản ($j \leq n - m$)

(1) Thiết lập con trỏ quét bắt đầu từ cuối cửa sổ ($i = m - 1$). Thực hiện lùi dần về bên trái và so sánh từng ký tự:

- Nếu ký tự mẫu khớp với văn bản ($x[i] == y[j]$), thuật toán giảm i để kiểm tra tiếp ký tự phía trước.
- Lặp lại đến khi gặp ký tự không khớp hoặc quét hết toàn bộ mẫu ($i < 0$).

(2) Kiểm tra trạng thái khớp: dựa trên giá trị của con trỏ i sau khi thoát vòng lặp, đổi chiều như sau:

- Nếu $i < 0$ (khớp hoàn toàn): ghi nhận tìm thấy mẫu tại vị trí j và dịch chuyển cửa sổ dựa trên cấu trúc chu kỳ của toàn bộ mẫu thông qua ô đầu tiên của bảng hậu tố: $\text{shift} = \text{bmGs}[0]$.
- Nếu $i \geq 0$ (xảy ra sai khác): chuyển sang tính toán khoảng trượt tối ưu.

(3) Dịch cửa sổ: khi xảy ra sai khác tại bất kỳ vị trí i nào, thuật toán trích xuất cặp ký tự liên tiếp nằm ở cuối cửa sổ hiện tại để tra ma trận ztBc . Khoảng cách tịnh tiến cửa sổ được quyết định bởi giá trị lớn nhất: $\text{shift} = \max(\text{bmGs}[i], \text{ztBc}[y[j + m - 2]][y[j + m - 1]])$

Sau đó, cập nhật vị trí cửa sổ mới: $j = j + \text{shift}$.

3.3.3 Đánh giá độ phức tạp thuật toán

3.3.3.1 Độ phức tạp không gian

Thuật toán Zhu-Takaoka đòi hỏi không gian lưu trữ lớn hơn do cấu trúc ma trận:

(1) Ma trận hai chiều ztBc tiêu tốn lượng bộ nhớ tỷ lệ thuận với bình phương kích thước bảng chữ cái: $|\Sigma|^2$.

(2) Mảng một chiều bmGs có kích thước bằng chiều dài mẫu m .

Do đó, tổng độ phức tạp không gian của thuật toán đạt mức: $S(n) = O(m + |\Sigma|^2)$

3.3.3.2 Độ phức tạp thời gian

(1) Giai đoạn tiền xử lý: thuật toán xử lý những việc sau:

- Khởi tạo ma trận và thiết lập giá trị ban đầu mất thời gian $O(|\Sigma|^2)$.
- Vòng lặp cập nhật cặp ký tự chạy dọc theo chiều dài mẫu mất thời gian $O(m)$.
- Tính toán bảng hậu tố tốt bmGs tiêu tốn thời gian $O(m)$.

Do vậy, tổng thời gian tiền xử lý được chặn trong giới hạn: $T_{preprocessing} = O(m + |\Sigma|^2)$

(2) Giai đoạn tìm kiếm:

Trường hợp trung bình: nhờ khả năng nhận diện các cặp ký tự xấu có độ lặp thấp, thuật toán sở hữu những bước nhảy dài, bỏ qua phần lớn các ký tự trên văn bản. Độ phức tạp thời gian tiệm cận thời gian tuyến tính $T_{average} = O(n)$.

Trường hợp xấu nhất: khi văn bản và mẫu đều được cấu thành từ một ký tự lặp lại duy nhất, hoặc khi cấu trúc chuỗi có các cặp ký tự trùng nhau liên tục, thuật toán quay về trạng thái quét từng ký tự đơn lẻ với độ dịch chuyển tối thiểu bằng 1 ở mỗi bước. Độ phức tạp thời gian trường hợp xấu nhất là: $T_{worst} = O(m \times n)$

3.3.4 Kiểm nghiệm thuật toán

3.3.4.1 Dữ liệu vào

Cho hai xâu dưới đây. Thực hiện tìm kiếm mẫu x trong văn bản y

x="aaba"
y="aaabaaabaaabaa"

3.3.4.2 Quy trình kiểm nghiệm thuật toán

(1) Giai đoạn tiền xử lý

Text and Pattern character															Full string
pos j	0	1	2	3	4	5	6	7	8	9	10	11	12	13	
y[j]	a	a	b	a	a	b	a	a	c	a	a	b	a	a	aaabaaabaaabaa
x[j]	a	a	b	a											aaba

Chạy bảng ztBc (Zhu-Takaoka Bad Character)				
Bước	i	pattern [i-1..i]	bmBc	atBc[x[i-1]][x[i]]
0	-	-	4 4 4 4 4 4 4 4 4	Khởi tạo
1			3 4 4 3 4 4 3 4 4	ztBc[i][a]=4-1=3
	1	aa	2 4 4 3 4 4 3 4 4	ztBc[a][a]=4-1-1=2
	2	ab	2 1 4 3 4 4 3 4 4	ztBc[a][b]=4-1-2=1

Bảng ztBc			
	a	b	khác
a	2	1	4
b	3	4	4
khác	3	4	4

Bảng suff: suff[i]=maxk x[i-k+1...i] =x[m-k...m-1]				
i	x[i]	suffix x[0..i]	Hậu tố x khớp dài nhất với suffix x[0..i]	Suff[i]
0	a	a	a	1
1	a	a, aa	a	1
2	b	b, ab, aab	Không	0
3	a	a, ba, aba, aaba	aaba	4

Chạy bảng bmGs						
Bước	i	j	bmGs trước	Điều kiện	Cập nhật	bmGs sau
1	init	-	[4, 4, 4, 4]			
2	2	-	[4, 4, 4, 4]	suff[2] = 0 != 2+1		[4, 4, 4, 4]
3	1	-	[4, 4, 4, 4]	suff[1] = 1 != 1+1		[4, 4, 4, 4]
4	0	j = 0	[4, 4, 4, 4]	suff[0] = 1 == 0+1	bmGs[j], j=0..2	[3, 3, 3, 4]
5	suffix i=0	2	[3, 3, 3, 4]	m - 1 - suff[0] = 2	bmGs[2] = 3	[3, 3, 3, 4]
6	suffix i=1	2	[3, 3, 3, 4]	m - 1 - suff[1] = 2	bmGs[2] = 2	[3, 3, 2, 4]
7	suffix i=2	3	[3, 3, 2, 4]	m - 1 - suff[2] = 3	bmGs[3] = 1	[3, 3, 2, 1]

Bảng bmGs	
i	bmGs[i]
0	3
1	3
2	2
3	1

(2) Giai đoạn tìm kiếm

Kiểm nghiệm thuật toán												
Bước	j	i	y[j..i+m-1]	x[i]	y[j+i]	x[i]==y[j+i]?	bmGs[i] (1)	ztBc[y[i+m-2]][y[i+m-1]] (2)	Status	j_next = max ((1), (2))	i_next	Output
1	0	3	aaba	a	a	TRUE					2	
2	0	2	aaba	b	b	TRUE					1	
3	0	1	aaba	a	a	TRUE					0	
4	0	0	aaba	a	a	TRUE	3	ztBc[b][a] = 3	MATCH	j = 0 + 3 = 3	3	0
5	3	3	aaba	a	a	TRUE	1				2	
6	3	2	aaba	b	b	TRUE	2				1	
7	3	1	aaba	a	a	TRUE	3				0	
8	3	0	aaba	a	a	TRUE	3	ztBc[b][a] = 3	MATCH	j = 3 + 3 = 6	3	3
9	6	3	aaca	a	a	TRUE	1				2	
10	6	2	aaca	b	b	TRUE	2				1	
11	6	1	aaca	a	c	FALSE	3	ztBc[c][a] = 3	MISS	j = 6 + 3 = 9	3	
12	9	3	aaba	a	a	TRUE	1				2	
13	9	2	aaba	b	b	TRUE	2				1	
14	9	1	aaba	a	a	TRUE	3				0	
15	9	0	aaba	a	a	TRUE	3	ztBc[b][a] = 3	MATCH	j = 9 + 3 = 12	3	9
15	12	0	aa									

Sau khi kiểm nghiệm, kết quả cho thấy mẫu x="aaba" tồn tại trong văn bản y="aaabaaabaaabaa" tại 3 vị trí là {0, 3, 9}

3.3.5 Cài đặt thuật toán bằng C++

3.3.5.1 Hàm tiền xử lý: lập bảng *ztBc* (ZhuTakaoka Bad Character), *bmGs* (Boyer-Moore Good Suffix)

```
const int ASIZE = 256;

// Tiền xử lý bảng dịch chuyển bad character
void preZtBc(const string &pattern, vector<vector<int>> &ztBc) {
    int m = pattern.size();

    for (int i = 0; i < ASIZE; ++i) {
        for (int j = 0; j < ASIZE; ++j) ztBc[i][j] = m;
    }

    // Gán giá trị cho mọi cặp ký tự có ký tự thứ 2 là pattern[0]
    for (int i = 0; i < ASIZE; ++i) ztBc[i][(unsigned char)pattern[0]] = m - 1;
}
```

```

// Tính khoảng dịch cho từng cặp ký tự liên tiếp trong pattern
for (int i = 1; i < m - 1; ++i) {
    ztBc[(unsigned char)pattern[i - 1]][(unsigned char)pattern[i]] = m - 1 - i;
}

void suffixes(const string &x, vector<int> &suff) {
    int m = x.size();

    suff[m - 1] = m;

    int g = m - 1;
    int f = 0;

    for (int i = m - 2; i >= 0; --i) {
        if (i > g && suff[i + m - 1 - f] < i - g) suff[i] = suff[i + m - 1 - f];

        else {
            if (i < g) g = i;
            while (g >= 0 && x[g] == x[g + m - 1 - f]) --g;
            suff[i] = f - g;
        }
    }
}

// Tiền xử lý good suffix
void preBmGs(const string &x, vector<int> &bmGs) {
    int m = x.size();

    vector<int> suff(m);

    suffixes(x, suff);

    for (int i = 0; i < m; ++i) bmGs[i] = m;

    int j = 0;

    for (int i = m - 1; i >= 0; --i) {
        if (suff[i] == i + 1) {
            for (; j < m - 1 - i; ++j) {
                if (bmGs[j] == m) bmGs[j] = m - 1 - i;
            }
        }
    }

    for (int i = 0; i <= m - 2; ++i) bmGs[m - 1 - suff[i]] = m - 1 - i;
}

```

3.3.5.2 Hàm tìm kiếm bằng thuật toán Zhu-Takaoka

```
// Thuật toán Zhu-Takaoka
```

```
void zhuTakaoka(const string &pattern, const string &text) {
    int m = pattern.size(), n = text.size();

    vector<vector<int>> ztBc(ASIZE, vector<int>(ASIZE));
    vector<int> bmGs(m);

    preZtBc(pattern, ztBc); preBmGs(pattern, bmGs);

    int j = 0;
    while (j <= n - m) {
        int i = m - 1;
        while (i >= 0 && pattern[i] == text[i + j]) --i;

        if (i < 0) {
            cout << "Match at position: " << j << endl;
            // Dịch theo good suffix
            j += bmGs[0];
        } else {
            // Bước dịch theo good suffix
            int shift1 = bmGs[i];
            // Bước dịch theo cặp 2 ký tự cuối của số hiện tại
            int shift2 = ztBc[(unsigned char)text[j + m - 2]][(unsigned
char)text[j + m - 1]];

            j += max(shift1, shift2);
        }
    }
}
```

3.4 Thuật toán Berry-Ravindran

3.4.1 Giới thiệu thuật toán

Thuật toán Berry-Ravindran (do Berry và Ravindran đề xuất) là một thuật toán lai (hybrid), kết hợp tư duy nhìn xa của thuật toán Quick Search và cấu trúc ma trận hai ký tự của thuật toán Zhu-Takaoka.

Các thuật toán truyền thống như Boyer-Moore hay Zhu-Takaoka chỉ tính toán bước dịch dựa trên các ký tự bên trong cửa sổ khi xảy ra sai khác. Thuật toán Quick Search cải tiến bằng cách nhìn vào một ký tự ngay sau cửa sổ. Còn thuật toán Berry-Ravindran xét cùng lúc một cặp hai ký tự liên tiếp nằm ngay phía sau cửa sổ văn bản để quyết định bước trượt.

Thuật toán Berry-Ravindran có các ưu điểm sau:

(1) Nhờ kiểm tra cặp ký tự nằm ngoài cửa sổ ($y[j + m]$ và $y[j + m + 1]$), nếu cặp ký tự này hoàn toàn không xuất hiện trong mẫu, thuật toán có thể trượt một khoảng cách tối đa bằng $m+2$, vượt qua cả chiều dài mẫu m .

(2) Việc xét cặp ký tự phía sau làm giảm tỷ lệ trùng lặp ngẫu nhiên xuống mức cực thấp, biến thuật toán Berry-Ravindran trở thành một trong những thuật toán có tốc độ thực tế nhanh nhất khi tìm kiếm trên các bảng chữ cái nhỏ (như chuỗi DNA) hoặc văn bản dài.

3.4.2 Quy trình thực hiện thuật toán

Giả sử có chuỗi văn bản y độ dài n và chuỗi mẫu x độ dài m . Quy trình của thuật toán bao gồm hai giai đoạn:

3.4.2.1 Tiền xử lý mẫu

Giai đoạn này tập trung xây dựng ma trận ký tự xấu mở rộng hai chiều brBc (Berry-Ravindran Bad Character) có kích thước $|\Sigma| \times |\Sigma|$:

(1) Gán tất cả các ô trong ma trận bằng giá trị dịch chuyển cực đại: $\text{brBc}[a][b] = m + 2$

(2) Với mọi ký tự a , nếu cặp ký tự kết thúc bằng chữ cái đầu tiên của mẫu ($x[0]$), khoảng dịch được đặt là: $\text{brBc}[a][x[0]] = m + 1$

(3) Chạy chỉ số i từ 0 đến $m-2$ dọc theo mẫu x . Tại mỗi vị trí, cập nhật khoảng cách từ cặp ký tự liên tiếp ($x[i]$, $x[i + 1]$) tới đầu cửa sổ tiếp theo: $\text{brBc}[x[i]][x[i + 1]] = m - i$

(4) Với mọi ký tự a , nếu cặp ký tự bắt đầu bằng chữ cái cuối cùng của mẫu ($x[m - 1]$), khoảng dịch tối thiểu được đặt là: $\text{brBc}[x[m - 1]][a] = 1$

3.4.2.2 Quá trình so khớp và tìm kiếm

Lưu ý: để tránh lỗi tràn bộ nhớ khi kiểm tra các ký tự vượt biên ở cuối văn bản, thuật toán tiến hành đặt hai ký tự lính canh: $y[n] = '\0'$ và $y[n+1] = '\0'$.

Vị trí ban đầu của cửa sổ đặt tại chỉ số $j = 0$. Vòng lặp chính duy trì khi $j \leq n - m$

(1) So sánh trực tiếp mẫu và cửa sổ văn bản: thuật toán tiến hành so sánh toàn bộ đoạn văn bản trong cửa sổ hiện tại ($y[j \dots j+m-1]$) với chuỗi mẫu x .

- Nếu khớp hoàn toàn, ghi nhận tìm thấy mẫu tại vị trí j của văn bản.
- Nếu có ký tự sai khác, thuật toán chuyển ngay sang bước trượt.

(2) Dịch chuyển cửa sổ:

Thuật toán nhảy ra phía sau cửa sổ hiện tại, trích xuất cặp hai ký tự liên tiếp đặt tại chỉ số văn bản: $a = y[j+m]$ và $b = y[j+m+1]$

Tính tiến cửa sổ một khoảng được tra trực tiếp từ ma trận hai chiều: $j = j + \text{brBc}[a][b]$

3.4.3 Đánh giá độ phức tạp thuật toán

3.4.3.1 Độ phức tạp không gian

Thuật toán Berry-Ravindran sử dụng ma trận hai chiều brBc (Berry-Ravindran Bad Character) cố định để lưu trữ khoảng dịch. Ma trận brBc tiêu tốn lượng bộ nhớ tỷ lệ thuận với bình phương kích thước bảng chữ cái: $|\Sigma|^2$.

Do đó, tổng độ phức tạp không gian thuật toán là: $S(n) = O(|\Sigma|^2)$

3.4.3.2 Độ phức tạp thời gian

(1) Giai đoạn tiền xử lý:

- Thuật toán khởi tạo toàn bộ ma trận ban đầu với thời gian $O(|\Sigma|^2)$.
- Vòng lặp nạp các cặp ký tự dựa trên chiều dài mẫu mất $O(m)$.

Do vậy, tổng thời gian tiền xử lý là: $T_{\text{preprocessing}} = O(m + |\Sigma|^2)$

(2) Giai đoạn tìm kiếm:

Trường hợp trung bình: nhờ cơ chế nhìn xa 2 ký tự ngoài cửa sổ, thuật toán đạt được các bước trượt dài (thường xuyên đạt mức $m+1$ hoặc $m+2$). Số lần phải dịch chuyển cửa sổ giảm đi đáng kể, giúp thời gian tìm kiếm thực tế tiệm cận mức tuyến tính: $T_{\text{average}} \approx O(n)$.

Trường hợp xấu nhất: (ví dụ văn bản và mẫu gồm toàn một ký tự lặp lại duy nhất), ma trận brBc sẽ liên tục trả về khoảng trượt bằng 1. Độ phức tạp thời gian trường hợp tệ nhất đạt mức hình học: $T_{\text{worst}} = O(m \times n)$

3.4.4 Kiểm nghiệm thuật toán

3.4.4.1 Dữ liệu vào

Cho hai chuỗi dưới đây. Thực hiện tìm kiếm mẫu x trong văn bản y

x="aaba"

y="aaabaaabaaabaa"

3.4.4.2 Quy trình kiểm nghiệm thuật toán

(1) Giai đoạn tiền xử lý: lập bảng brBc

Text and Pattern character															Full string
pos j	0	1	2	3	4	5	6	7	8	9	10	11	12	13	
y[j]	a	a	b	a	a	b	a	a	c	a	a	b	a	a	aabaabaacaabaa
x[j]	a	a	b	a											aaba

Chạy bảng Berry-Ravindran Bad Character brBc			
Bước	i	brBc	Giải thích
1		6 6 6 6 6 6 6 6 6	Khởi tạo mảng brBc
2		5 6 6 5 6 6 5 6 6	x[0] = a brBc[a][a] = brBc[b][a] = brBc[khác][a] = m + 1 = 5
3	0	4 6 6 5 6 6 5 6 6	brBc[x[0]][x[1]] = brBc[a][a] = m - i = 4 - 0 = 4
3	1	4 3 6 5 6 6 5 6 6	brBc[x[1]][x[2]] = brBc[a][b] = m - i = 4 - 1 = 3
3	2	4 3 6 2 6 6 5 6 6	brBc[x[2]][x[3]] = brBc[b][a] = m - i = 4 - 2 = 2
4		1 1 1 2 6 6 5 6 6	x[m-1] = x[3] = a brBc[a][a] = brBc[a][b] = brBc[a][khác] = 1

Bảng brBc			
	a	b	khác
a	1	1	1
b	2	6	6
khác	5	6	6

Thêm \0		
j	14	15
y[j]	\0	\0

(2) Giai đoạn tìm kiếm

Kiểm nghiệm thuật toán								
Bước	j	x	y[j..j+m-1]	x=y[j..j+m-1]?	y[j+m..j+m+1]	shift = brBc[y[j+m]][y[j+m+1]]	j_new = j + shift	Output
1	0	aaba	aaba	TRUE	ab	brBc[a][b]	1	0
2	1	aaba	abaa	FALSE	ba	brBc[b][a]	2	3
3	3	aaba	aaba	TRUE	ac	brBc[a][b]	1	3
4	4	aaba	abaa	FALSE	ca	brBc[c][a]	5	9
5	9	aaba	aaba	TRUE	a\0	brBc[a][\0]	1	9
6	10	aaba	abaa	FALSE	a\0	brBc[\0][\0]	6	16
7	16							Kết thúc

Sau khi kiểm nghiệm, kết quả cho thấy mẫu x="aaba" tồn tại trong văn bản y="aaabaaabaaabaa" tại 3 vị trí là {0, 3, 9}

3.4.5 Cài đặt thuật toán bằng C++

3.4.5.1 Hàm tiền xử lý: lập bảng *brBc* (Berry-Ravindran Bad Character)

```
const int ASIZE = 256; // Bảng mã ASCII

// Hàm tiền xử lý bảng bad character của thuật toán Berry-Ravindran
void preBrBc(string pattern, int brBc[ASIZE][ASIZE]) {
    int m = pattern.length(); // Độ dài pattern

    for (int a = 0; a < ASIZE; ++a) {
        for (int b = 0; b < ASIZE; ++b) brBc[a][b] = m + 2;
    }

    // Nếu ký tự đầu tiên của pattern xuất hiện ở vị trí thứ 2 sau cửa sổ
    for (int a = 0; a < ASIZE; ++a) brBc[a][(unsigned char)pattern[0]] = m + 1;

    // Gán giá trị dịch cho từng cặp ký tự liên tiếp trong pattern
    for (int i = 0; i < m - 1; ++i) brBc[(unsigned char)pattern[i]][(unsigned char)pattern[i + 1]] = m - i;

    // Nếu ký tự cuối pattern xuất hiện ngay sau cửa sổ
    for (int a = 0; a < ASIZE; ++a) brBc[(unsigned char)pattern[m - 1]][a] = 1;
}
```

3.4.5.2 Hàm tìm kiếm bằng thuật toán Berry-Ravindran

```
// Hàm tìm kiếm Berry-Ravindran
void berryRavindran(string pattern, string text) {
    int m = pattern.length(), n = text.length();

    int brBc[ASIZE][ASIZE];
    preBrBc(pattern, brBc);

    // Thêm 2 ký tự giả để tránh truy cập vượt mảng
    text += '\0'; text += '\0';

    int j = 0;
    while (j <= n - m) {
        // So sánh trực tiếp pattern với đoạn text hiện tại
        if (text.substr(j, m) == pattern) {
            // In ra vị trí tìm thấy
            cout << "Match at position: " << j << endl;
        }

        // Dịch chuyển theo bảng Berry-Ravindran
        j += brBc[(unsigned char)text[j + m]][(unsigned char)text[j + m + 1]];
    }
}
```

CHƯƠNG 4: CÁC THUẬT TOÁN TÌM KIẾM MẪU TỪ VỊ TRÍ XÁC ĐỊNH

4.1 Thuật toán Two-Way

4.1.1 Giới thiệu thuật toán

Thuật toán Two-Way (được phát triển bởi Maxime Crochemore và Dominique Perrin vào năm 1991) là một đỉnh cao kiệt xuất trong trường phái xử lý chuỗi chính xác. Thuật toán này có khả năng đạt được hiệu năng thời gian tuyến tính tối ưu mà hoàn toàn không tiêu tốn thêm bất kỳ không gian bộ nhớ bổ trợ nào.

Hầu hết các thuật toán hiệu năng cao như Boyer-Moore, Zhu-Takaoka hay Berry-Ravindran đều phải đánh đổi bằng bộ nhớ tiền xử lý (các mảng dịch chuyển một chiều hoặc ma trận hai chiều có kích thước phụ thuộc vào bảng chữ cái $|\Sigma|$). Two-Way phá vỡ giới hạn này bằng cách khai thác sâu sắc các tính chất tổ hợp của từ ngữ và định lý chu kỳ vị trí chặn (Critical Factorization Theorem).

Thuật toán có các đặc điểm sau:

(1) Thuật toán chia mẫu x thành hai phần: tiền tố x_l và hậu tố x_r ($x = x_l x_r$). Điểm cắt này được chọn sao cho chu kỳ cục bộ tại điểm cắt trùng khít với chu kỳ toàn cục của toàn bộ mẫu, giúp tối đa hóa thông tin thu được sau mỗi phép so khớp.

(2) Cửa sổ khi so khớp không quét tuần tự một chiều. Tại mỗi lượt thử, thuật toán đối chiếu đoạn hậu tố x_r từ trái sang phải trước. Nếu hoàn toàn trùng khớp, đoạn tiền tố x_l mới được đem ra đối chiếu ngược từ phải sang trái.

4.1.2 Định nghĩa toán học của thuật toán Two-way

(1) Chu kỳ của chuỗi x :

Một số nguyên p ($1 \leq p \leq |x|$) được gọi là một chu kỳ của chuỗi x nếu $x[i] = x[i + p]$ với mọi i thỏa mãn $0 \leq i < |x| - p$.

Chu kỳ nhỏ nhất của x được ký hiệu là $\text{per}(x)$.

(2) Sự lặp lại và chu kỳ cục bộ: xét một phân hoạch (u, v) của chuỗi x ($x = uv$).

Một chuỗi w được gọi là một sự lặp lại tại điểm cắt (u, v) nếu nó xuất hiện đồng thời ở cả hai bên điểm cắt (có thể tràn ra biên ngoài), nghĩa là:

- w là hậu tố của u hoặc u là hậu tố của w .
- w là tiền tố của v hoặc v là tiền tố của w .

Độ dài của chuỗi w ngắn nhất thỏa mãn điều kiện trên gọi là chu kỳ cục bộ tại điểm cắt (u, v) , ký hiệu là $r(u, v)$. (Lưu ý: w có thể được kết hợp từ u và v thay vì suy ra trực tiếp)

(3) Phân hoạch tối hạn: một phân hoạch (u, v) của chuỗi x được gọi là tối hạn nếu chu kỳ cục bộ tại điểm cắt đó đúng bằng chu kỳ toàn cục của chuỗi: $r(u, v) = \text{per}(x)$

(4) Định lý phân hoạch tối hạn: mọi chuỗi x có độ dài $|x| \geq 2$ luôn tồn tại ít nhất một phân hoạch tối hạn (u, v) sao cho $|u| < \text{per}(x)$.

4.1.3 Quy trình thực hiện thuật toán

Giả sử có chuỗi văn bản y độ dài n và chuỗi mẫu x độ dài m . Thuật toán Two-Way được thực hiện theo quy trình sau:

4.1.3.1 Tiền xử lý mẫu

(1) Tìm điểm cắt tối hạn l và chu kỳ nhỏ nhất tương ứng $p = \text{per}(x)$ của chuỗi mẫu x . Mẫu được phân hoạch thành $x = x_l x_r$ với $x_l = x[0 \dots l]$ và $x_r = x[l + 1 \dots m - 1]$.

(2) Xác định mẫu có dạng lặp tuần hoàn hay không: bằng cách đối chiếu đoạn đầu mẫu với đoạn dịch chuyển theo chu kỳ. Dựa vào kết quả này, thuật toán chia giai đoạn tìm kiếm thành 2 nhánh logic độc lập (mẫu tuần hoàn và mẫu không tuần hoàn).

(3) Khởi tạo vị trí cửa sổ văn bản $j = 0$ và biến ghi nhớ phân đoạn trùng khớp $\text{memory} = -1$.

4.1.3.2 Quy trình so khớp và tìm kiếm

Vòng lặp duy trì khi cửa sổ chưa vượt quá giới hạn văn bản ($j \leq n - m$). Quy trình so khớp và tìm kiếm được thực hiện theo các bước sau

(1) Đối chiếu hậu tố (x_r): quét từ trái sang phải

Thuật toán đặt con trỏ bắt đầu ngay sau điểm cắt tối hạn: $i = l + 1$. Nếu có giá trị từ lượt trước ($\text{memory} > -1$), thuật toán sẽ tận dụng để bỏ qua các ký tự đã biết chắc là khớp.

- Tiến hành so sánh $x[i]$ với $y[j+i]$ và tăng dần i về phía bên phải.
- Nếu xảy ra sai khác tại vị trí i trước khi quét hết mẫu, cửa sổ lập tức trượt sang phải một khoảng bằng: $\text{shift} = i - 1$
- Đặt lại $\text{memory} = -1$ và chuyển sang lượt thử tiếp theo (bỏ qua bước 2).

(2) Đối chiếu tiền tố (x_l): quét từ phải sang trái.

Nếu toàn bộ đoạn hậu tố x_r đã khớp hoàn toàn, thuật toán quay ngược lại điểm cắt để kiểm tra phần tiền tố còn lại.

- Đặt con trỏ xuất phát tại điểm cắt: $i = l$. Quét lùi dần về phía bên trái hướng về chỉ số 0.

- Quá trình quét lùi dừng lại khi i chạm đến giới hạn của vùng nhớ đã khớp từ trước ($i \leq \text{memory}$).

(3) Xác định trạng thái và dịch chuyển

- Nếu tiền tố khớp hoàn toàn:
 - Ghi nhận tìm thấy mẫu tại vị trí j .
 - Do điểm cắt là tối hạn, thuật toán được quyền trượt cửa sổ một khoảng bằng đúng chu kỳ toàn cục: $j = j + p$.
 - Lưu vết độ dài đoạn trùng lặp cho lượt thử sau: $\text{memory} = m - p - 1$.
- Nếu xảy ra sai khác ở đoạn tiền tố: cửa sổ trượt sang phải một khoảng cố định bằng chu kỳ p , giải phóng bộ nhớ ghi nhớ: $\text{memory} = -1$.

4.1.4 Đánh giá độ phức tạp thuật toán

4.1.4.1 Độ phức tạp không gian

Thuật toán chỉ lưu trữ các biến vị trí điểm cắt l , chu kỳ p , và biến memory . Do vậy, độ phức tạp không gian của thuật toán là: $S(n) = O(1)$

4.1.4.2 Độ phức tạp thời gian

(1) Giai đoạn tiền xử lý: Thuật toán tìm hậu tố cực đại bằng cách duyệt qua mẫu với cơ chế dịch chuyển thông minh dạng tiệm cận. Tổng thời gian xử lý mẫu đạt mức tuyến tính:

$$T_{\text{preprocessing}} = O(m).$$

(2) Giai đoạn tìm kiếm:

Nhờ cơ chế ghi nhớ tiền tố kết hợp với tính chất của điểm cắt tối hạn, thuật toán loại bỏ hoàn toàn các phép so sánh lặp thừa. Ngay cả trong kịch bản dữ liệu tệ nhất, thuật toán vẫn giữ vững cấu trúc thời gian tuyến tính: $T_{\text{average}} = T_{\text{worst}} = O(n)$

Trong trường hợp xấu nhất, thuật toán Two-Way được chứng minh toán học là thực hiện không quá $2n - m$ phép so sánh ký tự trên văn bản.

4.1.5 Kiểm nghiệm thuật toán

4.1.5.1 Dữ liệu vào

Cho hai xâu dưới đây. Thực hiện tìm kiếm mẫu x trong văn bản y

```
x="aaba"
y="aaabaaabaaabaa"
```

4.1.5.2 Quy trình kiểm nghiệm thuật toán

(1) Giai đoạn tiền xử lý

Text and Pattern character														Full string	
pos j	0	1	2	3	4	5	6	7	8	9	10	11	12		13
y[j]	a	a	b	a	a	b	a	a	c	a	a	b	a		a
x[j]	a	a	b	a											aaba

Tìm chu kì nhỏ nhất per(x)	
p	Kiểm tra
1	$x[0] == x[1]; x[1] != x[2] \Rightarrow$ Không là chu kỳ
2	$x[0] != x[2] \Rightarrow$ Không là chu kỳ
3	$x[0] == x[3] \Rightarrow$ Là chu kỳ
4	Luôn là chu kỳ
$per(x) = \min(p)$	
3	

Xác định $r(u,v)_{\min}$					
Điểm cắt	u	v	w_min	Giải thích	$r(u,v) = \text{len}(w_{\min})$
0	""	aaba	a	"" là hậu tố mọi chuỗi, a là tiền tố của aaba	1
1	a	aba	a	a là hậu tố của a, a là tiền tố của aba	1
2	aa	ba	baa	aa là hậu tố của baa, ba là tiền tố của baa. (ghép u và v)	3
3	aab	a	ab	ab là hậu tố của aab, a là tiền tố của aab	2
4	aaba	""	a	a là hậu tố của aaba, "" là tiền tố của mọi chuỗi	1

Xác định phân hoạch mẫu	
$l(\text{ell}) = \min(r(u,v))$	1
$x_l = x[0..l]$	aa
$x_r = x[l+1..m-1]$	ba

(2) Giai đoạn tìm kiếm

Kiểm nghiệm thuật toán								
Bước	j	$y[j..j+m-1]$	So khớp hậu tố với x_r	So khớp tiền tố với x_l	memory	shift	$j_{\text{new}} = j + \text{shift}$	Output
1	0	aaba	$x[2] == y[2] (b == b)$ $x[3] == y[3] (a == a)$ $\Rightarrow$ Khớp xr	$x[1] == y[1] (a == a)$ $x[0] == y[0] (a == a)$ $\Rightarrow$ Khớp xl	memory = $m - p - 1 = 4 - 3 - 1 = 0$	shift = $p = 3$ (khớp hoàn toàn)	$0 + 3 = 3$	0
2	3	aaba	$x[2] == y[5] (b == b)$ $x[3] == y[6] (a == a)$ $\Rightarrow$ Khớp xr	$x[1] == y[4] (a == a)$ Dừng quét lùi vì i chạm memory ($i > 0$) $\Rightarrow$ Khớp xl	memory = $m - p - 1 = 4 - 3 - 1 = 0$	shift = $p = 3$ (khớp hoàn toàn)	$3 + 3 = 6$	3
3	6	aaca	$x[2] != y[8] (b != c)$ $\Rightarrow$ Sai khác tại $i=2$	Không xét	memory = -1 (reset do sai khác)	shift = $i - l = 2 - 1 = 1$ (sai khác hậu tố)	$6 + 1 = 7$	
4	7	acaa	$x[2] != y[9] (b != a)$ $\Rightarrow$ Sai khác tại $i=2$	Không xét	memory = -1 (reset do sai khác)	shift = $i - l = 2 - 1 = 1$ (sai khác hậu tố)	$7 + 1 = 8$	
5	8	caab	$x[2] != y[10] (b != a)$ $\Rightarrow$ Sai khác tại $i=2$	Không xét	memory = -1 (reset do sai khác)	shift = $i - l = 2 - 1 = 1$ (sai khác hậu tố)	$8 + 1 = 9$	
6	9	aaba	$x[2] == y[11] (b == b)$ $x[3] == y[12] (a == a)$ $\Rightarrow$ Khớp xr	$x[1] == y[10] (a == a)$ $x[0] == y[9] (a == a)$ $\Rightarrow$ Khớp xl	memory = $m - p - 1 = 4 - 3 - 1 = 0$	shift = $p = 3$ (khớp hoàn toàn)	$9 + 3 = 12$	9
7	12		Cửa sổ vượt text y					

Sau khi kiểm nghiệm, kết quả cho thấy mẫu $x = \text{"aaba"}$ tồn tại trong văn bản $y = \text{"aaabaaabaaabaa"}$ tại 3 vị trí là $\{0, 3, 9\}$

4.1.6 Cài đặt thuật toán bằng C++

4.1.6.1 Hàm tiền xử lý: tính maximal suffix với thứ tự $\leq$ và $\geq$

```
// Hàm tính maximal suffix với thứ tự  $\leq$ 
```

```
int maxSuf(string pattern, int& p) {
    int m = pattern.length(); // Độ dài pattern

    int ms = -1; // Vị trí maximal suffix
    int j = 0;
    int k = 1;    // Độ dài đoạn đang so sánh

    p = 1; // Chu kỳ ban đầu

    // Duyệt cho tới khi vượt quá pattern
    while (j + k < m) {
        char a = pattern[j + k]; // Ký tự phía sau
        char b = pattern[ms + k]; // Ký tự phía trước

        if (a < b) {
            j += k; // Bỏ qua đoạn hiện tại
            k = 1; // Reset độ dài so sánh
            p = j - ms; // Cập nhật chu kỳ
        }
        else if (a == b) {
            // Nếu chưa đạt chu kỳ
            if (k != p) ++k; // Tăng độ dài so sánh
            else {
                j += p; // Nhảy theo chu kỳ
                k = 1; // Reset
            }
        }
        else {
            ms = j; // Cập nhật maximal suffix
            j = ms + 1; // Dịch sang vị trí tiếp theo
            k = 1;
            p = 1;
        }
    }
    return ms;
}

// Hàm tính maximal suffix với thứ tự >=
int maxSufTilde(string pattern, int& p) {
    int m = pattern.length();

    int ms = -1;
    int j = 0;
    int k = 1;

    p = 1;

    while (j + k < m) {
        char a = pattern[j + k], b = pattern[ms + k];

        if (a > b) {
```

```

        j += k;
        k = 1;
        p = j - ms;
    }
    else if (a == b) {
        if (k != p) ++k;
        else {
            j += p;
            k = 1;
        }
    }
    else {
        ms = j;
        j = ms + 1;
        k = 1;
        p = 1;
    }
}
return ms;
}

```

4.1.6.2 Hàm tìm kiếm mẫu bằng thuật toán Two-Way

```

// Thuật toán Two-Way String Matching
void twoWay(string pattern, string text) {
    int m = pattern.length(); // Độ dài pattern
    int n = text.length();    // Độ dài text

    int p, q; // Chu kỳ cục bộ
    int ell;  // Điểm cắt
    int per;  // Chu kỳ toàn cục nhỏ nhất

    int i = maxSuf(pattern, p);
    int j = maxSufTilde(pattern, q);

    // Chọn suffix lớn hơn
    if (i > j) {
        ell = i;
        per = p;
    }
    else {
        ell = j;
        per = q;
    }

    // Kiểm tra pattern có tuần hoàn không
    if (pattern.substr(0, ell + 1) == pattern.substr(per, ell + 1)) {
        j = 0;
        int memory = -1;

        while (j <= n - m) {
            // So sánh nửa phải trước

```

```
i = max(ell, memory) + 1;
while (i < m && pattern[i] == text[i + j]) ++i;

// Nếu nửa phải khớp
if (i >= m) {
    i = ell; // So sánh nửa trái

    while (i > memory && pattern[i] == text[i + j]) --i;
    if (i <= memory) cout << "Match at position: " << j << endl;

    j += per; // Dịch theo chu kỳ
    memory = m - per - 1;
}
else {
    j += (i - ell); // Dịch theo vị trí mismatch
    memory = -1;
}
}
}
else {
    per = max(ell + 1, m - ell - 1) + 1; // Tính chu kỳ mới

    j = 0;
    while (j <= n - m) {
        i = ell + 1; // So sánh nửa phải trước

        while (i < m && pattern[i] == text[i + j]) ++i;

        // Nếu nửa phải khớp
        if (i >= m) {
            i = ell; // So sánh nửa trái
            while (i >= 0 && pattern[i] == text[i + j]) --i;

            if (i < 0) cout << "Match at position: " << j << endl;

            j += per; // Dịch theo period
        }

        // Dịch theo mismatch
        else j += (i - ell);
    }
}
}
```

4.2 Thuật toán Optimal Mismatch

4.2.1 Giới thiệu thuật toán

Thuật toán Optimal Mismatch (được Daniel Sunday đề xuất) là một biến thể tối ưu hóa dựa trên nền tảng của thuật toán Quick Search. Điểm khác biệt cốt lõi tạo nên hướng tiếp cận của thuật toán này là việc tích hợp thông tin về tần suất xuất hiện của các ký tự trong ngôn ngữ hoặc trong văn bản đang xét.

Trong quá trình tìm kiếm chuỗi, các thuật toán truyền thống thường so khớp ký tự theo một thứ tự cố định từ trước (như từ phải sang trái hoặc từ trái sang phải). Hướng tiếp cận này chưa tối ưu vì nếu kiểm tra các ký tự có xác suất xuất hiện cao trước, khả năng trùng khớp tạm thời sẽ tăng lên, dẫn đến việc hệ thống phải thực hiện nhiều phép so sánh vô ích trước khi phát hiện ra sai khác.

Optimal Mismatch giải quyết vấn đề này bằng chiến lược đảo ngược trật tự. Thuật toán sắp xếp và tiến hành so khớp các ký tự trong chuỗi mẫu theo thứ tự tăng dần của tần suất xuất hiện (ưu tiên những ký tự hiếm gặp nhất). Bằng cách kiểm tra các ký tự có tỷ lệ xuất hiện thấp trước, thuật toán tăng xác suất xảy ra sai khác ngay ở những phép thử đầu tiên. Khi sai khác được phát hiện sớm, cửa sổ dịch chuyển có thể trượt đi ngay lập tức mà không cần tốn chi phí đối chiếu các ký tự còn lại, giúp đẩy nhanh tốc độ duyệt qua văn bản.

Thuật toán Optimal Mismatch có một số đặc sau:

(1) Trật tự đối chiếu ký tự không phụ thuộc vào vị trí hình học của chúng trên chuỗi mẫu, mà phụ thuộc hoàn toàn vào bảng thống kê tần suất xuất hiện của từng ký tự trong bảng chữ cái.

(2) Khi xảy ra hiện tượng lệch pha hoặc tìm thấy kết quả, thuật toán tra cứu bảng ký tự xấu mở rộng (qsBc) của thuật toán Quick Search, dựa trên ký tự nằm ngay sau cửa sổ hiện tại để quyết định khoảng cách dịch chuyển.

(3) Bảng dịch chuyển hậu tố tốt (adaptedGs) được xây dựng dựa trên sự biến đổi trật tự quét, đảm bảo tính an toàn tổ hợp khi các ký tự được kiểm tra theo mô hình phi tuần tự.

4.2.2 Quy trình thực hiện thuật toán

Giả sử có chuỗi văn bản y độ dài n , chuỗi mẫu x độ dài m , và một bảng thống kê tần suất cho trước của các ký tự trong bảng chữ cái. Thuật toán Optimal Mismatch được thực hiện theo quy trình sau

4.2.2.1 Tiền xử lý mẫu: xác định trật tự quét mẫu

Sắp xếp lại danh sách các ký tự trong chuỗi mẫu x theo thứ tự tăng dần của tần suất xuất hiện (ký tự nào ít xuất hiện trong thực tế hơn sẽ đứng trước).

Trường hợp nhiều ký tự có tần suất xuất hiện bằng nhau, ký tự nào có chỉ số vị trí lớn hơn sẽ được xếp trước.

4.2.2.2 Tiền xử lý mẫu: lập bảng ký tự xấu *Quick Search Bad Character*

Khởi tạo bảng $qsBc$ với kích thước bảng chữ cái Σ , tất cả các phần tử ban đầu được gán giá trị mặc định là $m + 1$.

Duyệt qua toàn bộ mẫu x từ trái sang phải từ chỉ số 0 đến $m-1$. Tại mỗi vị trí i , khoảng cách dịch chuyển được tính từ vị trí đó đến móc ngay sau mẫu: $qsBc[x[i]] = m - i$

Lưu ý: Vòng lặp này duyệt đến hết ký tự cuối cùng ($i = m - 1$), do đó nếu ký tự cuối cùng của mẫu xuất hiện, giá trị trượt nhỏ nhất của nó sẽ là $m - (m - 1) = 1$.

4.2.2.3 Tiền xử lý mẫu: lập bảng hậu tố tốt thích ứng (*Adapted Good Suffix*)

(1) Khởi tạo ý nghĩa cho các trạng thái

Mảng $adaptedGs$ sẽ có kích thước $m + 1$. Chỉ số i (chạy từ 0 đến m) đại diện cho số lượng ký tự đã khớp liên tiếp theo trật tự quét ưu tiên trước khi gặp sai khác:

- Nếu $i = 0$: sai khác ngay từ ký tự đầu tiên (chưa có thông tin văn bản).
- Nếu $i = 1 \dots m - 1$: đã khớp vài ký tự trong danh sách ưu tiên thì sai khác.
- Nếu $i = m$: khớp hoàn toàn cả mẫu.

(2) Dịch chuyển giả định và kiểm tra mâu thuẫn

Với mỗi trạng thái i , giả định trượt mẫu sang phải một khoảng shift (bắt đầu thử từ 1, 2, 3...). Tại mỗi khoảng shift, thực hiện phép đối chiếu ngược: ký tự tại các vị trí đã khớp trên văn bản có bị xung đột với các ký tự ở vị trí mới trên mẫu hay không

- Nếu vị trí mới trên mẫu là một ký tự khác với ký tự đã biết trên văn bản, thì khoảng trượt này an toàn (không sợ bỏ sót mẫu vì nếu đặt vào đây chắc chắn sẽ sai khác). Tiếp tục tăng shift lên để kiểm tra.

- Nếu tất cả các vị trí đối ứng trên mẫu trùng khớp với ký tự văn bản đã biết (hoặc mẫu trượt hẳn ra ngoài biên văn bản cũ), thì đây là vị trí đầu tiên có cơ hội khớp chuỗi.

(3) Chốt giá trị dịch chuyển an toàn

Giá trị shift nhỏ nhất không gây mâu thuẫn ở bước 2 chính là khoảng cách an toàn cần tìm. Lập tức chốt giá trị shift này cho trạng thái hiện tại: $adaptedGs[i] = shift$

4.2.2.4 Quá trình so khớp và tìm kiếm

Vị trí xuất phát của cửa sổ trên văn bản được đặt tại chỉ số $j = 0$. Vòng lặp chính duy trì liên tục khi cửa sổ chưa vượt quá giới hạn cuối cùng của văn bản ($j \leq n - m$):

(1) Đối chiếu phi tuyến tính: quét theo danh sách tần suất thấp.

Thuật toán so sánh các ký tự từ đầu danh sách ưu tiên đã được sắp xếp. Tại mỗi lượt, hệ thống lấy ký tự mẫu và đối chiếu trực tiếp với ký tự tại vị trí tương ứng trên văn bản

- Nếu ký tự trùng khớp, thuật toán chuyển sang ký tự tiếp theo trong danh sách có tần suất cao hơn.

- Quá trình đối chiếu của lượt thử hiện tại sẽ dừng lại ngay khi xuất hiện một ký tự sai khác hoặc khi đã duyệt qua hết chiều dài mẫu.

(2) Nếu tất cả các ký tự trong danh sách sắp xếp đều trùng khớp hoàn toàn với phân đoạn văn bản tương ứng, ghi nhận tìm thấy mẫu tại vị trí j của văn bản.

(3) Xác định khoảng cách dịch chuyển bằng cách chọn giá trị lớn nhất giữa hai cơ chế:

- Giá trị tra cứu từ bảng hậu tố tốt thích ứng dựa trên số lượng ký tự đã khớp thành công trước khi dừng.

- Giá trị tra cứu từ bảng ký tự xấu $qsBc$ dựa trên ký tự nằm ngay sau cửa sổ hiện tại trên văn bản.

Cửa sổ văn bản được dịch một khoảng bằng giá trị lớn nhất vừa tìm được, giải phóng trạng thái kiểm tra và bắt đầu một lượt thử mới.

4.2.3 Đánh giá độ phức tạp thuật toán

4.2.3.1 Độ phức tạp không gian

Thuật toán Optimal Mismatch duy trì các cấu trúc lưu trữ hỗ trợ bao gồm:

- Một danh sách cấu trúc lưu trật tự quét gồm m phần tử.
- Mảng `adaptedGs` có độ dài bằng chiều dài $m + 1$: $O(m)$.
- Bảng ký tự xấu và bảng tần suất có kích thước phụ thuộc vào dung lượng bảng chữ cái: $O(|\Sigma|)$.

Do đó, độ phức tạp không gian của thuật toán là: $S(n) = O(m + |\Sigma|)$

4.2.3.2 Độ phức tạp thời gian

(1) Giai đoạn tiền xử lý: quá trình sắp xếp lại trật tự chuỗi mẫu kết hợp với việc xây dựng bảng hậu tố tốt thích ứng thông qua các phép đối chiếu chuỗi con nội tại yêu thời gian

tỷ lệ theo bình phương chiều dài mẫu: $O(m^2)$. Khi tính thêm bước khởi tạo bảng chữ cái, tổng thời gian của giai đoạn này là: $T_{preprocessing} = O(m^2 + |\Sigma|)$.

(2) Giai đoạn tìm kiếm:

Trường hợp trung bình: do các ký tự hiếm gặp được đưa lên kiểm tra đầu tiên, tỷ lệ xuất hiện sai khác ngay từ những phép so sánh đầu tiên là rất cao.

- Cửa sổ thường xuyên được dịch chuyển sớm mà không phải quét toàn bộ các ký tự còn lại trong mẫu, giúp thời gian vận hành thực tế đạt hiệu năng cao và tiệm cận mức tuyến tính: $T_{average} \approx O(n)$.

Trường hợp xấu nhất: trong kịch bản phân phối dữ liệu không đều hoặc chuỗi văn bản và chuỗi mẫu được cấu tạo từ các ký tự lặp lại giống nhau hoàn toàn, việc sắp xếp theo tần suất không tạo ra sự khác biệt về xác suất xuất hiện sai khác.

- Thuật toán khi đó vẫn phải thực hiện đầy đủ các phép so sánh ký tự tại mỗi vị trí và dịch chuyển cửa sổ theo từng bước ngắn, dẫn đến độ phức tạp thời gian đạt mức hình học: $T_{worst} = O(m \times n)$

4.2.4 Kiểm nghiệm thuật toán

4.2.4.1 Dữ liệu vào

Cho hai xâu dưới đây. Thực hiện tìm kiếm mẫu x trong văn bản y

x="aaba"
y="aaabaaabaaabaa"

4.2.4.2 Quy trình kiểm nghiệm thuật toán

(1) Giai đoạn tiền xử lý

Text and Pattern character															Full string
pos j	0	1	2	3	4	5	6	7	8	9	10	11	12	13	
y[j]	a	a	b	a	a	b	a	a	c	a	a	b	a	a	aabaabaacaabaa
x[j]	a	a	b	a											aaba

Chạy bảng qsBc						
Bước	i	pattern [i]	qsBc[a]	qsBc[b]	qsBc[khác]	Giải thích qsBc[x[i]] = m-i
0	-	-	5	5	5	
1	0	a	4	5	5	qsBc[a]=4-0=4
2	1	a	3	5	5	qsBc[a]=4-1=3
3	2	b	3	2	5	qsBc[b]=4-2=2
3	2	b	1	2	5	qsBc[a]=4-3=1

Bảng qsBc	
chr	qsBc [chr]
a	1
b	2
khác	5

Trật tự quét x		
i	x[i]	freq [x[i]]
2	b	2
3	a	3
1	a	3
0	a	3

Chạy bảng adaptedGs						
i	Hậu tố đã khớp	shift	Dịch mẫu	Đối chiếu chính	Giải thích ngắn	Kết luận
0		1			Chưa có thông tin văn bản	Chọn
1	b (2)	1	<u>b</u> [*] aaba	b ≠ a	Mẫu mới đặt a lên vị trí đã biết là b	
1	b (2)	2	<u>b</u> [*] aaba	b ≠ a		
2	b (2)	3	<u>b</u> [*] aaba	* = a	* cần khác a, nhưng mẫu mới đặt a ngay lên đó	
3	b (2)	4	a <u>b</u> a aaba		Phần khớp trượt khỏi vùng cũ	Chọn
2	ba (2-3)	1	a <u>b</u> a aaba	b ≠ a	a của mẫu mới đè lên b đã biết	
2	ba (2-3)	2	a <u>b</u> a aaba	b ≠ a	a của mẫu mới đè lên b đã biết	
2	ba (2-3)	3	a <u>b</u> a aaba		Phần khớp gần trượt khỏi vùng cũ	Chọn
3	aba (2-3-1)	1	a <u>b</u> a aaba	a=a, b≠a	a của mẫu mới đè lên b đã biết	
3	aba (2-3-1)	2	a <u>b</u> a aaba	b ≠ a	a của mẫu mới đè lên b đã biết	
3	aba (2-3-1)	3	a <u>b</u> a aaba	a = a	Phần khớp gần trượt khỏi vùng cũ	Chọn
4	aaba (2-3-1-0)	1	a <u>b</u> a aaba	a=a, b≠a	a của mẫu mới đè lên b đã biết	
4	aaba (2-3-1-0)	2	a <u>b</u> a aaba	b ≠ a	a của mẫu mới đè lên b đã biết	
4	aaba (2-3-1-0)	3	a <u>b</u> a aaba	a = a	Phần khớp gần trượt khỏi vùng cũ	Chọn

Bảng adaptedGs	
i	adaptedGs[i]
0	1
1	4
2	3
3	3
4	3

Sau khi kiểm nghiệm, kết quả cho thấy mẫu x="aaba" tồn tại trong văn bản y="aaabaaabaaabaa" tại 3 vị trí là {0, 3, 9}

(2) Giai đoạn tìm kiếm

Kiểm nghiệm thuật toán										
Bước	j	y[j..j+m-1]	So khớp theo ưu tiên (2-3-1-0)	Khớp?	adaptedGs[i]	y[j+m]	qsBc[y[j+m]]	shift	j_new	Output
1	0	aaba	b=b, a=a, a=a, a=a	TRUE	3	a	1	3	3	0
2	3	aaba	b=b, a=a, a=a, a=a	TRUE	3	a	1	3	6	3
3	6	aaca	b≠c	FALSE	1	a	1	1	7	
4	7	acaa	b≠a	FALSE	1	b	2	2	9	
5	9	aaba	b=b, a=a, a=a, a=a	TRUE	3	a	1	3	12	9
6	12		Dừng							

Sau khi kiểm nghiệm, kết quả cho thấy mẫu x="aaba" tồn tại trong văn bản y="aaabaaabaaabaa" tại 3 vị trí là {0, 3, 9}

4.2.5 Cài đặt thuật toán bằng C++

4.2.5.1 Khai báo cấu trúc và biến toàn cục

```
const int ASIZE = 256;

struct Pattern {
    int loc;
    char c;
};

vector<int> freq(ASIZE, 0);
```

4.2.5.2 Hàm tiền xử lý: lập bảng *qsBc*

```
void preQsBc(const string &pattern, vector<int> &qsBc) {
    int m = pattern.length();

    for (int i = 0; i < ASIZE; ++i) qsBc[i] = m + 1;
    for (int i = 0; i < m; ++i) qsBc[(unsigned char)pattern[i]] = m - i;
}
```

4.2.5.3 Hàm tiền xử lý: xác định thứ tự duyệt mẫu ưu tiên

```
bool optimalPcmp(const Pattern &pat1, const Pattern &pat2) {
    // So sánh theo tần suất ký tự
    int fx = freq[(unsigned char)pat1.c] - freq[(unsigned char)pat2.c];

    if (fx != 0) return fx < 0; // Xếp tăng dần
    return pat1.loc > pat2.loc; // Nếu bằng nhau thì ưu tiên vị trí lớn hơn
}

void orderPattern(const string &pattern, vector<Pattern> &pat) {
    int m = pattern.length();
    pat.resize(m);

    for (int i = 0; i < m; ++i) {
        pat[i].loc = i;
        pat[i].c = pattern[i];
    }
    sort(pat.begin(), pat.end(), optimalPcmp);
}
```

4.2.5.4 Hàm tiền xử lý: lập bảng *adaptedGs*

```
// Hàm tìm leftward matching shift tiếp theo
int matchShift(const string &pattern, int ploc, int lshift, vector<Pattern> &pat)
{
    int m = pattern.size(), i, j;

    // Thử từng shift
    for (; lshift < m; ++lshift) {
        i = ploc;

        while (--i >= 0) {
            j = pat[i].loc - lshift; // Vị trí tương ứng sau khi dịch

            if (j < 0) continue; // Nếu vượt trái thì bỏ qua
            if (pat[i].c != pattern[j]) break; // Nếu mismatch thì dừng
        }
        if (i < 0) break; // Nếu match hoàn toàn
    }
    return lshift;
}
```

```
// Hàm tiền xử lý adapted good suffix
void preAdaptedGs(const string &pattern, vector<int> &adaptedGs, vector<Pattern>
&pat) {
    int m = pattern.size(), lshift, i, ploc;

    adaptedGs.resize(m + 1);
    adaptedGs[0] = lshift = 1;

    // Xây shift cơ bản
    for (ploc = 1; ploc <= m; ++ploc) adaptedGs[ploc] = matchShift(pattern, ploc,
lshift, pat);

    for (ploc = 0; ploc <= m; ++ploc) {
        lshift = adaptedGs[ploc];

        while (lshift < m) {
            i = pat[ploc].loc - lshift; // Vị trí kiểm tra tiếp theo
            if (i < 0 || pat[ploc].c != pattern[i]) break; // Vượt biên/mismatch

            ++lshift;
            // Tìm shift hợp lệ tiếp theo
            lshift = matchShift(pattern, ploc, lshift, pat);
        }
        adaptedGs[ploc] = lshift; // Cập nhật shift tối ưu
    }
}
```

4.2.5.5 Hàm tìm kiếm bằng thuật toán Optical Mismatch

```
// Thuật toán Optimal Mismatch
void optimalMismatch(string pattern, string text) {
    int m = pattern.length(), n = text.length();

    vector<int> adaptedGs, qsBc(ASIZE);
    vector<Pattern> pat;

    orderPattern(pattern, pat);
    preQsBc(pattern, qsBc);
    preAdaptedGs(pattern, adaptedGs, pat);

    int j = 0;
    while (j <= n - m) {
        int i = 0;
        // So sánh theo thứ tự đã sắp xếp
        while (i < m && pat[i].c == text[j + pat[i].loc]) ++i;

        if (i >= m) cout << "Match at position: " << j << endl;

        j += max(adaptedGs[i], qsBc[(unsigned char) text[j + m]]);
    }
}
```

4.3 Thuật toán Maximal Shift

4.3.1 Giới thiệu thuật toán

Thuật toán Maximal Shift (được Daniel Sunday đề xuất) là một biến thể nâng cao và tối ưu hóa của thuật toán Quick Search.

Ý tưởng cốt lõi của đa số các thuật toán họ Boyer-Moore là so khớp ký tự theo một hướng cố định (thường là từ phải sang trái). Tuy nhiên, Daniel Sunday nhận thấy rằng thứ tự so sánh này chưa tối ưu. Trong một chuỗi mẫu, có những ký tự nếu xảy ra sai khác sẽ tạo ra bước dịch chuyển của số rất lớn, trong khi một số ký tự khác lại chỉ cho bước dịch rất ngắn.

Thuật toán Maximal Shift giải quyết triệt để vấn đề này bằng chiến lược sắp xếp và tiến hành so khớp các ký tự trong mẫu theo thứ tự giảm dần của khả năng tăng chiều dài bước dịch. Bằng cách ưu tiên kiểm tra các ký tự đem lại bước trượt lớn trước, thuật toán có thể phát hiện sai khác sớm và tối thiểu hóa số phép so sánh ký tự trong thực tế.

Thuật toán Maximal Shift có một số đặc điểm chính như sau:

(1) Chiến lược quét tối ưu hóa không tuân theo quy tắc từ trái sang phải hay từ phải sang trái. Thứ tự quét được quyết định sau khi đã sắp xếp các ký tự của mẫu dựa trên mảng khoảng cách dịch chuyển tối thiểu (*minShift*).

(2) Thuật toán tận dụng bảng ký tự xấu của thuật toán Quick Search (*qsBc*) dựa trên ký tự ngay sau cửa sổ ($y[j+m]$) kết hợp với bảng hậu tố tốt thích ứng (*adaptedGs*) được xây dựng riêng theo thứ tự quét đã sắp xếp.

(3) Do khả năng đẩy các ký tự dễ gây sai khác và cho bước trượt dài lên đầu danh sách đối chiếu, thuật toán cực kỳ hiệu quả đối với các hệ thống tìm kiếm dữ liệu lớn.

4.3.2 Quy trình thực hiện thuật toán

Giả sử có chuỗi văn bản y độ dài n và chuỗi mẫu x độ dài m . Thuật toán Maximal Shift được thực hiện theo quy trình sau

4.3.2.1 Tiền xử lý mẫu: lập bảng dịch tối thiểu *minShift*

Thuật toán tiến hành duyệt qua từng vị trí chỉ số i của mẫu x (với $0 \leq i < m$) nhằm xác định khoảng cách hình học từ ký tự hiện tại đến ký tự trùng khớp với nó gần nhất nằm ở phía bên trái.

Nếu tìm thấy ký tự trùng khớp tại chỉ số j (với $j < i$), khoảng cách dịch chuyển tối thiểu được ghi nhận: $\text{minShift}[i] = i - j$

4.3.2.2 *Tiền xử lý mẫu: xác định trật tự quét mẫu*

Dựa trên các giá trị đã thiết lập ở bước 1, tiến hành cấu trúc lại trật tự kiểm tra ký tự thông qua một mảng cấu trúc hỗ trợ pat gồm m phần tử.

Quy trình sắp xếp được thực hiện theo nguyên tắc:

- Sắp xếp các phần tử theo thứ tự giảm dần của giá trị minShift (ký tự nào có tiềm năng tạo ra bước trượt dài hơn khi xảy ra sai khác sẽ được ưu tiên đẩy lên đầu danh sách để đối chiếu trước).
- Trong trường hợp các ký tự có cùng giá trị minShift, thuật toán ưu tiên xếp trước cho ký tự nào nằm ở vị trí bên phải hơn trong chuỗi mẫu ban đầu (ký tự có chỉ số vị trí gốc lớn hơn).

4.3.2.3 *Tiền xử lý mẫu: lập bảng qsBc và adaptedGs*

(1) Xây dựng bảng ký tự xấu Quick Search (qsBc): tính toán khoảng trượt dựa trên ký tự nằm ngay sau cửa sổ hiện tại. (có thể xem lại cách xây dựng tại thuật toán Optimal Mismatch hoặc Quick Search)

(2) Xây dựng bảng hậu tố tốt thích ứng (adaptedGs): tính toán bước dịch chuyển hậu tố tốt, nhưng được thiết kế đặc biệt để thích ứng hoàn hảo với trật tự quét phi tuyến tính của mảng pat. (có thể xem lại cách xây dựng tại thuật toán Optimal Mismatch)

4.3.2.4 *Quy trình so khớp và tìm kiếm*

Vị trí ban đầu của cửa sổ văn bản được đặt tại $j = 0$. Vòng lặp duy trì liên tục khi cửa sổ chưa vượt quá giới hạn văn bản ($j \leq n - m$):

(1) Đối chiếu phi tuyến tính: quét theo trật tự mảng *pat*.

Thuật toán bắt đầu so sánh cửa sổ văn bản từ chỉ số thứ tự $i = 0$. Tiến hành đối chiếu ký tự mẫu $pat[i].c$ với ký tự tương ứng trên văn bản tại vị trí $j + pat[i].loc$

- Nếu khớp, tăng i để kiểm tra ký tự tiếp theo trong danh sách ưu tiên.
- Quá trình dừng lại ngay khi xuất hiện một ký tự sai khác hoặc khi đã quét hết mẫu ($i \geq m$).

(2) Nếu con trỏ i duyệt qua hết toàn bộ m ký tự của mảng sắp xếp ($i \geq m$), ghi nhận tìm thấy mẫu tại vị trí j của văn bản

(3) Dịch cửa sổ: tra cứu hai bảng dịch chuyển xác định khoảng cách lớn nhất

$$shift = \max(adaptedGs[i], qsBc[y[j + m]])$$

Sau đó cập nhật vị trí cửa sổ mới: $j = j + shift$ và bắt đầu lượt thử tiếp theo.

4.3.3 Đánh giá độ phức tạp thuật toán

4.3.3.1 Độ phức tạp không gian

Thuật toán cần các không gian bộ nhớ hỗ trợ cố định bao gồm

- Mảng minShift và mảng hậu tố tốt thích ứng adaptedGs có độ dài phụ thuộc vào chiều dài mẫu: $O(m)$.
- Mảng cấu trúc pat để lưu trật tự quét gồm m phần tử: $O(m)$.
- Bảng ký tự xấu qsBc kích thước phụ thuộc bảng chữ cái: $O(|\Sigma|)$.

Do đó, tổng độ phức tạp không gian hỗ trợ đạt mức: $S(n) = O(m + |\Sigma|)$

4.3.3.2 Độ phức tạp thời gian

(1) Giai đoạn tiền xử lý: việc tính toán mảng minShift kết hợp với thuật toán sắp xếp trật tự chuỗi mẫu cần thời gian đạt mức hình học theo chiều dài mẫu: $O(m^2)$. Khi cộng thêm thời gian khởi tạo bảng chữ cái, tổng thời gian tiền xử lý là: $T_{preprocessing} = O(m^2 + |\Sigma|)$.

(2) Giai đoạn tìm kiếm:

Trường hợp trung bình: nhờ chiến lược đẩy các ký tự có bước trượt lớn lên quét trước, thuật toán có tỷ lệ phát hiện sai khác cực kỳ sớm (ngay từ phép thử đầu tiên $i=0$). Khoảng dịch chuyển của cửa sổ thường xuyên đạt mức tối đa, giúp thuật toán vận hành với tốc độ thực tế tiệm cận mức tuyến tính $T_{average} = O(n)$.

Trường hợp xấu nhất: (ví dụ: văn bản và mẫu gồm các ký tự lặp giống hệt nhau), việc sắp xếp thứ tự quét không phát huy được tác dụng giảm phép đối chiếu. Tại mỗi vị trí cửa sổ, thuật toán vẫn phải so sánh đủ m lần và chỉ trượt lên 1 đơn vị. Độ phức tạp thời gian trong kịch bản tệ nhất này sẽ rơi vào mức: $T_{worst} = O(m \times n)$

4.3.4 Kiểm nghiệm thuật toán

4.3.4.1 Dữ liệu vào

Cho hai xâu dưới đây. Thực hiện tìm kiếm mẫu x trong văn bản y

x="aaba"
y="aaabaaabaaabaa"

4.3.4.2 Quy trình kiểm nghiệm thuật toán

(1) Giai đoạn tiền xử lý

Text and Pattern character															Full string
pos j	0	1	2	3	4	5	6	7	8	9	10	11	12	13	
y[j]	a	a	b	a	a	b	a	a	c	a	a	b	a	a	aaabaaabaaabaa
x[j]	a	a	b	a											aaba

CHƯƠNG 4: CÁC THUẬT TOÁN TÌM KIẾM MẪU TỪ VỊ TRÍ XÁC ĐỊNH

Tìm minShift				
Bước	i	j_max x[i]=x[j]	minShift[i] = i - j	minShift
0				3 3 3 3
	0	-1	0 - (-1) = 1	1 3 3 3
	1	0	1 - 0 = 1	1 1 3 3
	2	-1	2 - (-1) = 3	1 1 3 3
	3	1	3 - 1 = 2	1 1 3 2

MinShift	
i	minShift[i]
0	1
1	1
2	3
3	2

Trật tự quét x (minShift giảm dần)		
i	x[i]	minShift[i]
2	b	3
3	a	2
1	a	1
0	a	1

Chạy bảng qsBc						
Bước	i	pattern [i]	qsBc[a]	qsBc[b]	qsBc [khác]	Giải thích qsBc[x[i]] = m-i
0	-	-	5	5	5	
1	0	a	4	5	5	qsBc[a]=4-0=4
2	1	a	3	5	5	qsBc[a]=4-1=3
3	2	b	3	2	5	qsBc[b] = 4-2=2
3	2	b	1	2	5	qsBc[a] = 4-3=1

Bảng qsBc	
chr	qsBc [chr]
a	1
b	2
khác	5

Chạy bảng adaptedGs						
i	Hậu tố đã khớp	shift	Dịch mẫu	Đối chiếu chính	Giải thích ngắn	Kết luận
0		1			Chưa có thông tin văn bản	Chọn
1	b (2)	1	<u>b</u> aaba	b ≠ a	Mẫu mới đặt a lên vị trí đã biết là b	
1	b (2)	2	<u>b</u> aaba	b ≠ a		
2	b (2)	3	<u>b</u> aaba	* = a	* cần khác a, nhưng mẫu mới đặt a ngay lên đó	
3	b (2)	4	a <u>b</u> a aaba		Phần khớp trượt khỏi vùng cũ	Chọn
2	ba (2-3)	1	a <u>b</u> a aaba	b ≠ a	a của mẫu mới đè lên b đã biết	
2	ba (2-3)	2	a <u>b</u> a aaba	b ≠ a	a của mẫu mới đè lên b đã biết	
2	ba (2-3)	3	a <u>b</u> a aaba		Phần khớp gần trượt khỏi vùng cũ	Chọn
3	aba (2-3-1)	1	a <u>b</u> a aaba	a=a, b≠a	a của mẫu mới đè lên b đã biết	
3	aba (2-3-1)	2	a <u>b</u> a aaba	b ≠ a	a của mẫu mới đè lên b đã biết	
3	aba (2-3-1)	3	a <u>b</u> a aaba	a = a	Phần khớp gần trượt khỏi vùng cũ	Chọn
4	aaba (2-3-1-0)	1	a <u>b</u> a aaba	a=a, b≠a	a của mẫu mới đè lên b đã biết	
4	aaba (2-3-1-0)	2	a <u>b</u> a aaba	b ≠ a	a của mẫu mới đè lên b đã biết	
4	aaba (2-3-1-0)	3	a <u>b</u> a aaba	a = a	Phần khớp gần trượt khỏi vùng cũ	Chọn

Bảng adaptedGs	
i	adaptedGs[i]
0	1
1	4
2	3
3	3
4	3

(2) Giai đoạn tìm kiếm:

Kiểm nghiệm thuật toán										
Bước	j	y[j..j+m-1]	So khớp theo ưu tiên (2-3-1-0)	Khớp?	adaptedGs[i]	y[j+m]	qsBc[y[j+m]]	shift	j_new	Output
1	0	aaba	b=b, a=a, a=a, a=a	TRUE	3	a	1	3	3	0
2	3	aaba	b=b, a=a, a=a, a=a	TRUE	3	a	1	3	6	3
3	6	aaca	b≠c	FALSE	1	a	1	1	7	
4	7	acaa	b≠a	FALSE	1	b	2	2	9	
5	9	aaba	b=b, a=a, a=a, a=a	TRUE	3	a	1	3	12	9
6	12		Dừng							

Sau khi kiểm nghiệm, kết quả cho thấy mẫu x="aaba" tồn tại trong văn bản y="aaabaaabaaabaa" tại 3 vị trí là {0, 3, 9}. Tuy nhiên, bộ test cho ra kết quả tiền xử lý giống so với thuật toán Optimal Mismatch.

4.3.5 Kiểm chứng nâng cao trên chuỗi mẫu dài

Để phá vỡ tính đồng nhất ngẫu nhiên của các chuỗi mẫu ngắn và đơn giản, cần tiến hành một lượt kiểm nghiệm bổ sung với cấu trúc chuỗi có độ dài lớn hơn và có tính tuần hoàn phức tạp nhằm cô lập hành vi logic của từng thuật toán:

Chuỗi mẫu (x): GCAGAGAG (Độ dài $m = 8$)

Văn bản (y): GCATCGCAGAGAGTATACAGTACG (Độ dài $n = 24$)

4.3.5.1 Phân tích kết quả từ phía thuật toán *Optimal Mismatch*

Trình tự quét mẫu (kí tự và index):

C 1 | A 6 | A 4 | A 2 | G 7 | G 5 | G 3 | G 0

Mảng adaptedGs (0-8): 1 3 4 2 8 7 7 7 8

Kết quả:

Match at position: 5

(1) Dựa trên cấu hình mảng tần suất kí tự freq, thuật toán nhận diện ký tự 'C' có xác suất xuất hiện thấp nhất trong bảng chữ cái kiểm thử. Do đó, vị trí chỉ số 1 (chữ 'C') lập tức được đẩy lên đầu danh sách nhằm tối đa hóa tỷ lệ gây sai khác sớm. Các vị trí tiếp theo được xếp theo nhóm chữ cái tăng dần về tần suất: nhóm chữ 'A' ($6 \rightarrow 4 \rightarrow 2$) và cuối cùng là nhóm chữ có tần suất cao nhất 'G' ($7 \rightarrow 5 \rightarrow 3 \rightarrow 0$).

(2) Chính nhờ việc ưu tiên kiểm tra ký tự hiếm 'C' ngay từ đầu, thông tin thu được khi xảy ra sai lệch tại các trạng thái sau vô cùng giàu tính loại trừ cấu trúc. Bước trượt tại trạng thái $i=4$ đạt giá trị cực đại bằng 8 (bằng chiều dài mẫu), giúp cửa sổ nhảy cóc qua toàn bộ phân đoạn văn bản một cách tối ưu.

4.3.5.2 Phân tích kết quả từ phía *Maximal Shift*

Mảng minShift (0-8): 1 2 3 3 2 2 2 2

Trình tự quét mẫu (kí tự và index):

G 3 | A 2 | G 7 | A 6 | G 5 | A 4 | C 1 | G 0

Mảng adaptedGs (0-8): 1 3 3 7 4 7 7 7 7

Kết quả:

Match at position: 5

(1) Thuật toán thiết lập trật tự quét dựa thuần túy trên cấu trúc hình học nội tại minShift. Cặp ký tự tuần hoàn AG xuất hiện liên tục ở các phân đoạn [2,3], [4,5], [6,7] tạo ra các khoảng trượt ngắn bằng 2. Riêng vị trí chỉ số 3 (chữ 'G') có khoảng cách dịch chuyển hình học đến ký tự trùng khớp phía bên trái nó (chữ 'G' tại vị trí 0) là $3 - 0 = 3$. Vì chiếm giữ giá trị minShift lớn nhất, vị trí 3 được ưu tiên đẩy lên đầu danh sách quét để đối chiếu trước.

(2) Do xuất phát từ một trật tự đối chiếu ký tự khác so với Optimal Mismatch, các phép lọc mâu thuẫn chuỗi con trong hàm tiền xử lý cho ra một mảng adaptedGs mang cấu trúc số hoàn toàn độc lập với Optimal Mismatch (1 3 3 7 4 7 7 7 7).

4.3.5.3 Tổng luận giải trình thực nghiệm bổ sung

Sự phân rã rõ rệt từ dữ liệu log thực tế của case kiểm thử nâng cao này cho phép rút ra hai kết luận học thuật quan trọng:

(1) Thực nghiệm chứng minh hai thuật toán đi theo hai triết lý tối ưu trái ngược nhau. Optimal Mismatch là thuật toán dựa trên xác suất thống kê bên ngoài (tìm ký tự hiếm gặp nhất để ép sai khác sớm). Trong khi đó, Maximal Shift là thuật toán dựa trên hình học tổ hợp nội tại (tìm vị trí có bước nhảy đơn lẻ dài nhất).

(2) Dựa vào hai kết quả trên, dù sở hữu trật tự quét đảo lộn nhau và hệ thống mảng dịch chuyển adaptedGs hoàn toàn khác biệt, cả hai thuật toán đều định vị chính xác vị trí xuất hiện của chuỗi mẫu tại chỉ số 5 trên văn bản. Điều này khẳng định tính đúng đắn của cả hai thuật toán và chứng minh rằng Maximal Shift hoàn toàn có khả năng đạt hiệu năng tìm kiếm tương đương Optimal Mismatch mà không cần phụ thuộc vào bất kỳ bộ nạp dữ liệu tần suất nào từ môi trường.

4.3.6 Cài đặt thuật toán bằng C++

4.3.6.1 Khai báo cấu trúc và biến toàn cục

```
const int ASIZE = 256;

struct Pattern {
    int loc;
    char c;
};

vector<int> minShift;
```

4.3.6.2 Hàm tiền xử lý: lập bảng qsBc

```
// Hàm tiền xử lý Quick Search Bad Character
void preQsBc(const string &pattern, vector<int> &qsBc) {
    int m = pattern.length();

    for (int i = 0; i < ASIZE; ++i) qsBc[i] = m + 1;
    for (int i = 0; i < m; ++i) qsBc[(unsigned char)pattern[i]] = m - i;
}
```

4.3.6.3 Hàm tiền xử lý: tính minShift

```
// Tính bảng minShift
void computeMinShift(const string &pattern) {
    int m = pattern.length();
    minShift.resize(m);
```

```

for (int i = 0; i < m; ++i) {
    int j;

    // Tìm ký tự giống gần nhất bên trái
    for (j = i - 1; j >= 0; --j) if (pattern[i] == pattern[j]) break;

    minShift[i] = i - j; // Khoảng cách tới lần xuất hiện trước
}
}

```

4.3.6.4 Hàm tiền xử lý: xác định thứ tự duyệt mẫu theo minShift

```

// Hàm so sánh cho Maximal Shift
bool maxShiftPcmp(const Pattern &pat1, const Pattern &pat2) {
    int dsh = minShift[pat2.loc] - minShift[pat1.loc]; // So sánh theo minShift
    if (dsh != 0) return dsh < 0; // Nếu khác nhau
    return pat1.loc > pat2.loc; // Nếu bằng nhau ưu tiên vị trí lớn hơn
}

// Sắp xếp pattern theo Maximal Shift
void orderPattern(const string &pattern, vector<Pattern> &pat) {
    int m = pattern.length();
    pat.resize(m);

    for (int i = 0; i < m; ++i) {
        pat[i].loc = i;
        pat[i].c = pattern[i];
    }
    sort(pat.begin(), pat.end(), maxShiftPcmp);
}

```

4.3.6.5 Hàm tiền xử lý: lập bảng adaptedGs

```

// Tìm leftward matching shift tiếp theo
int matchShift(const string &pattern, int ploc, int lshift, vector<Pattern> &pat)
{
    int m = pattern.size(), i, j;

    // Thử từng shift
    for (; lshift < m; ++lshift) {
        i = ploc;

        while (--i >= 0) {
            j = pat[i].loc - lshift;

            if (j < 0) continue; // Nếu vượt trái thì bỏ qua
            if (pat[i].c != pattern[j]) break; // Nếu mismatch
        }
        if (i < 0) break; // Nếu match hoàn toàn
    }
    return lshift;
}

```

```
// Tiền xử lý adapted good suffix
void preAdaptedGs(const string &pattern, vector<int> &adaptedGs, vector<Pattern>
&pat) {
    int m = pattern.size(), lshift, i, ploc;

    adaptedGs.resize(m + 1);
    adaptedGs[0] = lshift = 1;

    // Xây bảng shift cơ bản
    for (ploc = 1; ploc <= m; ++ploc) {
        lshift = matchShift(pattern, ploc, lshift, pat);
        adaptedGs[ploc] = lshift;
    }

    for (ploc = 0; ploc <= m; ++ploc) {
        lshift = adaptedGs[ploc];

        while (lshift < m) {
            i = pat[ploc].loc - lshift;
            if (i < 0 || pat[ploc].c != pattern[i]) break; // Vượt biên/mismatch

            ++lshift;
            lshift = matchShift(pattern, ploc, lshift, pat);
        }

        adaptedGs[ploc] = lshift;
    }
}
```

4.3.6.6 Hàm tìm kiếm bằng thuật toán Maximal Shift

```
void maximalShift(string pattern, string text) {
    int m = pattern.length(), n = text.length();

    vector<int> adaptedGs, qsBc(ASIZE);
    vector<Pattern> pat;

    computeMinShift(pattern);
    orderPattern(pattern, pat);
    preQsBc(pattern, qsBc);
    preAdaptedGs(pattern, adaptedGs, pat);

    int j = 0;
    while (j <= n - m) {
        int i = 0;
        while (i < m && pat[i].c == text[j + pat[i].loc]) ++i;

        if (i >= m) cout << "Match at position: " << j << endl;

        j += max(adaptedGs[i], qsBc[(unsigned char) text[j + m]]);
    }
}
```

4.4 Thuật toán Skip Search

4.4.1 Giới thiệu thuật toán

Thuật toán Skip Search là một thuật toán tìm kiếm chuỗi được thiết kế dựa trên cách tiếp cận phân nhóm vị trí (bucket-based approach) đối với từng ký tự trong bảng chữ cái.

Điểm hạn chế lớn của các thuật toán cổ điển là thường phải duyệt tuần tự hoặc chỉ tận dụng thông tin dịch chuyển tại một vài vị trí cố định của cửa sổ, dẫn đến việc phải thực hiện nhiều phép so sánh ký tự không cần thiết khi kích thước bảng chữ cái lớn hoặc mẫu ngắn.

Để tối ưu hóa quá trình này, Skip Search đề xuất một cơ chế lưu trữ trước toàn bộ các vị trí xuất hiện của từng ký tự trong chuỗi mẫu vào các danh sách (gọi là các bucket).

Thuật toán có một số đặc điểm sau:

(1) Thay vì thử dịch chuyển cửa sổ theo các bước nhảy tuyến tính, thuật toán nhảy cách quãng cố định trên văn bản và sử dụng các bucket để xác định ngay lập tức các vị trí bắt đầu khả thi của mẫu.

(2) Khi chiều dài chuỗi mẫu ngắn hơn nhiều so với kích thước bảng chữ cái, phần lớn các bucket sẽ trống, giúp thuật toán nhanh chóng bỏ qua các đoạn văn bản không chứa ký tự của mẫu.

(3) Thuật toán không cần kết hợp các bảng dịch chuyển phức tạp như hậu tố tốt mà vẫn đạt được số lần kiểm tra ký tự kỳ vọng ở mức tối ưu trên thực tế.

4.4.2 Quy trình thực hiện thuật toán

Giả sử có chuỗi văn bản y độ dài n , chuỗi mẫu x độ dài m , và bảng chữ cái có kích thước $|\Sigma|$. Thuật toán Skip Search được thực hiện theo những quy trình sau

4.4.2.1 Tiền xử lý mẫu: khởi tạo các bucket vị trí

Mục tiêu của giai đoạn này là xây dựng một cấu trúc dữ liệu gồm các nhóm vị trí $z[c]$ ứng với mỗi ký tự $c \in \Sigma$.

- Khởi tạo $|\Sigma|$ nhóm trống ứng với toàn bộ các ký tự có trong bảng chữ cái.
- Duyệt qua chuỗi mẫu x từ trái sang phải. Tại mỗi vị trí i , thực hiện thu thập chỉ số vào nhóm tương ứng: $z[x[i]] = \{i \mid 0 \leq i \leq m - 1 \text{ và } x[i] = x[i]\}$

Nếu một ký tự xuất hiện k lần trong mẫu x , nhóm của ký tự đó sẽ chứa chính xác k giá trị chỉ số tương ứng với các vị trí xuất hiện của nó.

4.4.2.2 Quá trình so khớp và tìm kiếm

Quá trình tìm kiếm không trượt cửa sổ tuần tự từng ký tự mà thực hiện kiểm tra nhảy cóc trên văn bản.

(1) Vòng lặp chính (nhảy cách quãng): thuật toán thực hiện kiểm tra các ký tự tại các vị trí cách đều nhau một khoảng bằng đúng độ dài mẫu m trên văn bản y . Con trỏ văn bản j được khởi tạo tại $j = m - 1$ và tăng tiến theo quy luật $j = j + m$ sau mỗi vòng lặp xử lý, miễn là $j < n$.

(2) Xác định vị trí bắt đầu khả thi (p): tại mỗi bước nhảy j , thuật toán kiểm tra ký tự văn bản hiện tại là $y[j]$:

- Thuật toán truy xuất nhóm vị trí $z[y[j]]$ để lấy ra các chỉ số i nơi mà ký tự $y[j]$ xuất hiện trong chuỗi mẫu x (tức là $x[i] = y[j]$).
- Với mỗi chỉ số i thuộc $z[y[j]]$, thuật toán tính toán một vị trí khởi đầu tiềm năng p của mẫu x trên văn bản y theo công thức: $p = j - i$

(3) Với mỗi vị trí ứng viên p hợp lệ, thuật toán tiến hành so sánh trực tiếp, toàn diện chuỗi mẫu x với chuỗi con của văn bản y bắt đầu từ vị trí p (đoạn từ $y[p]$ đến $y[p + m - 1]$):

- Nếu tất cả các ký tự trùng khớp hoàn toàn, thuật toán ghi nhận một vị trí xuất hiện của mẫu tại p và xuất kết quả p .
- Nếu xảy ra bất kỳ một ký tự nào không trùng khớp, quá trình so khớp tại ứng viên p dừng lại ngay lập tức để chuyển sang ứng viên tiếp theo trong nhóm vị trí.

4.4.3 Đánh giá độ phức tạp thuật toán

4.4.3.1 Độ phức tạp không gian

Thuật toán Skip Search tối ưu lượng bộ nhớ dựa vào kích thước bảng chữ cái và độ dài chuỗi mẫu: cấu trúc lưu trữ cần $|\Sigma|$ con trỏ đầu để quản lý các nhóm. Tổng số phần tử được lưu trữ trong toàn bộ các nhóm chính bằng đúng chiều dài của chuỗi mẫu m (vì mỗi vị trí của mẫu chỉ thuộc về duy nhất một nhóm ký tự).

Do đó, tổng độ phức tạp không gian của thuật toán đạt mức: $S(n) = O(m + |\Sigma|)$

4.4.3.2 Độ phức tạp thời gian

(1) Giai đoạn tiền xử lý: thuật toán khởi tạo $|\Sigma|$ nhóm trống mất thời gian $O(|\Sigma|)$. Vòng lặp duyệt qua chuỗi mẫu để phân bổ các chỉ số vào các nhóm chạy chính xác m lần, tiêu tốn thời gian $O(m)$.

Do đó, tổng thời gian tiền xử lý được chặn trong giới hạn: $T_{preprocessing} = O(m + |\Sigma|)$.

(2) Giai đoạn tìm kiếm:

- Trường hợp trung bình: nhờ cơ chế chỉ kiểm tra tại các điểm cách nhau m ký tự trên văn bản, số vòng lặp chính chỉ là n/m . Khi kích thước bảng chữ cái lớn và các ký tự phân bố ngẫu nhiên, số lượng phần tử trong mỗi nhóm $z[y[j]]$ là rất nhỏ. Vậy, tổng số lần so sánh ký tự kỳ vọng trên toàn bộ văn bản đạt mức tuyến tính: $T_{average} = O(n)$

- Trường hợp xấu nhất: khi tất cả các ký tự trong chuỗi mẫu x và văn bản y đều giống hệt nhau (ví dụ: xâu toàn chuỗi ký tự 'a'), mỗi nhóm $z[y[j]]$ sẽ chứa toàn bộ m vị trí của mẫu, dẫn đến việc thuật toán phải thực hiện so khớp đầy đủ m ký tự tại mọi bước nhảy. Độ phức tạp thời gian trong trường hợp xấu nhất là: $T_{worst} = O(m \times n)$

4.4.4 Kiểm nghiệm thuật toán

4.4.4.1 Dữ liệu vào

Cho hai xâu dưới đây. Thực hiện tìm kiếm mẫu x trong văn bản y

$x = \text{"aaba"}$
 $y = \text{"aaabaaabaaabaa"}$

4.4.4.2 Quy trình kiểm nghiệm thuật toán

(1) Giai đoạn tiền xử lý

Text and Pattern character															Full string
pos j	0	1	2	3	4	5	6	7	8	9	10	11	12	13	
y[j]	a	a	b	a	a	b	a	a	c	a	a	b	a	a	aabaabaacaabaa
x[j]	a	a	b	a											aaba

Bucket z	
z[a]	3, 1, 0 // Các vị trí của kí tự trong Link List
z[b]	2
z[c]	Không

(2) Giai đoạn tìm kiếm

Kiểm nghiệm thuật toán									
Bước	Loop	j	y[j]	i trong z[y[j]]	Vị trí ứng viên (p=j-i)	y[p..p+m-1]	y[p..p+m-1] == x?	j_new = j + m	Output
1	Vòng 1	j=3	a	i=3	p=3-3=0	y[0..3]= aaba	TRUE		0
2				i=1	p=3-1=2	y[2..5]= baab	FALSE		
3				i=0	p=3-0=3	y[3..6]= aaba	TRUE	j = 3 + 4 = 7	3
4	Vòng 2	j=7	a	i=3	p=7-3=4	y[4..7]= baab	FALSE		
5				i=1	p=7-1=6	y[6..9]= aaca	FALSE		
6				i=0	p=7-0=7	y[7..10]= acaa	FALSE	j = 7 + 4 = 11	
7	Vòng 3	j=11	b	i=2	p=11-2=9	y[9..12]= aaba	TRUE	j = 11 + 4 = 15	9
8		j=15	Dừng tìm						

Sau khi kiểm nghiệm, kết quả cho thấy mẫu $x = \text{"aaba"}$ tồn tại trong văn bản $y = \text{"aaabaaabaaabaa"}$ tại 3 vị trí là $\{0, 3, 9\}$

4.4.5 Cài đặt thuật toán bằng C++

4.4.5.1 Khai báo Link List

```
const int ASIZE = 256;

struct Node {
    int element; // Vị trí trong pattern
    Node* next;
};
```

4.4.5.2 Hàm tìm kiếm bằng thuật toán Skip Search

```
// Thuật toán SKIP
void skipSearch(string pattern, string text) {
    int m = pattern.length(), n = text.length();
    vector<Node*> z(ASIZE, nullptr);

    for (int i = 0; i < m; ++i) {
        Node* ptr = new Node;
        ptr->element = i;
        ptr->next = z[(unsigned char)pattern[i]];
        z[(unsigned char)pattern[i]] = ptr;
    }

    for (int j = m - 1; j < n; j += m) {
        // Lấy linked list tương ứng ký tự cuối
        Node* ptr = z[(unsigned char)text[j]];

        while (ptr != nullptr) {
            int start = j - ptr->element; // Vị trí bắt đầu cần kiểm tra

            if (start >= 0 && start <= n - m) {
                if (text.substr(start, m) == pattern)
                    cout << "Match at position: "<< start << endl;
            }
            ptr = ptr->next;
        }
    }

    // Giải phóng linked list
    for (int i = 0; i < ASIZE; ++i) {
        Node* ptr = z[i];

        while (ptr != nullptr) {
            Node* temp = ptr;
            ptr = ptr->next;
            delete temp;
        }
    }
}
```

4.5 Thuật toán KMP Skip Search

4.5.1 Giới thiệu thuật toán

Thuật toán KMP Skip Search là một biến thể cải tiến vượt trội của thuật toán Skip Search nguyên bản. Mặc dù Skip Search có hiệu năng thực tế rất cao nhờ cơ chế nhảy cóc cách quãng và phân nhóm vị trí (bucket), thuật toán này vẫn bị giới hạn bởi độ phức tạp thời gian trong trường hợp xấu nhất lên tới mức vô cấp $O(m \times n)$. Nguyên nhân là do khi xảy ra hiện tượng sai khác, Skip Search phải hủy bỏ toàn bộ kết quả so khớp trước đó và thực hiện lại từ đầu với ứng viên tiếp theo trong bucket, dẫn đến việc các ký tự trên văn bản bị duyệt đi duyệt lại nhiều lần.

Để giải quyết triệt để điểm yếu này, KMP Skip Search kết hợp tư tưởng phân nhóm của Skip Search với các bảng dịch chuyển tiền xử lý của thuật toán Morris-Pratt (MP) và Knuth-Morris-Pratt (KMP).

Thuật toán KMP Skip Search có một số đặc điểm sau:

(1) Bằng cách tích hợp mảng tiền xử lý biên (border arrays) của KMP và MP, thuật toán tận dụng thông tin của các ký tự đã trùng khớp trước đó để dịch chuyển thông minh, đưa độ phức tạp trường hợp xấu nhất về mức tuyến tính $O(n)$.

(2) Thuật toán duy trì một biến ranh giới đại diện cho vị trí xa nhất trên văn bản đã được quét qua. Khi thực hiện so khớp tại một ứng viên mới, thuật toán không cần so sánh lại các ký tự nằm phía sau bức tường này, loại bỏ hoàn toàn các phép so sánh trùng lặp.

(3) Thuật toán tính toán song song hai khoảng dịch độc lập: một khoảng dựa trên cấu trúc phân nhóm vị trí của ký tự nhảy cóc (skipStart), và một khoảng dựa trên hàm dịch chuyển tiền tố của KMP (kmpStart). Việc đối chiếu liên tục giữa hai khoảng dịch này đảm bảo thuật toán luôn tìm được bước nhảy an toàn và tối ưu nhất.

4.5.2 Quy trình thực hiện thuật toán

Giả sử có chuỗi văn bản y độ dài n , chuỗi mẫu x độ dài m , và bảng chữ cái Σ . Thuật toán KMP Skip Search được thực hiện theo quy trình sau:

4.5.2.1 Tiền xử lý mẫu: lập bảng dịch chuyển và phân nhóm vị trí

(1) Xây dựng mảng dịch chuyển MP (mpNext): với mỗi chỉ số $1 \leq i \leq m$, giá trị $mpNext[i]$ được định nghĩa là độ dài của biên (border) dài nhất của chuỗi con $x[0 \dots i-1]$, với điều kiện khởi tạo $mpNext[0] = -1$. (Có thể xem lại thuật toán Morris-Pratt)

(2) Xây dựng mảng dịch chuyển KMP (kmpNext): với mỗi chỉ số $1 \leq i < m$, giá trị $kmpNext[i]$ là độ dài của biên dài nhất của $x[0 \dots i-1]$ mà ký tự kế tiếp chuỗi biên đó phải khác với $x[i]$. (Có thể xem lại thuật toán Knuth-Morris-Pratt)

(3) Xây dựng bảng phân nhóm vị trí phẳng (z và $list$): thuật toán sử dụng mảng z kích thước $|\Sigma|$ (khởi tạo bằng -1) để quản lý chỉ số đầu của các bucket, và mảng $list$ kích thước m (khởi tạo bằng -1) để lưu vết liên kết.

Khi duyệt mẫu x từ trái sang phải với chỉ số i chạy từ 0 đến $m-1$, ký tự $c = x[i]$ được phân bổ vào cấu trúc mảng phẳng theo quy tắc gán liên tiếp sau:

$$\begin{cases} list[i] = z[c] \\ z[c] = i \end{cases}$$

Phép gán này thực hiện chen ngược phần tử mới vào đầu bucket. Mảng $z[c]$ luôn giữ vị trí xuất hiện cuối cùng (chỉ số lớn nhất) của ký tự c , trong khi $list[i]$ đóng vai trò là con trỏ trở ngược về vị trí xuất hiện trước đó của ký tự này. Quá trình truy vết bucket kết thúc khi gặp giá trị liên kết bằng -1.

4.5.2.2 Quá trình so khớp và tìm kiếm

Thuật toán duy trì một biến $wall$ (khởi tạo bằng 0) để đánh dấu vị trí xa nhất trên văn bản đã được kiểm tra, giúp tránh so sánh trùng lặp.

(1) Khởi động cửa sổ: tịnh tiến con trỏ j trên văn bản y theo từng bước chiều dài m ($j = j + m$) cho đến khi tìm được ký tự $y[j]$ có trong mẫu ($z[y[j]] \geq 0$). Lấy chỉ số lớn nhất của ký tự này trong mẫu: $i = z[y[j]]$ và đặt vị trí bắt đầu là $start = j - i$.

(2) Thử so khớp: nếu $start > wall$, cập nhật $wall = start$.

- Tiến hành so sánh từ trái sang phải chuỗi mẫu x với văn bản y bắt đầu từ vị trí ứng viên $start$. Nhờ biến $wall$, thuật toán bỏ qua các ký tự đã biết và nhảy thẳng đến chỉ số $k = wall - start$ để so sánh tiếp:

Tìm $k \geq wall - start$ nhỏ nhất sao cho $x[k] \neq y[start + k]$ hoặc $k = m$

- Cập nhật ranh giới mới: $wall = start + k$. Nếu $k = m$, ghi nhận tìm thấy mẫu tại vị trí $start$ của văn bản và dịch lùi chỉ số mẫu theo chu kỳ:

$$i = i - per(x) = i - (m - kmpNext[m]).$$

- Nếu $k < m$, chuyển sang chỉ số tiếp theo trong bucket: $i = list[i]$.

(3) Cơ chế dịch chuyển phối hợp (KMP & skip):

Tính toán hai vị trí dịch chuyển độc lập:

$$\begin{cases} kmpStart = start + k - kmpNext[k] \\ skipStart = j - i \end{cases}$$

Thuật toán đối chiếu $skipStart$ và $kmpStart$ để quyết định vị trí cửa sổ mới theo các trường hợp sau:

- Trường hợp 1: bước dịch của *skip* quá ngắn ($\text{skipStart} < \text{kmpStart}$): thuật toán loại bỏ ứng viên hiện tại bằng cách cập nhật $i = \text{list}[i]$ (nếu hết phần tử trong bucket thì tăng j thêm m để lấy bucket mới), sau đó tính lại skipStart và quay lại kiểm tra.

- Trường hợp 2: vị trí của *skip* nằm trong vùng dữ liệu đã quét nhưng cấu trúc KMP bị lệch ($\text{kmpStart} < \text{skipStart} < \text{wall}$): thuật toán tịnh tiến kmpStart theo mảng Morris-Pratt: $\text{kmpStart} = \text{kmpStart} + (k - \text{mpNext}[k])$, cập nhật $k = \text{mpNext}[k]$

- Trường hợp 3: hai cơ chế đạt sự đồng thuận hoặc *skip* đã vượt qua ranh giới an toàn ($\text{skipStart} = \text{kmpStart}$ hoặc $\text{kmpStart} < \text{wall} \leq \text{skipStart}$): thuật toán thiết lập vị trí mới: $\text{start} = \text{skipStart}$ và quay lại bước 2 để thực hiện nỗ lực so khớp tiếp theo.

Vòng lặp tìm kiếm duy trì liên tục cho đến khi $\text{start} > n - m$.

4.5.3 Đánh giá độ phức tạp thuật toán

4.5.3.1 Độ phức tạp không gian

Thuật toán KMP Skip Search tối ưu hóa cấu trúc lưu trữ bằng cách sử dụng các mảng số nguyên một chiều có kích thước cố định thay cho con trỏ động:

- Hai mảng mpNext và kmpNext có kích thước bằng chiều dài mẫu: $2 \cdot m$.
- Mảng liên kết phẳng list thay thế danh sách liên kết có kích thước bằng: m .
- Mảng quản lý đỉnh các bucket vị trí z có kích thước bằng chữ cái: $|\Sigma|$.

Do đó, tổng độ phức tạp không gian của thuật toán đạt mức: $S(n) = O(m + |\Sigma|)$

4.5.3.2 Độ phức tạp thời gian

(1) Giai đoạn tiền xử lý:

- Thuật toán tính các bảng dịch biên Morris-Pratt và Knuth-Morris-Pratt dựa trên cấu trúc chuỗi con lặp lại của mẫu tiêu tốn thời gian $O(m)$.
- Thuật toán khởi tạo mảng bucket z mất thời gian $O(|\Sigma|)$ và phân bổ các vị trí của mẫu vào mảng phẳng list mất thời gian $O(m)$.

Do đó, độ phức tạp thời gian khi tiền xử lý là: $T_{\text{preprocessing}} = O(m + |\Sigma|)$.

(2) Giai đoạn tìm kiếm:

Trường hợp trung bình: nhờ các bước nhảy cách quãng có độ dài m trên văn bản và loại bỏ sớm các ứng viên không hợp lệ thông qua bảng KMP, thuật toán thực hiện rất ít phép so sánh ký tự đơn lẻ. Độ phức tạp thời gian trung bình tìm kiếm là: $T_{\text{average}} = O(n)$

Trường hợp xấu nhất: (các chuỗi văn bản và mẫu gồm các ký tự trùng lặp liên tục), nhờ có biến ranh giới wall ngăn chặn việc lùi con trỏ quét văn bản và cơ chế trượt tăng tốc của

mảng mpNext/kmpNext, tổng số lần so sánh ký tự trên toàn bộ văn bản y không vượt quá một hằng số tỷ lệ thuận với n. Độ phức tạp thời gian trường hợp xấu nhất là: $T_{worst} = O(n)$.

4.5.4 Kiểm nghiệm thuật toán

4.5.4.1 Dữ liệu vào

Cho hai chuỗi dưới đây. Thực hiện tìm kiếm mẫu x trong văn bản y

x="aaba"
y="aaabaaabaaba"

4.5.4.2 Quy trình kiểm nghiệm thuật toán

(1) Giai đoạn tiền xử lý

Text and Pattern character															Full string
pos j	0	1	2	3	4	5	6	7	8	9	10	11	12	13	
y[j]	a	a	b	a	a	b	a	a	c	a	a	b	a	a	aabaabaacaaba
x[j]	a	a	b	a											aaba

Xác định border và mpNext					
i	x[0..i-1]	Prefix	Suffix	Border	mpNext[i] = Longest Border length
0	""	Không	Không	Không	-1
1	a	""	""	""	0
2	aa	"", a	"", a	"", a	1
3	aab	"", a, aa	"", b, ab	""	0
4	aaba	"", a, aa, aaa	"", a, ba, aba	"", a	1

Duyệt từng border để xác định kmpNext[i]						
i	Border	x[border.size()] (1)	x[i] (2)	isTaggedBorder? = ((1) != (2))	Tagged Border	kmpNext[i] = max(taggedBorder.size())
0	Không				Không	-1
1	""	a	a	false	Không	-1
2	""	a	b	true	"", a	1
2	a	a	b	true		
3	""	a	a	false	Không	-1
4	""	a		true	"", a	1
4	a	a		true		

$$\text{per}(x) = m - \text{kmpNext}[m] \\ = 4 - 1 = 3$$

Chạy bảng z và list					
Bước	i	list	z[a]	z[b]	z[c]
0		-1, -1, -1, -1	-1	-1	-1
1	0	-1, -1, -1, -1	0	-1	-1
2	1	-1, 0, -1, -1	1	-1	-1
3	2	-1, 0, -1, -1	1	2	-1
4	3	-1, 0, -1, 1	3	2	-1

(2) Giai đoạn tìm kiếm

Kiểm nghiệm thuật toán														
Vòng lặp	j	y[j]	i	start	wall_old	k_start	k	wall_new	i_new	kmpStar t	skipStart	start_ne w	j_new	Output
1	3	a	3	0	0	0	4	start+k=4	i-per=0	3	3	skipS=3	j=3	0
	3	a	0	3	4	wall-start=1	4	start+k=7	i-per=-3	6	4		j+m=7	3
2	7	a	3	j-i=4	7			wallold=7	list[3]=1	6	6	skipS=6	j=7	
	7	a	1	6	7	wall-start=1	2	start+k=8	list[1]=0	7	7	skipS=7	j=7	
	7	a	0	7	8	wall-start=1	1	start+k=8	list[0]=-1	9	9	skipS=9	j+m=11	
3	11	b	2	j-i=9	8	0	4	start+k=13	i-per=-1				j+m=15	9
Dừng	15													

Sau khi kiểm nghiệm, kết quả cho thấy mẫu x="aaba" tồn tại trong văn bản y="aaabaaabaaabaa" tại 3 vị trí là {0, 3, 9}

4.5.5 Cài đặt thuật toán bằng C++

4.5.5.1 Hàm tiền xử lý: lập bảng mpNext

```
const int ASIZE = 256;

vector<int> preMp(const string& pattern) {
    int m = pattern.length();
    vector<int> mpNext(m + 1);

    int i = 0, j = -1;
    mpNext[0] = -1;

    while (i < m) {
        while (j > -1 && pattern[i] != pattern[j]) j = mpNext[j];
        ++i; ++j;
        mpNext[i] = j;
    }
    return mpNext;
}
```

4.5.5.2 Hàm tiền xử lý: lập bảng kmpNext

```
vector<int> preKmp(const string& pattern) {
    int m = pattern.length();
    vector<int> kmpNext(m + 1);

    int i = 0, j = -1;
    kmpNext[0] = -1;

    while (i < m) {
        while (j > -1 && pattern[i] != pattern[j]) j = kmpNext[j];
        ++i; ++j;
        // Nếu ký tự tiếp theo cũng giống nhau thì thừa kế border
        if (i < m && pattern[i] == pattern[j]) kmpNext[i] = kmpNext[j];
        else kmpNext[i] = j;
    }
    return kmpNext;
}
```

4.5.5.3 Hàm tìm kiếm bằng thuật toán KMP Skip Search

```
// Hàm kiểm tra match bắt đầu từ start
int attempt(const string& text, const string& pattern, int start, int wall) {
    int m = pattern.length(), k = wall - start; // k là vị trí bắt đầu từ phần đã
    biết match
    while (k < m && pattern[k] == text[k + start]) ++k;
    return k;
}

void kmpSkipSearch(string pattern, string text) {
    int m = pattern.length(), n = text.length();

    vector<int> mpNext = preMp(pattern), kmpNext = preKmp(pattern);
    vector<int> list(m, -1), z(ASIZE, -1);
    z[(unsigned char)pattern[0]] = 0;

    for (int i = 1; i < m; ++i) {
        list[i] = z[(unsigned char)pattern[i]];
        z[(unsigned char)pattern[i]] = i;
    }

    int wall = 0, per = m - kmpNext[m]; // per là period của pattern
    int i = -1, j = -1;

    // Tìm block đầu tiên có ký tự xuất hiện
    do j += m;
    while (j < n && z[(unsigned char)text[j]] < 0); // Tìm block đầu tiên có ký tự
    xuất hiện

    if (j >= n) return; // Không tìm thấy

    i = z[(unsigned char)text[j]]; // Vị trí ký tự trong pattern

    int start = j - i;
    while (start <= n - m) {
        if (start > wall) wall = start;

        int k = attempt(text, pattern, start, wall); // Thử match

        wall = start + k;

        if (k == m) {
            cout << "Match at position: " << start << endl;
            i -= per;
        }
        else i = list[i];

        // Nếu không còn candidate
        if (i < 0) {
            // Tìm block tiếp theo

```

```
do j += m;
while (j < n && z[(unsigned char) text[j]] < 0);

if (j >= n) return; // Không còn block nào

i = z[(unsigned char)text[j]];
}

// Vị trí bắt đầu theo KMP
int kmpStart = start + k - kmpNext[k];

k = kmpNext[k];
start = j - i;

// Đồng bộ Skip + KMP
while (start < kmpStart || (kmpStart < start && start < wall)) {
    // Nếu start quá nhỏ
    if (start < kmpStart) {
        i = list[i];

        // Nếu hết candidate
        if (i < 0) {
            // Tìm block tiếp theo
            do j += m;
            while (j < n && z[(unsigned char) text[j]] < 0);

            if (j >= n) return; // Không còn block nào

            i = z[(unsigned char) text[j]];
        }
        start = j - i;
    }
    // Nếu KMP cần tiến thêm
    else {
        kmpStart += (k - mpNext[k]);
        k = mpNext[k];
    }
}
}
```

CHƯƠNG 5: CÁC THUẬT TOÁN TÌM KIẾM MẪU TỪ VỊ TRÍ BẤT KỲ

5.1 Thuật toán Horspool

5.1.1 Giới thiệu thuật toán

Thuật toán Horspool là một biến thể đơn giản hóa nhưng mang lại hiệu năng thực tế cao của thuật toán Boyer-Moore nguyên bản.

Ý tưởng cốt lõi của Horspool xuất phát từ nhận định: cơ chế ký tự xấu (Bad-character shift) trong Boyer-Moore tuy không mấy hiệu quả trên các bảng chữ cái kích thước nhỏ, nhưng lại trở nên vô cùng mạnh mẽ khi kích thước bảng chữ cái lớn hơn nhiều so với độ dài của mẫu (ví dụ điển hình là bảng mã ASCII trong các trình soạn thảo văn bản thông thường).

Thay vì phải duy trì cả hai bảng dịch chuyển phức tạp (ký tự xấu và dịch chuyển tốt), Horspool đề xuất một cải tiến là chỉ sử dụng duy nhất giá trị ký tự xấu của ký tự nằm ngoài cùng bên phải của cửa sổ tìm kiếm hiện tại để tính toán khoảng trượt cho toàn bộ cửa sổ.

Thuật toán Horspool có các ưu điểm sau:

(1) Loại bỏ hoàn toàn các bước tính toán tiền xử lý phức tạp của hàm dịch chuyển tốt, giúp mã nguồn trở nên gọn nhẹ, ít lỗi và dễ bảo trì.

(2) Khi tìm kiếm trên các văn bản tự nhiên sử dụng bảng chữ cái lớn (như tiếng Anh, mã nguồn), việc chỉ dựa vào ký tự cuối cùng giúp cửa sổ trượt đi những khoảng rất dài, thường đạt tốc độ tương đương hoặc nhanh hơn cả thuật toán Boyer-Moore gốc.

(3) Số lượng phép so sánh ký tự tính trên một ký tự văn bản trong trường hợp trung bình được chứng minh toán học nằm trong khoảng rất thấp, dao động từ $\frac{1}{|\Sigma|}$ đến $\frac{2}{|\Sigma|+1}$.

5.1.2 Quy trình thực hiện thuật toán

Giả sử có chuỗi văn bản y độ dài n và chuỗi mẫu x độ dài m . Thuật toán Horspool được thực hiện chi tiết qua các giai đoạn sau:

5.1.2.1 Tiền xử lý mẫu: lập bảng $bmBc$ (Boyer-Moore Bad Character)

Giai đoạn này chuẩn bị bảng dịch chuyển ký tự xấu một chiều: $bmBc$

(1) Khởi tạo tất cả các ô trong mảng $bmBc$ với kích thước bảng chữ cái Σ bằng chính chiều dài của mẫu m .

(2) Duyệt qua mẫu từ trái sang phải, với mỗi ký tự $x[i]$ từ chỉ số 0 đến $m-2$, cập nhật khoảng cách từ vị trí đó đến cuối mẫu: $bmBc[x[i]] = m - 1 - i$

- *Lưu ý: Ký tự cuối cùng của mẫu ($x[m-1]$) không được tính trong vòng lặp này để tránh việc giá trị trượt bị cập nhật về 0, gây ra vòng lặp vô hạn trong giai đoạn tìm kiếm.*

5.1.2.2 Quá trình so khớp và tìm kiếm

Vị trí bắt đầu của cửa sổ tìm kiếm trên văn bản được thiết lập tại $j = 0$. Vòng lặp chính duy trì khi cửa sổ chưa vượt quá giới hạn văn bản ($j \leq n - m$):

(1) Xác định ký tự chốt chặn:

Thuật toán lấy ra ký tự nằm ở vị trí cuối cùng bên phải của cửa sổ hiện tại trên văn bản: $c = y[j + m - 1]$ (theo lý thuyết, có thể chọn bất kỳ vị trí nào để so sánh trước, nhưng thông thường, ký tự cuối mẫu được chọn trước để tối ưu so sánh)

(2) So khớp đồng thời:

Thuật toán tiến hành kiểm tra hai điều kiện sau đây:

- Ký tự cuối cùng của mẫu phải khớp với ký tự chốt chặn ($x[m - 1] == c$)
- Toàn bộ đoạn tiền tố dài $m-1$ ký tự còn lại của mẫu phải khớp hoàn toàn với văn bản tại vị trí tương ứng.

Nếu cả hai điều kiện trên đều thỏa mãn: ghi nhận vị trí xuất hiện của mẫu là j

(3) Dịch chuyển cửa sổ: bất kể lượt đối sánh trên kết thúc là khớp hoàn toàn (Match) hay không khớp, thuật toán điều tra cứu bảng ký tự xấu dựa trên ký tự chốt chặn c để quyết định khoảng cách trượt tiếp theo: $j = j + \text{bmBc}[c]$

5.1.3 Đánh giá độ phức tạp thuật toán

5.1.3.1 Độ phức tạp không gian

Thuật toán Horspool chỉ sử dụng duy nhất một mảng một chiều lưu trữ giá trị dịch chuyển cho toàn bộ các ký tự có thể xuất hiện trong bảng chữ cái: mảng bmBc có kích thước cố định bằng kích thước bảng chữ cái $|\Sigma|$, hoàn toàn độc lập với chiều dài mẫu m .

Do đó, tổng độ phức tạp không gian của thuật toán đạt mức: $S(n) = O(|\Sigma|)$

5.1.3.2 Độ phức tạp thời gian

(1) Giai đoạn tiền xử lý: thuật toán thực hiện:

- Lập khởi tạo mảng mất thời gian phụ thuộc vào kích thước bảng chữ cái với độ phức tạp là $O(|\Sigma|)$.
- Lập cập nhật khoảng trượt chạy dọc theo chiều dài mẫu trong thời gian $O(m)$.

Do đó, tổng thời gian dành cho quá trình tiền xử lý của thuật toán Horspool được chặn ở mức: $T_{\text{preprocessing}} = O(m + |\Sigma|)$

(2) Giai đoạn tìm kiếm:

Trường hợp trung bình: nhờ cơ chế chỉ nhảy dựa trên ký tự cuối cùng giúp tối đa hóa bước nhảy trên văn bản tự nhiên, số phép so sánh trung bình trên một ký tự văn bản là rất nhỏ (chỉ khoảng $\frac{1}{|\Sigma|}$ đến $\frac{2}{|\Sigma|+1}$). Độ phức tạp tiệm cận tuyến tính: $T_{average} \approx O(n)$

Trường hợp xấu nhất: Khi gặp phải các chuỗi có cấu trúc lặp vô hạn tuần hoàn, mỗi bước dịch chuyển của sổ chỉ tịnh tiến được đúng 1 đơn vị, dẫn đến việc phải so sánh lại toàn bộ của sổ. Độ phức tạp thời gian trong trường hợp tệ nhất là: $T_{worst} = O(m \times n)$

5.1.4 Kiểm nghiệm thuật toán

5.1.4.1 Dữ liệu vào

Cho hai xâu dưới đây. Thực hiện tìm kiếm mẫu x trong văn bản y

x="aaba"
y="aaabaaabaabaa"

5.1.4.2 Quy trình kiểm nghiệm thuật toán

(1) Pha tiền xử lý

Text and Pattern character															Full string
pos j	0	1	2	3	4	5	6	7	8	9	10	11	12	13	
y[j]	a	a	b	a	a	b	a	a	c	a	a	b	a	a	aabaabaacaabaa
x[j]	a	a	b	a											aaba

Chạy bảng bmBc					
Bước	i	pattern [i]	bmBc[a]	bmBc[b]	bmBc [khác]
0	-	-	4	4	4
1	0	a	3	4	4
2	1	a	2	4	4
3	2	b	2	1	4

Bảng bmBc	
chr	bmBc [chr]
a	2
b	1
khác	4

(2) Pha tìm kiếm

Kiểm nghiệm thuật toán							
j	c= y[j+m-1]	x[m-1] ==c?	x[0..m-2] == y[j..j+m-2]?	Cập nhật trạng thái Giải thích	bmBc[c]	j mới	Output
0	a(y[3])	Đúng ('a'=='a')	Khớp("aab" == "aab")	Khớp hoàn toàn. Cửa sổ trượt dựa theo ký tự chốt 'a'.	2	0+2=2	0
2	b(y[5])	Sai ('a'!='b')	Không xét	Không khớp ký tự cuối. Trượt dựa theo ký tự chốt 'b'.	1	2+1=3	
3	a(y[6])	Đúng ('a'=='a')	Khớp("aab" == "aab")	Khớp hoàn toàn. Cửa sổ trượt dựa theo ký tự chốt 'a'.	2	3+2=5	3
5	a(y[8])	Đúng ('a'=='a')	Không khớp("aab" != "aca")	Trượt theo ký tự chốt 'a'.	2	5+2=7	
7	a(y[10])	Đúng ('a'=='a')	Không khớp("aab" != "caa")	Ký tự cuối khớp nhưng bị lệch bên trong. Trượt theo ký tự chốt 'a'.	2	7+2=9	
9	a(y[12])	Đúng ('a'=='a')	Khớp("aab" == "aab")	Khớp hoàn toàn. Cửa sổ trượt dựa theo ký tự chốt 'a'.	2	9+2=11	9
11				DỪNG vì j=11>(n-m=10). Thuật toán kết thúc.			

Sau khi kiểm nghiệm, kết quả cho thấy mẫu x="aaba" tồn tại trong văn bản y="aaabaaabaabaa" tại 3 vị trí là {0, 3, 9}

5.1.5 Cài đặt thuật toán với C++

5.1.5.1 Hàm lập bảng Bad Character (theo thuật toán Boyer-Moore)

```
const int ASIZE = 256;
// Tiền xử lý bảng bad character
void preBmBc(const string &pattern, vector<int> &bmBc) {
    int m = pattern.size();
    bmBc.assign(ASIZE, m);

    for (int i = 0; i < m - 1; i++) bmBc[(unsigned char)pattern[i]] = m - i - 1;
}
```

5.1.5.2 Hàm tìm kiếm bằng thuật toán Horspool

```
// Thuật toán Horspool
void horspool(const string &pattern, const string &text) {
    int m = pattern.size();
    int n = text.size();
    vector<int> bmBc;

    preBmBc(pattern, bmBc);

    int j = 0;

    while (j <= n - m) {
        char c = text[j + m - 1];

        // So sánh ký tự cuối trước
        if (pattern[m - 1] == c &&
            memcmp(pattern.c_str(), text.c_str() + j, m - 1) == 0) {
            cout << "Match at position: " << j << endl;
        }
        j += bmBc[(unsigned char)c];
    }
}
```

5.2 Thuật toán Quick Search

5.2.1 Giới thiệu thuật toán

Thuật toán Quick Search là một sự đơn giản hóa và cải tiến từ cơ chế ký tự xấu (Bad-character shift) của thuật toán Boyer-Moore.

Ý tưởng của thuật toán là sau bất kỳ một lượt thử nào, cửa sổ tìm kiếm chắc chắn sẽ phải trượt sang phải ít nhất là 1 đơn vị. Điều này có nghĩa là ký tự nằm ngay phía sau cửa sổ hiện tại chắc chắn sẽ tham gia vào lượt thử kế tiếp.

Do đó, thay vì dùng một ký tự bên trong cửa sổ, thuật toán Quick Search chọn hần ký tự đầu tiên nằm ngay bên phải, sát cạnh cửa sổ hiện tại để tính toán khoảng cách dịch chuyển.

Thuật toán Quick Search có các ưu điểm sau:

(1) Bước nhảy luôn được tối ưu hóa thêm 1 đơn vị: bằng cách nhìn trước một ký tự ngoài cửa sổ, khoảng trượt tối đa của Quick Search đạt tới $m + 1$ (trong khi Boyer-Moore và Horspool tối đa chỉ là m).

(2) Khác với các thuật toán khác ép buộc phải duyệt từ phải sang trái hoặc theo mảng chỉ số cố định, trong Quick Search, việc đối chiếu các ký tự bên trong cửa sổ có thể thực hiện theo bất kỳ thứ tự nào (từ trái sang phải, từ phải sang trái theo mức ưu tiên ký tự,...).

(3) Trong thực tế, đối với các chuỗi mẫu có độ dài ngắn và tìm kiếm trên tập dữ liệu sử dụng bảng chữ cái lớn, Quick Search thường là một trong những thuật toán cho tốc độ trượt cửa sổ nhanh nhất.

5.2.2 Quy trình thực hiện thuật toán

Giả sử có chuỗi văn bản y độ dài n và chuỗi mẫu x độ dài m . Thuật toán Quick Search được thực hiện chi tiết qua các giai đoạn sau:

5.2.2.1 Tiền xử lý mẫu: lập bảng $qsBc$ (Quick Search Bad Character)

Giai đoạn này chuẩn bị bảng dịch chuyển ký tự xấu sửa đổi Quick Search Bad Character:

(1) Khởi tạo bảng $qsBc$ với kích thước bảng chữ cái Σ , tất cả các phần tử ban đầu được gán giá trị mặc định là $m + 1$.

(2) Duyệt qua toàn bộ mẫu x từ trái sang phải từ chỉ số 0 đến $m-1$. Tại mỗi vị trí i , khoảng cách dịch chuyển được tính từ vị trí đó đến mốc ngay sau mẫu: $qsBc[x[i]] = m - i$

Lưu ý: Vòng lặp này duyệt đến hết ký tự cuối cùng ($i = m - 1$), do đó nếu ký tự cuối cùng của mẫu xuất hiện, giá trị trượt nhỏ nhất của nó sẽ là $m - (m - 1) = 1$.

5.2.2.2 Quá trình so khớp và tìm kiếm

Vị trí bắt đầu của cửa sổ tìm kiếm trên văn bản được thiết lập tại $j = 0$. Vòng lặp chính duy trì khi cửa sổ chưa vượt quá giới hạn văn bản ($j \leq n - m$):

(1) So khớp toàn bộ cửa sổ:

Thuật toán tiến hành so sánh đồng thời toàn bộ m ký tự của mẫu x với đoạn văn bản tương ứng trong cửa sổ $y[j \dots j + m - 1]$

Nếu toàn bộ các ký tự trùng khớp hoàn toàn, ghi nhận tìm thấy mẫu tại vị trí j .

(2) Dịch chuyển cửa sổ bằng ký tự kế tiếp:

Bất kể kết quả so khớp là đúng hay sai, thuật toán lập tức nhìn vào ký tự nằm ngay sau cửa sổ hiện tại trên văn bản, tức là ký tự tại vị trí $j + m$: $c = y[j + m]$

Tra cứu bảng $qsBc$ với ký tự c này để thực hiện tịnh tiến cửa sổ: $j = j + qsBc[c]$

5.2.3 Đánh giá độ phức tạp thuật toán

5.2.3.1 Độ phức tạp không gian

Thuật toán Quick Search sử dụng một cấu trúc mảng tĩnh: mảng một chiều $qsBc$ có kích thước cố định bằng kích thước bảng chữ cái $|\Sigma|$.

Do đó, tổng độ phức tạp không gian của thuật toán là: $S(n) = O(|\Sigma|)$

5.2.3.2 Độ phức tạp thời gian

(1) Giai đoạn tiền xử lý: thuật toán thực hiện:

- Lập khởi tạo mảng phụ thuộc vào kích thước bảng chữ cái với thời gian $O(|\Sigma|)$.
- Lập tính toán giá trị trượt chạy dọc theo chiều dài mẫu với thời gian $O(m)$.

Do đó, tổng thời gian dành cho tiền xử lý của Quick Search là:

$$T_{preprocessing} = O(m + |\Sigma|)$$

(2) Giai đoạn tìm kiếm:

Trường hợp trung bình: nhờ việc tận dụng ký tự nằm ngoài cửa sổ giúp tăng khả năng nhận được bước nhảy tối đa ($m+1$), thuật toán giảm thiểu đáng kể số lần dịch chuyển và số lần kiểm tra ký tự. Hiệu năng đạt tốc độ tuyến tính: $T_{average} = O(n)$

Trường hợp xấu nhất: tương tự các thuật toán họ Boyer-Moore giản lược khác, trong kịch bản tệ nhất (văn bản và mẫu đều cấu thành từ một ký tự lặp lại duy nhất), thuật toán sẽ phải so sánh m ký tự tại mỗi bước và chỉ dịch chuyển được 1 đơn vị. Độ phức tạp thời gian trường hợp xấu nhất là: $T_{worst} = O(m \times n)$

5.2.4 Kiểm nghiệm thuật toán

5.2.4.1 Dữ liệu vào

Cho hai chuỗi dưới đây. Thực hiện tìm kiếm mẫu x trong văn bản y

x="aaba"

y="aaabaaabaabaa"

5.2.4.2 Quy trình kiểm nghiệm thuật toán

(1) Giai đoạn tiền xử lý

Text and Pattern character															Full string
pos j	0	1	2	3	4	5	6	7	8	9	10	11	12	13	
y[j]	a	a	b	a	a	b	a	a	c	a	a	b	a	a	aabaabaacaabaa
x[j]	a	a	b	a											aaba

Chạy bảng qsBc						
Bước	i	pattern [i]	qsBc[a]	qsBc[b]	qsBc [khác]	Giải thích qsBc[x[i]] = m-i
0	-	-	5	5	5	
1	0	a	4	5	5	qsBc[a]=4-0=4
2	1	a	3	5	5	qsBc[a]=4-1=3
3	2	b	3	2	5	qsBc[b] = 4-2=2
3	2	b	1	2	5	qsBc[a] = 4-3=1

Bảng qsBc	
chr	qsBc [chr]
a	1
b	2
khác	5

(2) Giai đoạn tìm kiếm

Kiểm nghiệm thuật toán							
j	y[j..j+m-1]	x (aaba) == y[j..j+m-1]?	c = y[j+m]	Giải thích	qsBc[c]	j mới	Output
0	"aaba"	Khớp	a (y[4])	Khớp hoàn toàn. Nhìn ký tự kế tiếp là 'a' để dịch chuyển.	1	0+1=1	Match 0
1	"abaa"	Không khớp	b (y[5])	Lệch ký tự. Nhìn ký tự kế tiếp là 'b' để dịch chuyển.	2	1+2=3	
3	"aaba"	Khớp	a (y[7])	Khớp hoàn toàn. Nhìn ký tự kế tiếp là 'a' để dịch chuyển.	1	3+1=4	Match 3
4	"abaa"	Không khớp	c (y[8])	Lệch ký tự. Nhìn ký tự kế tiếp là 'c' để dịch chuyển.	5	4+5=9	
9	"aaba"	Khớp	a (y[13])	Khớp hoàn toàn. Nhìn ký tự kế tiếp là 'a' để dịch chuyển.	1	9+1=10	Match 9
10	"abaa"	Không khớp	Vượt biên (y[14])	Lệch ký tự. Không thể trượt tiếp do vòng lặp chính kết thúc.	—	Kết thúc	

Sau khi kiểm nghiệm, kết quả cho thấy mẫu x="aaba" tồn tại trong văn bản y="aaabaaabaabaa" tại 3 vị trí là {0, 3, 9}

5.2.5 Cài đặt thuật toán với C++

5.2.5.1 Hàm tiền xử lý: lập bảng qsBc (Quick Search Bad Character)

```
const int ASIZE = 256;

void preQsBc(const string &pattern, vector<int> &qsBc) {
    int m = pattern.length();

    // Khởi tạo mặc định m + 1 cho mọi ký tự
    for (int i = 0; i < ASIZE; ++i) qsBc[i] = m + 1;
    for (int i = 0; i < m; ++i) qsBc[(unsigned char)pattern[i]] = m - i;
}
```

5.2.5.2 *Hàm tìm kiếm bằng thuật toán QuickSearch*

```
// Thuật toán Quick Search
void quickSearch(const string &pattern, const string &text) {
    int m = pattern.length();
    int n = text.length();

    vector<int> qsBc(ASIZE);

    // Tiền xử lý
    preQsBc(pattern, qsBc);

    int j = 0;

    // Duyệt trên text
    while (j <= n - m) {
        // So sánh pattern với đoạn hiện tại của text
        if (text.substr(j, m) == pattern)
            cout << "Match at position: " << j << endl;

        // Nếu đã tới cuối thì dừng
        if (j + m >= n) break;

        // Dịch chuyển theo bảng qsBc
        j += qsBc[(unsigned char)text[j + m]];
    }
}
```

5.3 Thuật toán Tuned Boyer-Moore

5.3.1 Giới thiệu thuật toán

Thuật toán Tuned Boyer-Moore là một biến thể tối ưu hóa và đơn giản hóa của thuật toán Boyer-Moore nguyên bản.

Tác giả của thuật toán đã nhận diện được điểm nghẽn (bottleneck) lớn nhất làm giảm hiệu năng của các thuật toán tìm kiếm chuỗi. Đó là việc phải liên tục thực hiện các điều kiện kiểm tra vòng lặp và đối sánh ký tự ngay cả khi cửa sổ chưa nằm ở một vị trí tiềm năng.

Để giải quyết triệt để vấn đề này, Tuned Boyer-Moore đề xuất cơ chế trượt mù (blind shifts) bằng kỹ thuật trải vòng lặp (loop unrolling) kết hợp với ký tự lính canh (sentinel).

Thuật toán tập trung tối đa vào việc tìm kiếm ký tự cuối cùng của mẫu xuất hiện trên văn bản một cách nhanh nhất có thể. Bằng cách thực hiện liên tiếp 3 bước dịch chuyển trong một chu kỳ lặp mà không cần kiểm tra điều kiện biên hay đối sánh các ký tự khác, Tuned Boyer-Moore giảm thiểu tối đa chi phí rẽ nhánh của CPU.

Đây là một trong những thuật toán có tốc độ thực thi trong thực tế nhanh nhất, đặc biệt hiệu quả trên các hệ thống máy tính hiện đại nhờ tối ưu hóa được dòng lệnh nạp trước của phần cứng.

5.3.2 Quy trình thực hiện thuật toán

Giả sử có chuỗi văn bản y độ dài n và chuỗi mẫu x độ dài m . Thuật toán Tuned Boyer-Moore được thực hiện theo quy trình sau:

5.3.2.1 Tiền xử lý mẫu: lập bảng $bmBc$, khóa vòng lặp, đặt lính canh

(1) Xây dựng bảng ký tự xấu $bmBc$: tính toán khoảng cách từ các vị trí xuất hiện của ký tự trong mẫu (ngoại trừ ký tự cuối cùng) đến đuôi mẫu. Các ký tự không nằm trong mẫu nhận giá trị mặc định là m .

(2) Cấu hình bước nhảy lặp: lưu giá trị dịch chuyển của ký tự cuối cùng của mẫu vào một biến độc lập để làm khoảng trượt chuẩn khi có sự kiện khớp chuỗi xảy ra.

(3) Thiết lập trạng thái khóa vòng lặp: sửa đổi giá trị dịch chuyển của ký tự cuối cùng của mẫu trong bảng ký tự xấu về bằng 0. Giá trị 0 này đóng vai trò như một "tín hiệu dừng" để bẻ gãy vòng lặp trượt mù khi tìm thấy ký tự cần thiết.

(4) Đặt lính canh (Sentinel) trên văn bản: kéo dài vùng đệm phía sau văn bản bằng cách chèn thêm m ký tự trùng với ký tự cuối cùng của mẫu. Việc này đảm bảo vòng lặp trượt mù luôn tìm thấy mục tiêu và dừng lại một cách an toàn mà không bao giờ bị tràn (out of bounds) ra ngoài bộ nhớ văn bản.

5.3.2.2 Quá trình so khớp và tìm kiếm

Vị trí của sổ ban đầu được thiết lập tại chỉ số $j = 0$. Vòng lặp chính duy trì liên tục khi điều kiện $j < n$ còn thỏa mãn:

(1) Vòng lặp trượt mù (fast loop)

Tra cứu giá trị dịch chuyển của ký tự cuối của sổ ($y[j + m - 1]$):

- Nếu khác 0: Cộng dồn giá trị vào j để tịnh tiến cửa sổ. Quá trình này được trải ra lặp lại liên tiếp 3 lần trong một chu kỳ để bỏ qua việc kiểm tra điều kiện biên.
- Nếu bằng 0: Vòng lặp dừng lại ngay lập tức. Lúc này cửa sổ đã được định vị tại vị trí có ký tự cuối trùng khớp với mẫu.

(2) So khớp đoạn tiền tố: tiến hành đối chiếu đồng thời $m - 1$ ký tự tiền tố còn lại của mẫu với văn bản:

- Thứ tự đối chiếu các ký tự bên trong cửa sổ có thể diễn ra tự do (có thể từ trái sang phải, phải sang trái, hoặc theo thứ tự ưu tiên).
- Nếu đoạn tiền tố trùng khớp hoàn toàn và vị trí hiện tại hợp lệ ($j < n$), ghi nhận một vị trí xuất hiện của mẫu là j .

(3) Dịch chuyển: bất kể kết quả đối chiếu tiền tố ở bước trước là đúng hay sai, thuật toán sử dụng giá trị dịch chuyển chuẩn của mẫu đã được lưu từ giai đoạn tiền xử lý để tịnh tiến cửa sổ: $j = j + \text{shift}$

5.3.3 Đánh giá độ phức tạp thuật toán

5.3.3.1 Độ phức tạp không gian

Thuật toán duy trì một lượng bộ nhớ rất nhỏ gọn:

- Một mảng một chiều lưu trữ giá trị dịch chuyển với kích thước cố định bằng kích thước bảng chữ cái $|\Sigma|$.
- Yêu cầu văn bản đầu vào cần có một khoảng trống dự phòng ở cuối để chứa m ký tự lính canh.

Do đó, tổng độ phức tạp không gian hỗ trợ được tính là: $\mathcal{O}(|\Sigma|)$

5.3.3.2 Độ phức tạp thời gian

(1) Giai đoạn tiền xử lý:

- Thời gian khởi tạo và tính toán bảng ký tự xấu phụ thuộc vào kích thước bảng chữ cái và chiều dài mẫu là $\mathcal{O}(m + |\Sigma|)$.

- Việc gán lính canh ở cuối văn bản mất thời gian không đáng kể $O(m)$.

Do vậy, tổng thời gian tiền xử lý đạt mức tuyến tính: $T_{preprocessing} = O(m + |\Sigma|)$

(2) Giai đoạn tìm kiếm:

Trường hợp trung bình: nhờ cơ chế trải vòng lặp giải phóng tối đa tài nguyên xử lý của CPU, số lượng tập lệnh điều hướng giảm đi đáng kể. Thuật toán đạt tốc độ trượt tối ưu và thể hiện hiệu năng thực tế vượt trội, tiệm cận thời gian tuyến tính $T_{average} = O(n)$.

Trường hợp xấu nhất: khi văn bản cấu thành từ các chuỗi liên tục tuần hoàn lặp lại dẫn đến việc vòng lặp trượt mù bị dừng liên tục ở mỗi ký tự đơn lẻ, thuật toán phải quay về thực hiện so sánh toàn bộ các ký tự trong cửa sổ tại mỗi bước tịnh tiến tối thiểu. Độ phức tạp thời gian trường hợp xấu nhất là: $T_{worst} = O(m \times n)$

5.3.4 Kiểm nghiệm thuật toán

5.3.4.1 Dữ liệu vào

Cho hai xâu dưới đây. Thực hiện tìm kiếm mẫu x trong văn bản y

x="aaba"
y="aaabaaabaaba"

5.3.4.2 Quy trình kiểm nghiệm thuật toán

(1) Giai đoạn tiền xử lý

Text and Pattern character															Full string
pos j	0	1	2	3	4	5	6	7	8	9	10	11	12	13	
y[j]	a	a	b	a	a	b	a	a	c	a	a	b	a	a	aabaabaacaabaa
x[j]	a	a	b	a											aaba

Chạy bảng bmBc						
Bước	i	pattern [i]	bmBc[a]	bmBc[b]	bmBc [khác]	Giải thích
0	-	-	4	4	4	
1	0	a	3	4	4	bmBc[a]=4-1-0=3
2	1	a	2	4	4	bmBc[a]=4-1-1=2
3	2	b	2	1	4	bmBc[b] = 4-1-2=1
shift = bmBc[x[m-1]] = bmBc[a] = 2						
Khóa: bmBc[x[m-1]] bmBc[a] = 0						

Bảng bmBc	
chr	bmBc [chr]
a	0
b	1
khác	4

Đặt lính canh: thêm m ký tự x[m-1] vào text				
pos j	14	15	16	17
y[j]	a	a	a	a

(2) Giai đoạn tìm kiếm

Kiểm nghiệm thuật toán							
Bước	j	y[j+m-1]	k = bmBc[j]	Trạng thái Fast Loop	x[0..m-2]==y[j..j+m-2]	Kết quả / Hành động tiếp theo	Output
1	0	a' (y[3])	0	Dừng (do k==0).	Khớp("aab" == "aab")	Khớp toàn bộ. j=j+shift=0+2 = 2.	0
2	2	b' (y[5])	1	Tiếp tục		j=2+1=3.	
3	3	a' (y[6])	0	Dừng (do k==0).	Khớp("aab" == "aab")	Khớp toàn bộ. j=3+shift (2)=5.	3
4	5	a' (y[8])	0	Dừng (do k==0).	Không khớp("aab" != "aca")	Sai khác. j=5+shift (2)=7.	
5	7	a' (y[10])	0	Dừng (do k==0).	Không khớp("aab" != "caa")	Sai khác. j=7+shift (2)=9.	
6	9	a' (y[12])	0	Dừng (do k==0).	Khớp("aab" == "aab")	Khớp toàn bộ. j=9+shift (2)=11.	9
7	11	a' (y[14])	0	Dừng (do k==0).		(j≥n).	

5.3.5 Cài đặt thuật toán bằng C++

5.3.5.1 Hàm lập bảng Bad Character (bmBc)

```
const int ASIZE = 256;

// Tiền xử lý bảng bad character
void preBmBc(const string &pattern, vector<int> &bmBc) {
    int m = pattern.size();

    // Gán mặc định mọi giá trị bằng độ dài pattern
    bmBc.assign(ASIZE, m);

    // Tính khoảng dịch cho từng ký tự
    for (int i = 0; i < m - 1; i++) {
        bmBc[(unsigned char)pattern[i]] = m - i - 1;
    }
}
```

5.3.5.2 Hàm tìm kiếm bằng thuật toán TunedBM

```
// Thuật toán Tuned Boyer-Moore
void tunedBoyerMoore(const string &pattern, const string &text) {
    int m = pattern.size();
    int n = text.size();

    vector<int> bmBc;

    // Tiền xử lý bảng bad character
    preBmBc(pattern, bmBc);

    // Lấy giá trị dịch của ký tự cuối
    int shift = bmBc[(unsigned char)pattern[m - 1]];

    // Đặt ký tự cuối có giá trị dịch bằng 0
    bmBc[(unsigned char)pattern[m - 1]] = 0;

    // Tạo chuỗi phụ để tránh truy cập vượt biên
    string tempText = text + string(m, pattern[m - 1]);

    int j = 0;
```

```
// Bắt đầu tìm kiếm
while (j < n) {

    // Lấy khoảng dịch theo ký tự cuối hiện tại
    int k = bmBc[(unsigned char)tempText[j + m - 1]];

    // Dịch nhanh nhiều lần liên tiếp
    while (k != 0) {
        j += k;
        k = bmBc[(unsigned char)tempText[j + m - 1]];

        j += k;
        k = bmBc[(unsigned char)tempText[j + m - 1]];

        j += k;
        k = bmBc[(unsigned char)tempText[j + m - 1]];
    }

    // So sánh m - 1 ký tự đầu
    if (tempText.substr(j, m - 1) == pattern.substr(0, m - 1) && j < n) {
        cout << "Tim thay tai vi tri: " << j << endl;
    }

    // Dịch theo shift
    j += shift;
}
}
```

5.4 Thuật toán Smith

5.4.1 Giới thiệu thuật toán

Thuật toán Smith được đề xuất dựa trên một nhận xét của tác giả. Việc chỉ nhìn vào ký tự ngay sau cửa sổ (thuật toán Quick Search) để nhảy đôi khi lại cho một khoảng dịch ngắn hơn so với việc nhìn vào ký tự cuối cùng bên trong cửa sổ (thuật toán Horspool), và ngược lại. Ý tưởng này xuất phát từ việc cấu trúc của chuỗi mẫu có thể chứa các tập ký tự trùng lặp phân bố không đều ở đuôi.

Để tối ưu hóa độ dài của mỗi bước nhảy, Smith đề xuất giải pháp chọn giá trị lớn nhất giữa hai hàm dịch chuyển ký tự xấu này tại mỗi lượt thử.

Thuật toán có các đặc điểm sau:

(1) Bằng cách lấy $\max(\text{bmBc}, \text{qsBc})$, thuật toán luôn đảm bảo cửa sổ trượt đi một khoảng cách dài nhất có thể trong phạm vi thông tin thu được từ cả hai cơ chế.

(2) Thuật toán khắc phục được các điểm mù của riêng từng thuật toán khi gặp phải các cấu trúc chuỗi lặp đặc thù.

Giống như Quick Search, do khoảng trượt hoàn toàn được quyết định bởi hai ký tự biên, việc đối chiếu các ký tự bên trong cửa sổ có thể thực hiện theo bất kỳ thứ tự nào.

5.4.2 Quy trình thực hiện thuật toán

Giả sử có chuỗi văn bản y độ dài n và chuỗi mẫu x độ dài m . Quy trình được thực hiện qua các giai đoạn sau:

5.4.2.1 Tiền xử lý mẫu: lập bảng bmBc và qsBc

(1) Xây dựng bảng bmBc : Tính toán khoảng cách dịch chuyển ký tự xấu theo cơ chế của Boyer-Moore (bỏ qua ký tự cuối cùng của mẫu $x[m-1]$).

(2) Xây dựng bảng qsBc : Tính toán khoảng cách dịch chuyển ký tự xấu sửa đổi theo cơ chế của Quick Search (duyet đến hết ký tự cuối cùng).

5.4.3 Quá trình so khớp và tìm kiếm

Vị trí ban đầu của cửa sổ đặt tại chỉ số $j = 0$. Vòng lặp chính duy trì khi cửa sổ chưa vượt quá giới hạn văn bản ($j \leq n - m$):

(1) So khớp toàn bộ cửa sổ: tiến hành so sánh toàn bộ m ký tự của mẫu x với đoạn văn bản tương ứng trong cửa sổ hiện tại $y[j \dots j + m - 1]$. Nếu trùng khớp hoàn toàn, thuật toán ghi nhận đã tìm thấy mẫu tại vị trí j

(2) Xác định hai ký tự chốt chặn: thuật toán đồng thời xác định hai ký tự quan trọng tại biên của cửa sổ:

- Ký tự chốt trong (ký tự cuối cùng của cửa sổ): $c_{trong} = y[j + m - 1]$
- Ký tự chốt ngoài (ký tự ngay sau cửa sổ): $c_{ngoài} = y[j + m]$

(3) Dịch chuyển cửa sổ: tra cứu hai giá trị dịch chuyển tương ứng trong hai bảng tiền xử lý và chọn ra giá trị lớn nhất để tịnh tiến cửa sổ sang lượt thử mới:

$$j = j + \max(\text{bmBc}[c_{trong}], \text{qsBc}[c_{ngoài}])$$

5.4.4 Đánh giá độ phức tạp thuật toán

5.4.4.1 Độ phức tạp không gian

Thuật toán cần lưu trữ đồng thời cả hai bảng dịch chuyển một chiều:

- Mảng bmBc kích thước cố định bằng kích thước bảng chữ cái $|\Sigma|$.
- Mảng qsBc kích thước cố định bằng kích thước bảng chữ cái $|\Sigma|$.

Do đó, độ phức tạp không gian của thuật toán là: $S(n) = O(|\Sigma|)$

5.4.4.2 Độ phức tạp thời gian

(1) Giai đoạn tiền xử lý: do thực hiện tuần tự hai hàm dựng bảng độc lập, tổng thời gian tiền xử lý được chặn ở mức tuyến tính: $T_{\text{preprocessing}} = O(m + |\Sigma|)$

(2) Giai đoạn tìm kiếm:

Trường hợp trung bình: nhờ việc luôn chọn bước nhảy dài nhất từ việc so sánh $\max(\text{bmBc}, \text{qsBc})$, thuật toán giảm tối đa số lần cửa sổ phải dừng lại để kiểm tra trong thực tế. Thuật toán đạt tốc độ tuyến tính trên bảng chữ cái lớn. $T_{\text{average}} = O(n)$

Trường hợp xấu nhất: (văn bản và mẫu đều lặp lại một ký tự duy nhất), cả hai bảng đều trả về giá trị dịch chuyển tối thiểu là 1 đơn vị. Lúc này thuật toán phải thực hiện so sánh toàn bộ các ký tự trong cửa sổ tại mỗi bước với độ phức tạp thời gian là: $T_{\text{worst}} = O(m \times n)$

5.4.5 Kiểm nghiệm thuật toán

5.4.5.1 Dữ liệu vào

Cho hai xâu dưới đây. Thực hiện tìm kiếm mẫu x trong văn bản y

```
x="aaba"
y="aaabaaabaaabaa"
```

5.4.5.2 Quy trình kiểm nghiệm thuật toán

(1) Giai đoạn tiền xử lý:

Text and Pattern character															Full string
pos j	0	1	2	3	4	5	6	7	8	9	10	11	12	13	
y[j]	a	a	b	a	a	b	a	a	c	a	a	b	a	a	aabaabaacaabaa
x[j]	a	a	b	a											aaba

Chạy bảng bmBc						
Bước	i	pattern [i]	bmBc[a]	bmBc[b]	bmBc [khác]	Giải thích
0	-	-	4	4	4	
1	0	a	3	4	4	bmBc[a]=4-1-0=3
2	1	a	2	4	4	bmBc[a]=4-1-1=2
3	2	b	2	1	4	bmBc[b] = 4-1-2=1

Bảng bmBc	
chr	bmbc [chr]
a	2
b	1
khác	4

Chạy bảng qsBc						
Bước	i	pattern [i]	qsBc[a]	qsBc[b]	qsBc [khác]	Giải thích qsBc[x[i]] = m-i
0	-	-	5	5	5	
1	0	a	4	5	5	qsBc[a]=4-0=4
2	1	a	3	5	5	qsBc[a]=4-1=3
3	2	b	3	2	5	qsBc[b] = 4-2=2
3	2	b	1	2	5	qsBc[a] = 4-3=1

Bảng qsBc	
chr	qsBc [chr]
a	1
b	2
khác	5

(2) Giai đoạn tìm kiếm:

Kiểm nghiệm thuật toán									
j	y[j..j+m-1]	x==y[j..j+m-1]?	c_trong=y[j+m-1]	c_ngoài=y[j+m]	bmBc[c_trong]	qsBc[c_ngoài]	max(bmBc,qsBc)	j mới	Output
0	aaba	Khớp	a'(tại y[3])	a'(tại y[4])	bmBc['a']=2	qsBc['a']=1	max(2,1)=2	0+2=2	0
2	baab	Không khớp	b'(tại y[5])	a'(tại y[6])	bmBc['b']=1	qsBc['a']=1	max(1,1)=1	2+1=3	
3	aaba	Khớp	a'(tại y[6])	a'(tại y[7])	bmBc['a']=2	qsBc['a']=1	max(2,1)=2	3+2=5	3
5	baac	Không khớp	a'(tại y[8])	a'(tại y[9])	bmBc['a']=2	qsBc['a']=1	max(2,1)=2	5+2=7	
7	caab	Không khớp	a'(tại y[10])	a'(tại y[11])	bmBc['a']=2	qsBc['a']=1	max(2,1)=2	7+2=9	
9	aaba	Khớp	a'(tại y[12])	a'(tại y[13])	bmBc['a']=2	qsBc['a']=1	max(2,1)=2	9+2=11	9
11								j=11>(n-m=10)	

5.4.6 Cài đặt thuật toán với C++

5.4.6.1 Hàm lập bảng Boyer-Moore Bad Character (bmBc) và Quick Search Bad Character (qsBc)

```
const int ASIZE = 256;

// Tiền xử lý bảng bad character của Boyer-Moore
void preBmBc(const string &pattern, vector<int> &bmBc) {
    int m = pattern.size();

    // Gán mặc định mọi giá trị bằng độ dài pattern
    bmBc.assign(ASIZE, m);

    // Tính khoảng dịch cho từng ký tự
    for (int i = 0; i < m - 1; i++) {
        bmBc[(unsigned char)pattern[i]] = m - i - 1;
    }
}
```

```
// Tiền xử lý bằng Quick Search
void preQsBc(const string &pattern, vector<int> &qsbC) {
    int m = pattern.size();

    // Gán mặc định m + 1 cho mọi ký tự
    qsbC.assign(ASIZE, m + 1);

    // Tính khoảng dịch cho từng ký tự
    for (int i = 0; i < m; i++) {
        qsbC[(unsigned char)pattern[i]] = m - i;
    }
}
```

5.4.6.2 Hàm tìm kiếm bằng thuật toán Smith

```
// Thuật toán Smith
void SMITH(const string &pattern, const string &text) {
    int m = pattern.size();
    int n = text.size();

    vector<int> bmBc, qsBc;

    // Tiền xử lý hai bảng dịch chuyển
    preBmBc(pattern, bmBc);
    preQsBc(pattern, qsBc);

    int j = 0;

    // Bắt đầu tìm kiếm
    while (j <= n - m) {

        // So sánh pattern với đoạn hiện tại
        if (text.substr(j, m) == pattern) {
            cout << "Tim thay tai vi tri: " << j << endl;
        }

        // Nếu đã tới cuối text thì dừng
        if (j + m >= n)
            break;

        // Lấy giá trị lớn nhất giữa BM và QS
        int shift = max(
            bmBc[(unsigned char)text[j + m - 1]],
            qsBc[(unsigned char)text[j + m]]
        );

        // Dịch chuyển
        j += shift;
    }
}
```

5.5 Thuật toán Raita

5.5.1 Giới thiệu thuật toán

Thuật toán Raita được thiết kế dựa trên một triết lý tối ưu hóa hiệu năng, đó là thay đổi thứ tự so khớp các ký tự trong cửa sổ để phát hiện sai lệch càng sớm càng tốt.

Tác giả Raita nhận định rằng việc đối chiếu tuần tự từ trái sang phải hoặc từ phải sang trái đôi khi phải tốn rất nhiều phép so sánh vô nghĩa đối với các ngôn ngữ tự nhiên (như tiếng Anh), nơi mà các hậu tố và tiền tố thường xuyên lặp lại giống nhau.

Để giảm thiểu chi phí đối sánh bộ nhớ, Raita đề xuất một chiến lược kiểm tra cục bộ theo thứ tự ưu tiên ba điểm chốt: ký tự cuối cùng, ký tự đầu tiên, và ký tự ở chính giữa mẫu. Chỉ khi cả ba điểm chốt chiến lược này trùng khớp hoàn toàn, thuật toán mới kích hoạt hàm so sánh khối cho các ký tự còn lại.

Thuật toán Raita có các đặc điểm sau:

(1) Việc kiểm tra bộ ba ký tự biên và trung tâm hoạt động như một bộ lọc (filter) loại bỏ đến hơn 90% các vị trí không khả thi trước khi phải dùng đến hàm so sánh chuỗi diện rộng.

(2) Thuật toán sử dụng nguyên vẹn bảng dịch chuyển ký tự xấu $bmBc$ dựa trên ký tự cuối cùng của cửa sổ để tính toán khoảng trượt, giữ nguyên sự đơn giản và hiệu quả trong giai đoạn dịch chuyển cửa sổ.

(3) Thực nghiệm cho thấy Raita đạt tốc độ rất cao trên các tệp văn bản ngôn ngữ tự nhiên hoặc mã nguồn nhờ tận dụng tốt sự phụ thuộc ký tự và tối ưu hóa mã lệnh của các bộ biên dịch hiện đại.

5.5.2 Quy trình thực hiện thuật toán

Giả sử có chuỗi văn bản y độ dài n và chuỗi mẫu x độ dài m . Thuật toán Raita được thực hiện theo quy trình sau:

5.5.2.1 Tiền xử lý mẫu: lập bảng $bmBc$, trích xuất chốt

(1) Xây dựng bảng ký tự xấu $bmBc$: tính toán theo thuật toán Boyer Moore (bỏ qua ký tự cuối cùng $x[m-1]$).

(2) Thuật toán lưu trữ 4 giá trị đại diện của mẫu:

- Ký tự đầu tiên: $x_{đầu} = x[0]$
- Ký tự cuối cùng: $x_{cuối} = x[m-1]$
- Ký tự chính giữa: $x_{giữa} = x[\lfloor m/2 \rfloor]$

Lưu ý: con trỏ đoạn giữa: trở tới vị trí ký tự thứ hai của mẫu $(x+1)$ phục vụ cho việc đối chiếu phần lõi còn lại.

5.5.2.2 Quá trình so khớp và tìm kiếm

Vị trí ban đầu của cửa sổ đặt tại chỉ số $j = 0$. Vòng lặp chính duy trì khi cửa sổ chưa vượt quá giới hạn văn bản ($j \leq n - m$):

(1) Kiểm tra lần lượt ba điều kiện của cửa sổ hiện tại:

- Điều kiện 1: ($x_{cuối}$) phải khớp với ký tự cuối của cửa sổ văn bản ($y[j + m - 1]$).
- Điều kiện 2: ($x_{giữa}$) phải khớp với ký tự giữa của cửa sổ văn bản ($y[j + \lfloor m/2 \rfloor]$).
- Điều kiện 3: ($x_{đầu}$) phải khớp với ký tự đầu của cửa sổ văn bản ($y[j]$).

Lưu ý: Nếu bất kỳ điều kiện nào trong bộ ba này bị vi phạm, thuật toán lập tức bỏ qua Bước 2 để chuyển thẳng sang Bước 3.

(2) Nếu thỏa mãn cả 3 điều kiện, thuật toán tiến hành so sánh đoạn ký tự còn lại từ vị trí thứ hai đến vị trí sát cuối của mẫu (độ dài $m-2$ ký tự). Nếu phần lõi này khớp hoàn toàn, thuật toán ghi nhận đã tìm thấy mẫu tại j .

(3) Dịch chuyển cửa sổ: bất kể kết quả đối chiếu ở các bước trước là đúng hay sai, thuật toán lấy ký tự ở vị trí cuối cùng bên phải của cửa sổ hiện tại ($c = y[j + m - 1]$) làm khóa tra cứu bảng dịch chuyển để tính tiến cửa sổ: $j = j + \text{bmBc}[c]$

5.5.3 Đánh giá độ phức tạp thuật toán

5.5.3.1 Độ phức tạp không gian

Thuật toán Raita duy trì cấu trúc bộ nhớ tinh gọn tương đương với Boyer-Moore:

- Mảng một chiều bmBc có kích thước cố định bằng kích thước bảng chữ cái $|\Sigma|$
- Các biến ký tự đơn để lưu trữ 3 điểm chốt.

Do đó, tổng độ phức tạp không gian bổ trợ của thuật toán là: $S(n) = O(|\Sigma|)$

5.5.3.2 Độ phức tạp thời gian

(1) Giai đoạn tiền xử lý: thời gian dựng bảng ký tự xấu phụ thuộc vào kích thước bảng chữ cái và chiều dài mẫu, đạt mức tuyến tính: $T_{\text{preprocessing}} = O(m + |\Sigma|)$

(2) Giai đoạn tìm kiếm:

Trường hợp trung bình: vì có bộ lọc, hầu hết các lượt thử đều bị loại ngay từ phép so sánh ký tự đầu tiên hoặc ký tự giữa mà không tốn chi phí gọi hàm so sánh khối bộ nhớ. Kết

hợp với bước nhảy dài của thuật toán Boyer-Moore, thuật toán đạt hiệu năng tiệm cận thời gian tuyến tính $T_{average} = O(n)$.

Trường hợp xấu nhất: khi gặp phải các chuỗi ký tự có cấu trúc tuần hoàn liên tục (ví dụ văn bản và mẫu đều gồm toàn các ký tự 'a'), bộ lọc ba điểm chốt luôn đúng và thuật toán buộc phải kiểm tra toàn bộ cửa sổ tại mỗi bước với khoảng trượt tối thiểu là 1. Do vậy, độ phức tạp thời gian trường hợp tệ nhất là: $T_{worst} = O(m \times n)$

5.5.4 Kiểm nghiệm thuật toán

5.5.4.1 Dữ liệu vào

Cho hai xâu dưới đây. Thực hiện tìm kiếm mẫu x trong văn bản y

```
x="aaba"
y="aaabaaabaaabaa"
```

5.5.4.2 Quy trình kiểm nghiệm thuật toán

(1) Giai đoạn tiền xử lý

Text and Pattern character															Full string
pos j	0	1	2	3	4	5	6	7	8	9	10	11	12	13	
y[j]	a	a	b	a	a	b	a	a	c	a	a	b	a	a	aabaabaacaabaa
x[j]	a	a	b	a											aaba

Chạy bảng bmBc						
Bước	i	pattern [i]	bmBc[a]	bmBc[b]	bmBc [khác]	Giải thích
0	-	-	4	4	4	
1	0	a	3	4	4	bmBc[a]=4-1-0=3
2	1	a	2	4	4	bmBc[a]=4-1-1=2
3	2	b	2	1	4	bmBc[b] = 4-1-2=1

Bảng bmBc	
chr	bmBc [chr]
a	2
b	1
khác	4

Chốt		
Loại	x[i]	
x_đầu	x[0]	a
x_giữa	x[m//2]	b
x_cuối	x[m-1]	a

(2) Giai đoạn tìm kiếm

Kiểm nghiệm thuật toán										
j	y[j..j+m-1]	y[j+m-1]==x_cuối? (1)	y[j+m//2]==x_giữa a? (2)	y[j]==x_đầu? (3)	(1) AND (2) AND (3)	y[j+1...j+m-2] ==x[1..m-2]?	c= y[j+m-1]	bmBc[c]	j mới	Output
0	"aaba"	a' == 'a'(Đúng)	b' == 'b'(Đúng)	a' == 'a'(Đúng)	TRUE	ab==ab (true)	a'	2	0+2=2	0
2	"baab"	b' == 'a'(Sai)	Không xét	Không xét	FALSE	Không xét	b'	1	2+1=3	
3	"aaba"	a' == 'a'(Đúng)	b' == 'b'(Đúng)	a' == 'a'(Đúng)	TRUE	ab==ab (true)	a'	2	3+2=5	3
5	"baac"	c' == 'a'(Sai)	Không xét	Không xét	FALSE	Không xét	c'	4	5+4=9	
9	"aaba"	a' == 'a'(Đúng)	b' == 'b'(Đúng)	a' == 'a'(Đúng)	TRUE	ab==ab (true)	a'	2	9+2=11	9
11		j=11>(n-m=10)								

5.5.5 Cài đặt thuật toán bằng C++

5.5.5.1 Hàm tiền xử lý: lập bảng Bad Character (bmBc)

```
const int ASIZE = 256;

// Tiền xử lý bảng bad character của Boyer-Moore
void preBmBc(const string &pattern, vector<int> &bmBc) {
```

```
int m = pattern.size();

bmBc.assign(ASIZE, m);

// Tính khoảng dịch cho từng ký tự
for (int i = 0; i < m - 1; i++) bmBc[(unsigned char)pattern[i]] = m - i - 1;
}
```

5.5.5.2 Hàm tìm kiếm bằng thuật toán Raita

```
// Thuật toán Raita
void raita(const string &pattern, const string &text) {
    int m = pattern.size();
    int n = text.size();

    vector<int> bmBc;

    // Tiền xử lý bảng bad character
    preBmBc(pattern, bmBc);

    // Lấy các ký tự đặc biệt để kiểm tra nhanh
    char firstCh = pattern[0];
    char middleCh = pattern[m / 2];
    char lastCh = pattern[m - 1];

    int j = 0;

    // Bắt đầu tìm kiếm
    while (j <= n - m) {
        // Lấy ký tự cuối của cửa sổ hiện tại
        char c = text[j + m - 1];

        // Kiểm tra nhanh ký tự đầu, giữa, cuối
        if (lastCh == c &&
            middleCh == text[j + m / 2] &&
            firstCh == text[j]) {

            // So sánh phần còn lại của pattern
            if (text.substr(j + 1, m - 2) == pattern.substr(1, m - 2)) {
                cout << "Tim thay tai vi tri: " << j << endl;
            }
        }

        // Dịch chuyển theo bảng bad character
        j += bmBc[(unsigned char)c];
    }
}
```

DANH MỤC THAM KHẢO

Christian Charras, Thierry Lecroq. (2004). *Handbook of Exact String Matching Algorithms*. King's College.

Maxime Crochemore, Wojciech Rytter. (2002). *Jewels Of Stringology : Text Algorithms*. World Scientific.